Introductions

What is the point of this class?

Course Administration

Criteria	Percentage
Attendance during final project presentations	5%
Projects (10% x 3)	30%
Final project	25%
Midterm exam	15%
Final exam	25%
Total	100%

Projects

- 3 projects, all individual
- Will lose 10%/day (or fraction thereof) of maximum grade (***I'll be a stickler for this too***)
  - That is 1% of your overall final grade in the course!
- Additional details will be given when projects are assigned

Final Project

- 5% of your grade will be for showing up ***on time*** to the final project presentations
  - 1% for showing up each day, 0.5% for being on time that day
  - Only 0.5% for being on time the day you're set to present

- Final project content will be based on selecting a dataset that is of interest to you, parsing the dataset, selecting the appropriate machine learning protocol to analyze the data, then presenting to the rest of the class

Exams

- Exams will generally focus more on concepts and methods rather than details (note: “focus more” does not mean “exclusively cover”)

- Examples of the types of questions that you’re likely to encounter:
  - Which machine learning method would be most appropriate for the following data
  - Computing/problem solving either with math or with pseudocode
  - Data analysis plan, what’s wrong with it?
  - Potentially fact-based questions about major topics

Lecture 1

COM 307: Machine Learning & Data Mining

What is machine learning?

- Scientific study of algorithms and statistical models that computer systems use to perform a specific task *without using explicit instructions*

- Generally agreed to be a subfield of artificial intelligence

- Generally use “training data” to make predictions on what a certain kind of data generally “looks like” to solve problems never encountered before

What is learning?

- Learning is generally a process of *generalization*

What is 1598274+4987?

How do you know? Have you added these two numbers before?

You know how to do this because you *learned* addition, so you can solve addition problems, even those that you've never encountered before!

What is data mining?

- Process of discovering patterns in large data sets, particularly at the intersections of machine learning, statistics, and database systems

- Interdisciplinary field of computer science and statistics with the goal of extracting information and determining structure of the data

Machine learning vs data mining

- They are very closely related, which is why they’re being covered together in this course

- Machine learning approaches work on 2 sets of data:
  - Training data used to determine variables in a data set
  - Test data, the stuff you actually care about
- Data mining focuses on the discovery of unknown properties of data

Machine learning vs data mining

- They are very closely related, which is why they’re being covered together in this course

- Machine learning tools are evaluated based on their ability to reproduce known knowledge effectively
- Data mining is focused on discovery of previously unknown knowledge

Data mining vs data analysis

- Data analysis is used to test models on a data set
  - Example: Analyze the effectiveness of a marketing campaign, regardless-ish of how much data you have

- Data mining finds hidden patterns in large volumes of data – don’t necessarily need to know what you’re looking for

Some examples—machine learning

- Email filtering (spam or not spam)
- Computer vision
  - Self-driving cars
  - Surveillance
  - Manufacturing/QC
  - …
- Agriculture
- Handwriting recognition
- Bioinformatics

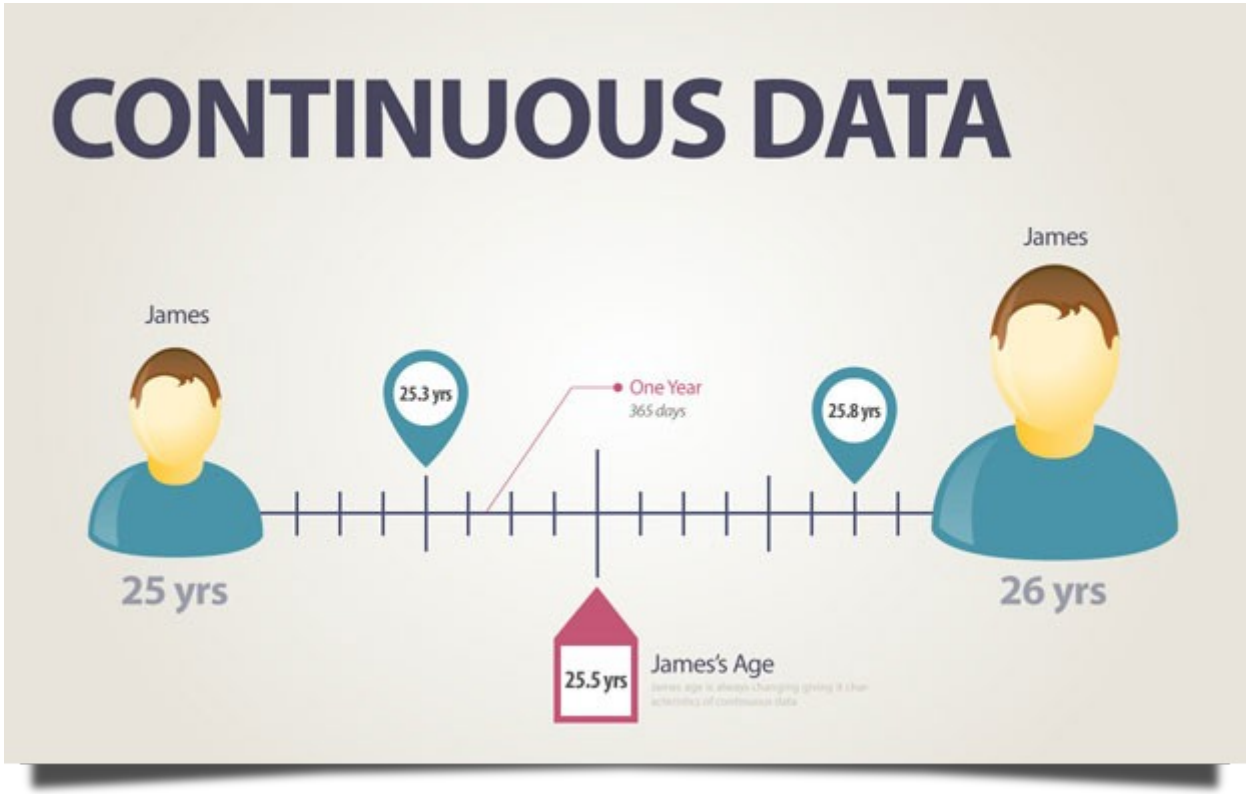
Some examples—data mining

- Business
  - Walmart processes 20 million sales/day, who, what, and where?
  - Credit cards & store loyalty cards
  - Determining employee success
- Human rights, particularly related to the justice system
- Medical data mining (“genes” & diseases)
- Educational research
  - Student success
  - Student retention

Lecture 2: Problems in Machine Learning

Is your data???

- Continuous or discrete?



How many days this week did you read the newspaper?

Number of days read	Frequency
0	44
1	24
2	18
3	16
4	20
5	22
6	26
7	30
Total	**200**

Is your data???

- Continuous or discrete?

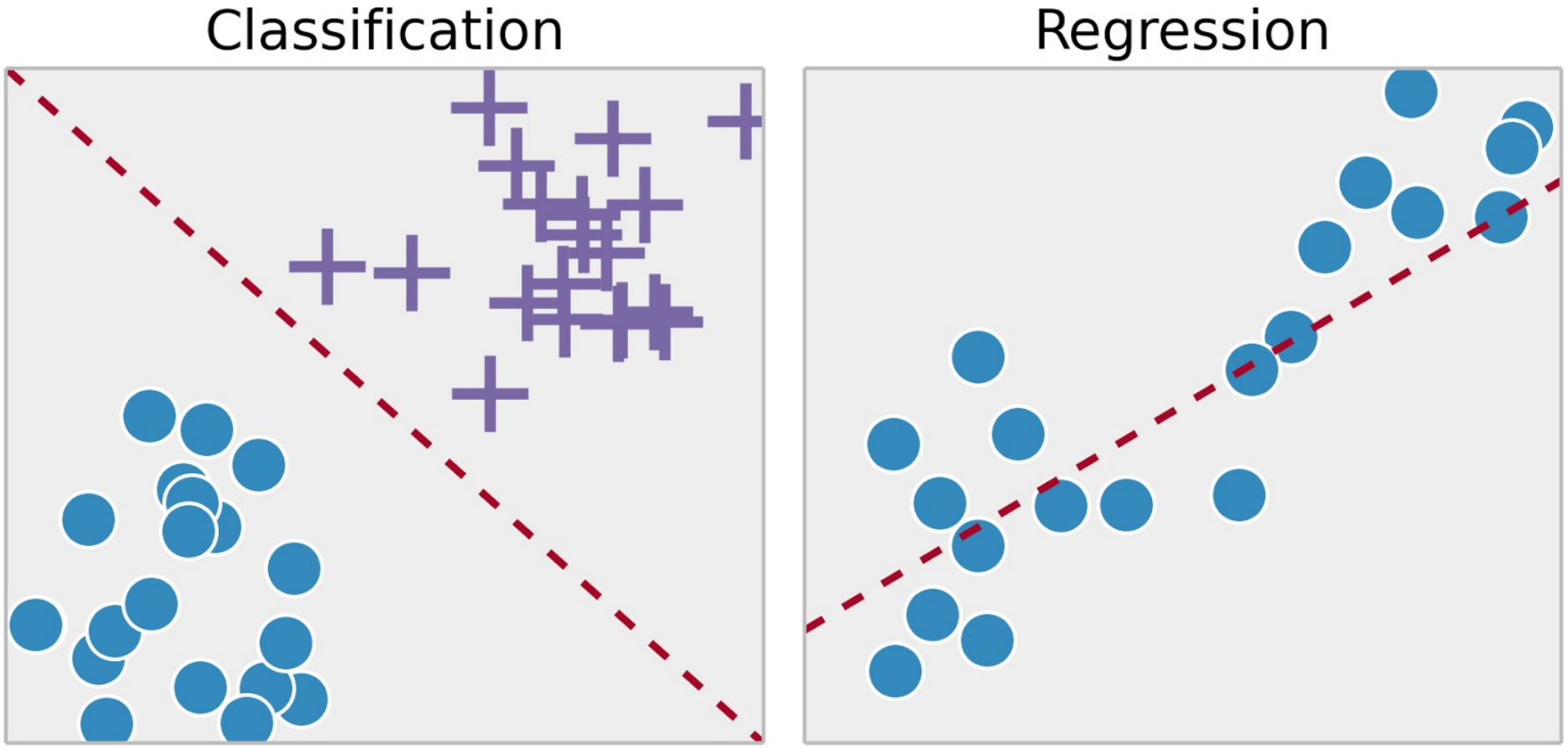
Differences between Continuous and Discrete Data	
Continuous	**Discrete**
Average Daily temperature	Days with temperature > 45 degree
No of minutes to board the plane	# of late departures due to late boarding
Number of litters of milk in the tank	Whether the tank is filled or not filled
Distance between the mile stones on a road	No of mile stones on a road

Is your data???

- Continuous or discrete – could you decide?

Is your data???

- Based on classification?



Is your data???

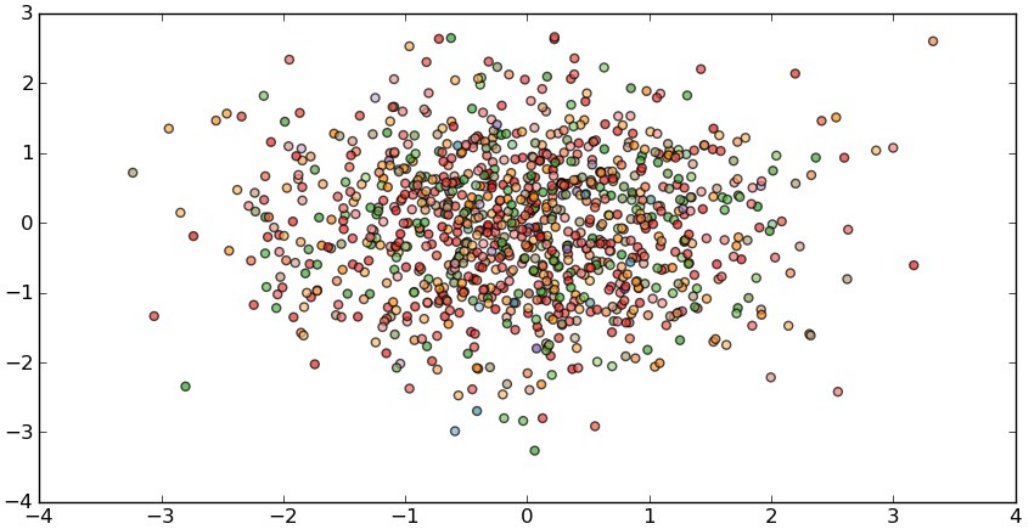
- Based on classification?
  - Spam vs not spam
  - Insure vs don’t insure

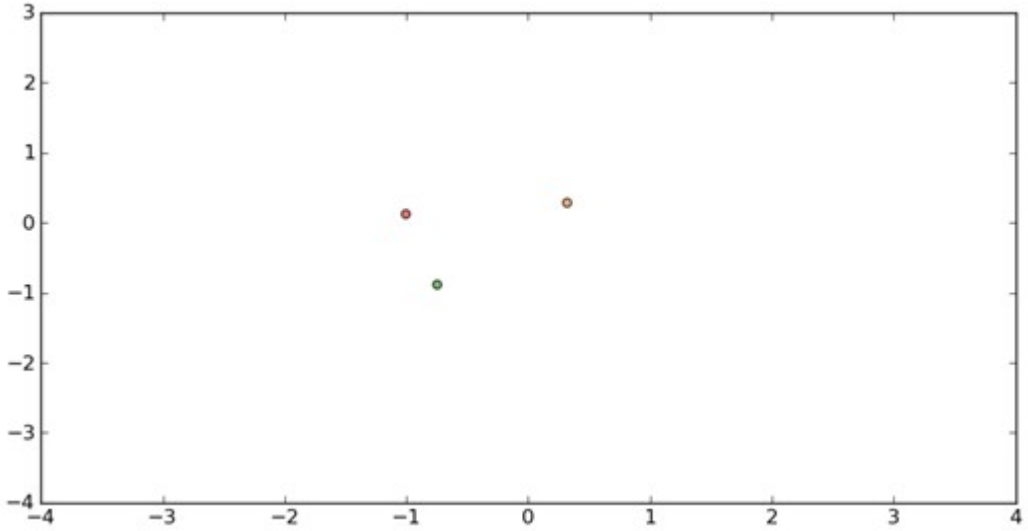
vs

  - Spam vs important vs “general”
  - Don’t insure, low rate, average rate, high rate

Is your data???

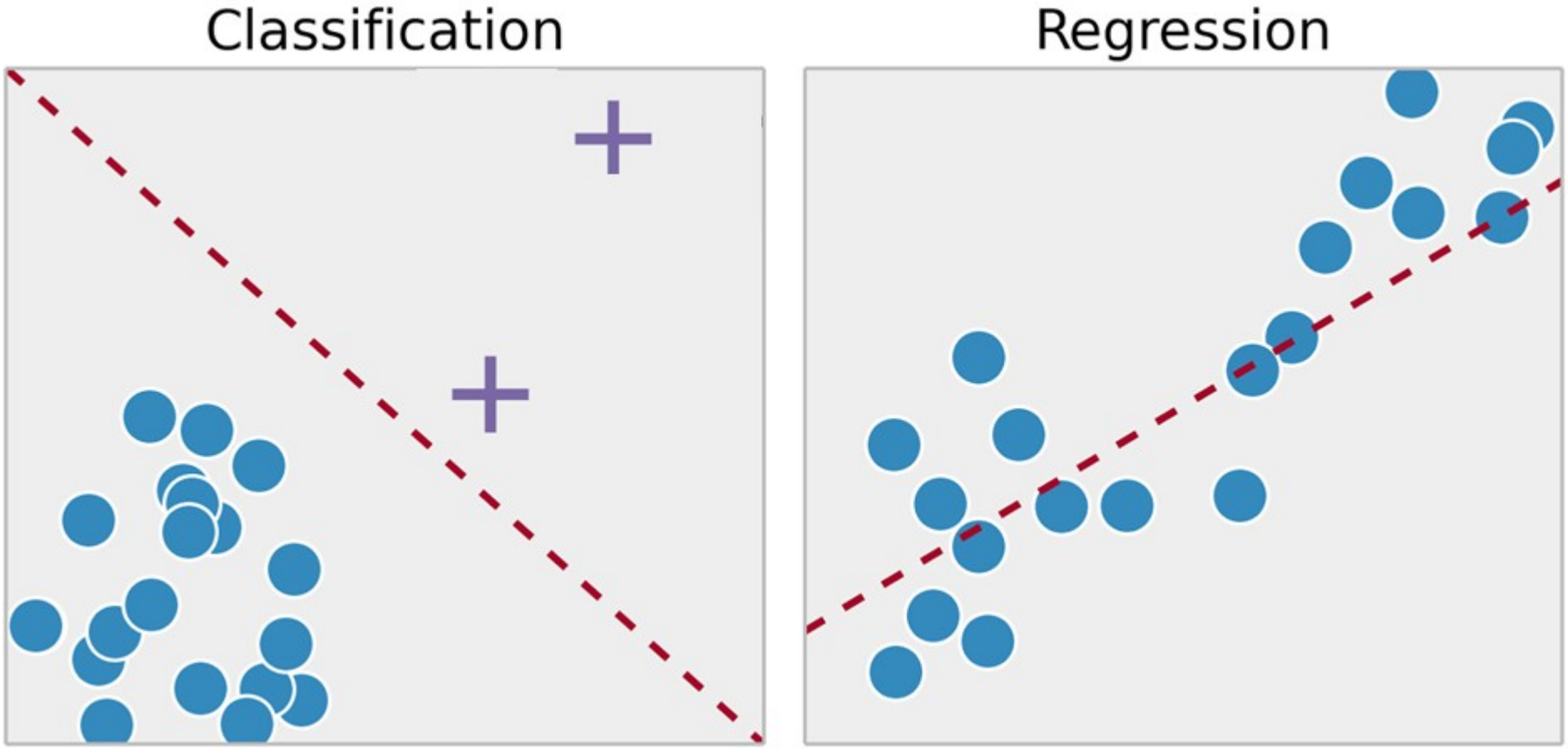
- Abundant vs rare





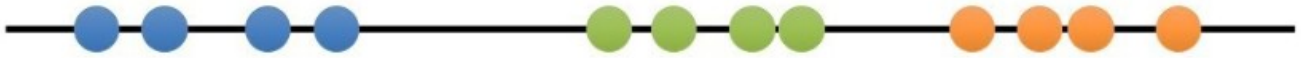
Is your data???

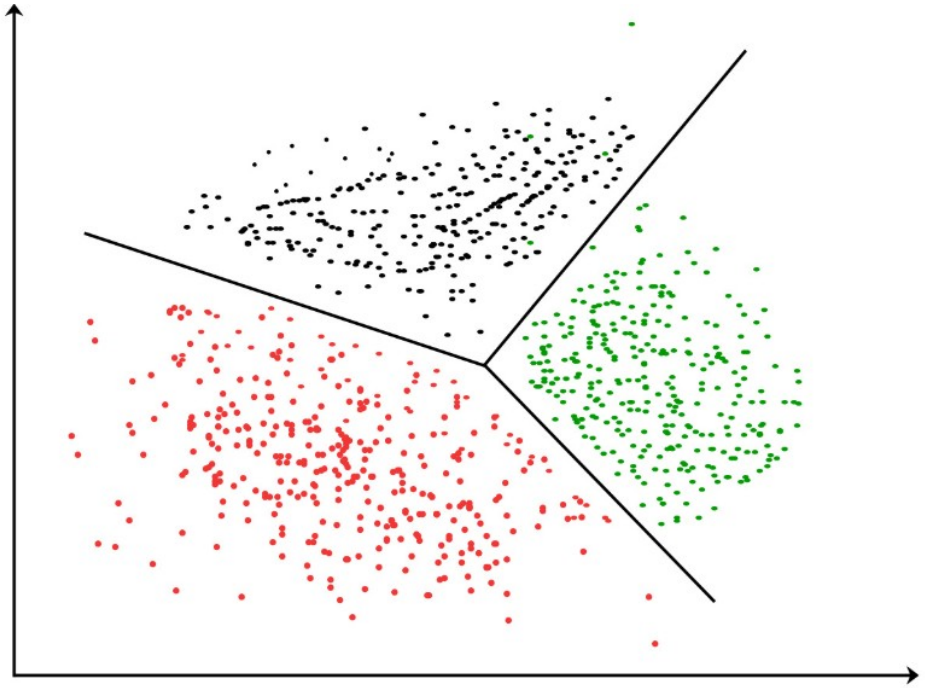
- Abundant vs rare – or does it depend on which kind of data?

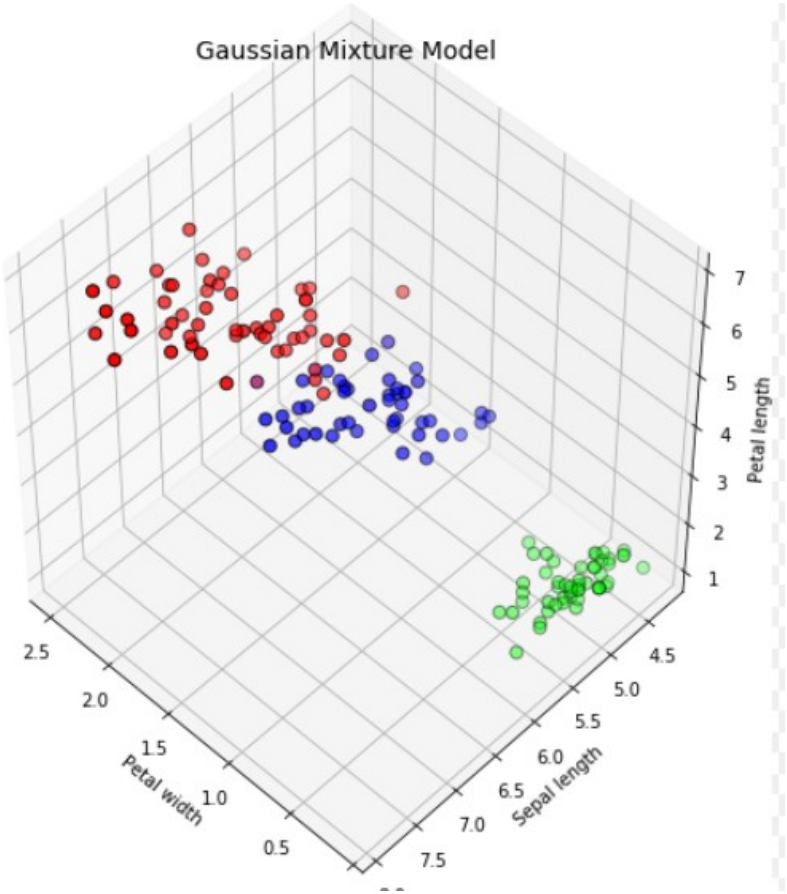


Is your data???

- Single- or multi-variable

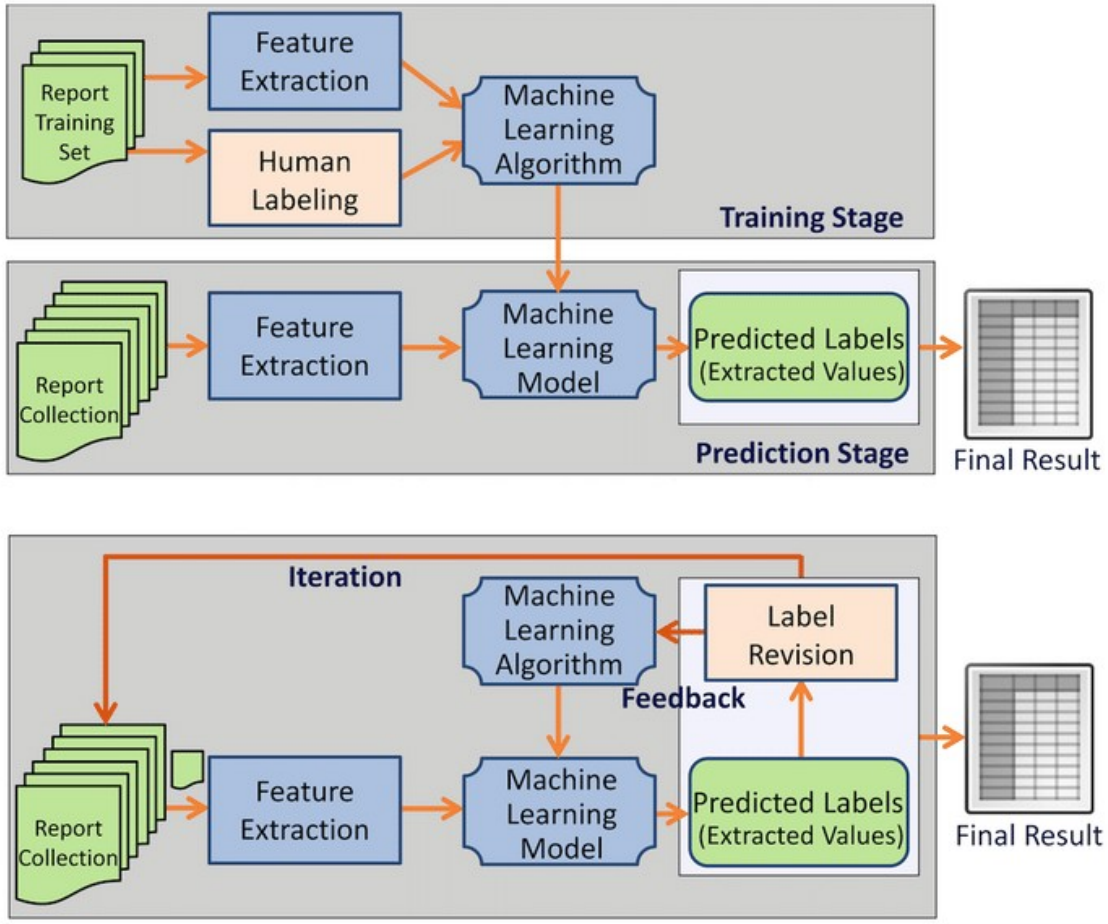






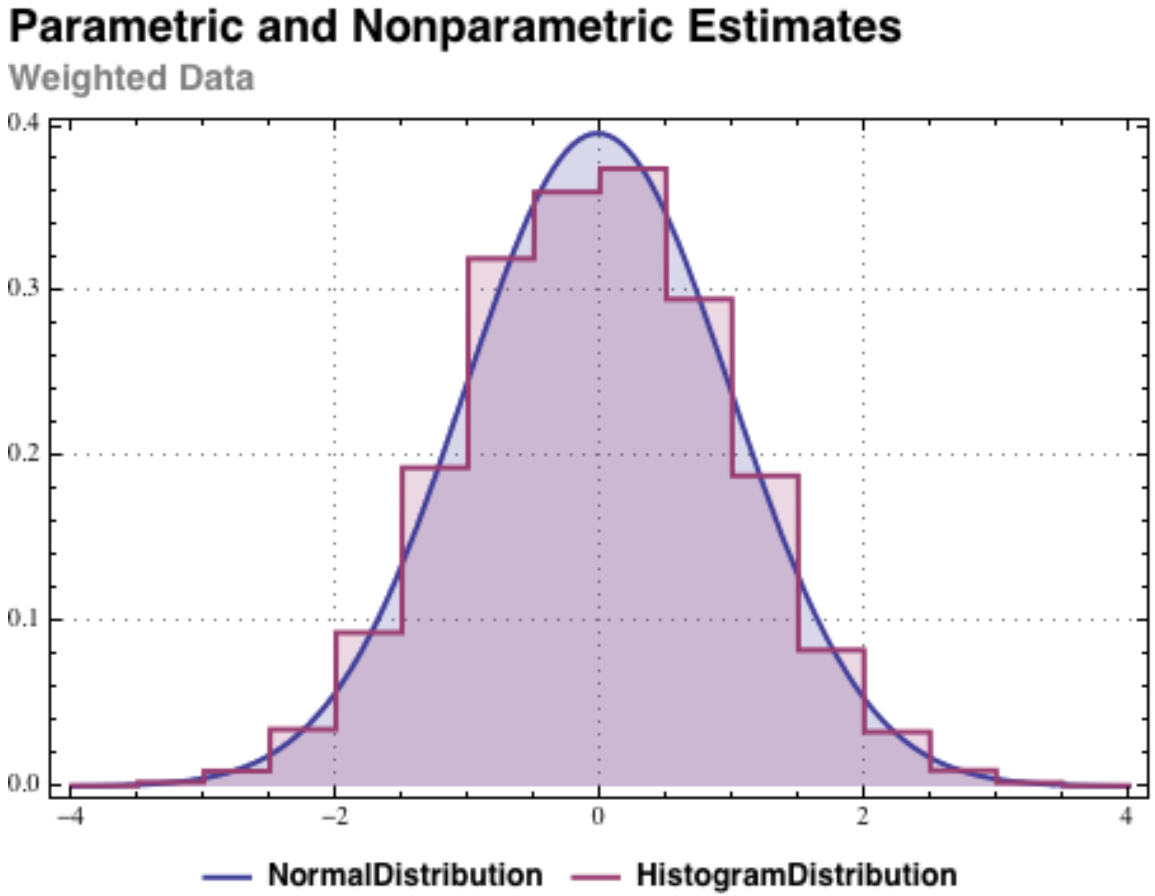
Is your data???

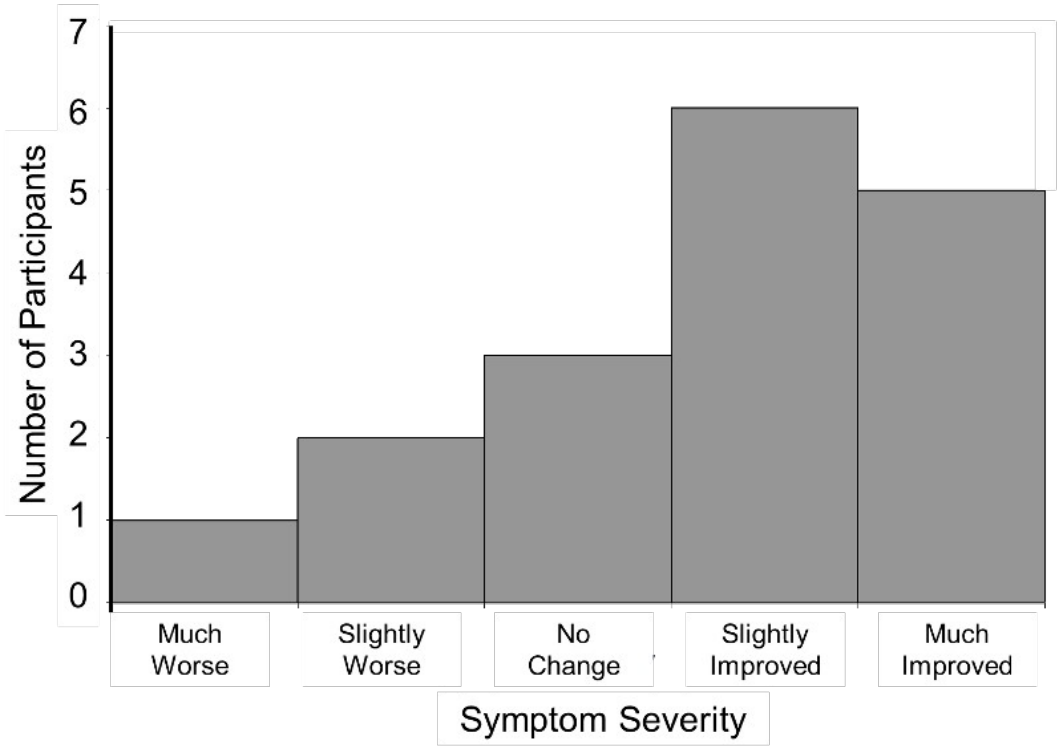
- Available all at once or given piece-by-piece



Is your data???

- Based upon an underlying statistical model?





Is your data???

- Based upon an underlying statistical model?

Is your data???

- You can (and will have to) combine all of these things (and more) just to *describe* your data

- Your choice of machine learning tool/algorithm will depend on all of these factors

- This diversity of data drives the diversity of solutions that exist to make sense of data

Lecture 3: Selecting a Solution

Remember: last time

- Data are diverse
  - Continuous vs discrete
  - Classification vs regression
  - Abundant vs rare (or asymmetric)
  - Multivariable
  - Available at once vs flowing
  - “Easily” modeled by statistical distributions

This time

- Machine learning solutions are based on these (and other) factors:
  - Eager vs lazy
  - Batch vs online
  - Parametric vs nonparametric
  - Discriminative vs generative

Lazy learners

- Instead of evaluating the training data, simply store it when it’s received

- When test data appear, classification happens using the subset of the training data that is most applicable to the test data

Eager learners

- Construct a model for classification when the training data is given, before seeing the test data

- Must be able to commit to a single hypothesis that can cover all data before seeing “all data” (the test data)

Eager learners

- Construct a model for classification when the training data is given, before seeing the test data
- Must be able to commit to a single hypothesis that can cover all data before seeing “all data” (the test data)
- Eager learners often take a long time to train and less time to predict

Batch (offline) learning

- “Standard” relative to online learning
  - Has been done for a much longer period of time
  - Is what most people would come up with if you described machine learning
  - Other X vs Y’s don’t have a “standard”

- Get a (training) dataset, build model with the dataset (which can be done in an eager or lazy way), then apply that model to another (test) dataset

Online learning

- Online learning processes data one-at-a-time, so you technically don’t need to store data (although you generally do)
- Online learning is adaptable because you don’t (can’t) make assumptions about the underlying distribution of the data
- Can react to changes in the data (for example, if consumers are responding to an event in the world)

When to use online learning

- Your data doesn’t fit into memory
- You expect your data to change over time (although there are ways to deal with this in batch learning, potentially obviously)
- Your data is a function of time and needs to be updated constantly (for example, stock prices)

Parametric learning

- Stays firm with the number of parameters needed to explain the data
  - No amount of training data will change the number of parameters
  - The training data *WILL* inform the coefficients of these parameters

$$b0 + b1*x1 + b2*x2 = 0$$

b0, b1, and b2 are coefficients that control the intercept and slope, x1 and x2 are variables

Parametric learning

- Can be bad or *really bad*
  - The previous example was modeling data as linear, but it may require a minor transformation to work out correctly
  - Alternatively, it could be modeling extremely non-linear data, which will likely cause the model to fail spectacularly

- Benefits of parametric learning:
  - Simple. Easy to understand and interpret
  - Fast. Can typically learn from data *very* quickly
  - Not data hungry. Can often learn quickly from little or incomplete data

Parametric learning

- Can be bad or *really bad*
  - The previous example was modeling data as linear, but it may require a minor transformation to work out correctly
  - Alternatively, it could be modeling extremely non-linear data, which will likely cause the model to fail spectacularly

- Limitations of parametric learning:
  - Constrained. By choosing a form, you're held fast to that form
  - Complexity. Only typically useful for low complexity problems
  - Poor fit. In practice, data rarely fits very well to underlying model

Nonparametric learning

- *Do not* make strong assumptions about the form of the mapping function

- Seek the best fit of the training data while maintaining the ability to generalize in case of unseen data

- “Nonparametric methods are good when you have a lot of data and no prior knowledge, and when you don’t want to worry too much about choosing just the right features.” (Artificial Intelligence: A Modern Approach, page 757)

Nonparametric learning

- Benefits:
  - Flexible
  - Power (no assumptions make for powerful methods)
  - Performance (can “fit” very well)

- Drawbacks:
  - Require a good amount of data
  - Slow (both per unit data and because they require more data)
  - Overfitting (we’ll talk about this more later, but nonparametric methods are more susceptible to overfitting, which is bad)

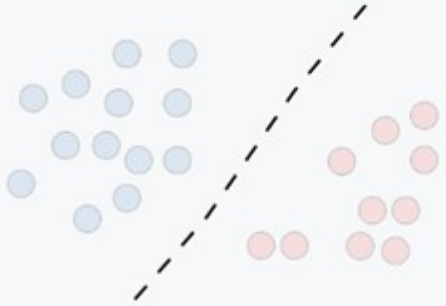
Discriminative vs generative learning -- analogies

- You want to figure out what language someone is speaking:
  - Generative – learn every language, determine which language the person is speaking by figuring out which one it belongs to
  - Discriminative – determine linguistic differences without learning any languages

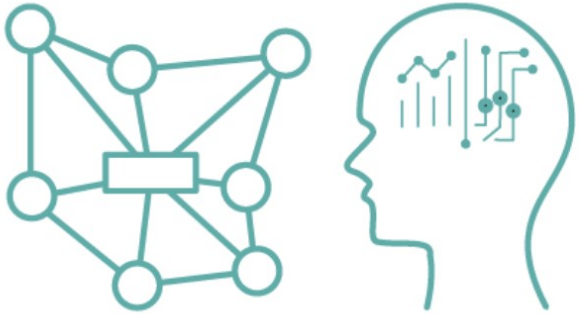
- 2 kids: 1 (“kid A”) learns by memorizing everything in depth, the other (“kid B”) learns the differences between what is seen. Both go to the zoo
  - “A” identifies a photo of a lion based on the **closest match** to the zoo animals
  - “B” identifies the same photo as a lion based on observed **characteristics** of lions

Discriminative vs generative learning

- To put it another way,
  - You can put together a model for what, say lions and tigers, look like, then you create a boundary that separates them. You can decide whether the current example falls in with the lions of the tigers (discriminative)
  - You could also build a model for tigers, build a model for lions, then compare the current “test data” point against both the tiger model and lion model and find which it behaves more like (generative)

	Discriminative model	Generative model
Goal	Directly estimate $P(y\|x)$	Estimate $P(x\|y)$ to then deduce $P(y\|x)$
What's learned	Decision boundary	Probability distributions of the data
Illustration		

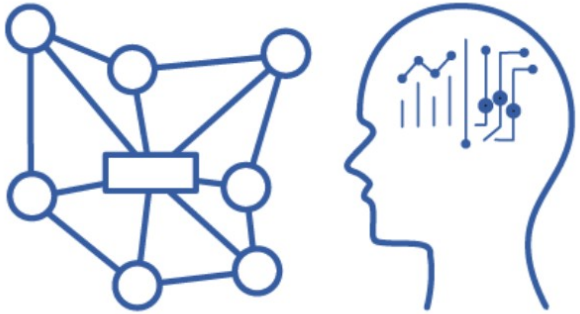
Additional consideration



Supervised

Classification

Regression



Unsupervised

Clustering

Collaborative Filtering

Frequent Pattern Mining

Lecture 4: Maximum Likelihood Estimation (MLE)

Intro to statistics notation

- p(event) is the probability of that event
  - p(fair die lands on 1) = 1/6
  - p(fair coin lands on heads) = 1/2
  - p(two fair coins both land on heads) = 1/4
- p(event|fact) is the probability of the event ***given*** the fact
  - p(2 fair dice equal 12|first die is 1) = 0
  - p(fair coin lands on heads|it landed on heads last time) = 1/2
  - p(two fair coins both land on heads|the first coin landed on heads) = ½
- E(event) is the *expected value* of that event

Experiment

- You buy a magical coin and want to know the probability of heads (or tails, for that matter) in the hopes that you can win coin flips more than 50% of the time. How do you figure it out?
  - Flip it a few times:

    Heads, heads, tails, heads, tails
  - What is the probability that a given flip will be heads?

    Well, 3/5 or 60% based on what we've observed
  - Well, why is this?

Experiment - data

- Sequence of events, D = (HHTHT…), k heads out of n flips
- Flips are *independent* and *identically distributed* (i.i.d.)
  - This means, $p(HHTHT) = p(H) * p(H) * p(T) * p(H) * p(T)$
- Hypothesis is that $p(H) = \theta$, therefore $p(T) = 1-\theta$
  - $p(HHTHT|\theta) = \theta^k * (1-\theta)^{n-k}$
- Generally, $p(D|\theta) = \theta^k(1-\theta)^{n-k}$

Experiment – **M**aximum **L**ikelihood **E**stimation (MLE)

- Maximum likelihood estimation: select θ such that it *maximizes* the probability (*likelihood*) of your observed data
  - Bad example: Say probability of heads, p(H), or θ is 1
  - This means that the probability of tails, p(T), or θ-1 is 0
  - So p(HHTHT) would be 0
  - Obviously, the probability of your *observed* data shouldn't be 0%…

Experiment – **M**aximum **L**ikelihood **E**stimation (MLE)

- Maximum likelihood estimation: select θ such that it *maximizes* the probability (*likelihood*) of your observed data
  - θ_{MLE} = arg max(θ) [p(D|θ)]
  - θ_{MLE} = arg max(θ) = [log[p(D|θ)]]
  - θ_{MLE} = arg max(θ) = [log[$\theta^k(1-\theta)^{n-k}$]]

- Set the derivative to zero:

$$\frac{d}{d\theta} \log P(\mathcal{D}|\theta) = 0$$

Experiment – **M**aximum **L**ikelihood **E**stimation (MLE)

- Maximum likelihood estimation: select θ such that it *maximizes* the probability (*likelihood*) of your observed data

  $d/d\theta[\log[p(D|\theta)] = 0$

  $d/d\theta[k*\log\theta + (n-k)*\log(1-\theta)] = 0$

  $k/\theta - (n-k)/(1-\theta) = 0$

  $k/\theta = (n-k)/(1-\theta)$

  $k*(1-\theta) = \theta*(n-k)$

  $k*(1-\theta) - \theta*(n-k) = 0$

  $k - \theta n = 0$

  $\theta_{MLE} = k/n$

Experiments – **M**aximum **L**ikelihood **E**stimation (MLE)

- $\theta_{MLE} = k/n$
- Two experiments:
  - Flip a coin 5 times, get 3 heads
  - Flip a coin 50 times, get 20 heads
- Results:
  - First experiment, $\theta_{MLE} = k/n = 3/5$ or 60%
  - Second experiment, $\theta_{MLE} = k/n = 20/50$ or 40%

- Which is right? Why?

MLE – more is better, but how much is needed?

- $\theta_{MLE} = k/n$
- If you go to infinity (flip the coin infinite times):

  $E(\theta_{MLE})$ = actual, real θ, which we'll call θ^*

- Hoeffding's inequality says that for an epsilon, $\varepsilon > 0$:

  $p(abs[\theta_{MLE} - \theta^*] \geq \varepsilon) \leq 2e^{-2n\varepsilon^*\varepsilon}$

  where n is the number of trials (coin flips in this example)

  As n increases, the right hand side gets very small (close to zero) and the probability that you're close to the true θ, θ^*, gets very close to one

Example with θ^*

- PAC (Probability Approximate Correct)
- Example: I want to know θ^* within $\varepsilon = 0.1$, with probability of at least $1-\delta = 0.95$ (probability within at least 95%). How many coin flips are needed for this?

$2e^{-2n\varepsilon^*\varepsilon} = \delta$

$\varepsilon = \text{sqrt}(\log(2/\delta)/2n)$

$\text{abs}(\theta_{MLE} - \theta^*) \leq \text{sqrt}(\log(2/\delta)/2n) = 0.1$

$\log(2/\delta)/2n = 0.01$

$\log(2/\delta) = 0.02n$

$n = \log(2/\delta)/0.02$

What about for continuous variables?

- What if you don’t have “heads” and “tails,” but 0.1234, 198.286, etc.?
- Gaussian model

$$P(x \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$

Properties of Gaussians

- Affine transformation:
  - If $X \sim N(\mu, \sigma^2)$
  - $Y = aX + b \rightarrow Y \sim N(a\mu + b, a^2\sigma^2)$

- Sum of Gaussians:
  - $X \sim N(\mu_x, \sigma_x^2)$
  - $Y \sim N(\mu_y, \sigma_y^2)$
  - $Z = X + Y \rightarrow Z \sim N(\mu_x + \mu_y, \sigma_x^2 + \sigma_y^2)$

Lecture 5: Bayesian Inference & Regression

Recall our coin last time

- It was allegedly magic, and we didn't know if it would produce heads at a frequency different from a standard, fair coin

- We used MLE to estimate the true probability, θ^*, of it producing heads on any given flip
  - The chances of it producing tails is $1-\theta^*$
  - The observed frequency/estimated probability of heads is θ
  - The number of heads was represented as k, the number of flips, n
  - Therefore, there are n-k tails

Our “magic” coin – prior probabilities

- A prior probability is a probability assessed *before* you make any relevant observations

- Sounds useless from the definition, but is *fundamentally important* for many, *many* applications
  - Probability of a person at Connecticut College being female?
  - Probability that a coin from my pocket lands on heads when I flip it?
  - Probability that a piece of DNA codes for a gene?
  - Note that you can make observations that may make you change your mind about each of these

An (important) aside: Bayes rule

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

An (important) aside: Bayes rule

- Example: What is the probability that someone develops lung cancer if they're a smoker?
  - Re-written more statistically: what is the probability that someone develops lung cancer *given* they're a smoker
  - p(lung cancer|smoker) = ?
  - According to Bayes:
    p(lung cancer|smoker) = [p(smoker|lung cancer)*p(lung cancer)] / p(smoker)

  - Note that our prior probability, p(lung cancer) might be what we start with, but we're updating this now that we have additional observations (that this person is a smoker)

An (important) aside: Bayes rule

- Example: What is the probability that someone develops lung cancer given they're a smoker?
  - p(lung cancer|smoker) = [p(smoker|lung cancer)*p(lung cancer)] / p(smoker)
    - p(lung cancer) = 0.06
    - p(smoker) = 0.14
  - Note that it is often fairly easy to get these two probabilities, therefore Bayes rule is useful whenever p(B|A) is easier to figure out than p(A|B)
    - p(smoker|lung cancer) = 0.9

p(lung cancer|smoker) = (0.9 * 0.06) / 0.14

p(lung cancer|smoker) = 0.39 or 39%

Taking the smoking example a bit further

- Example: What is the probability that someone develops lung cancer given they're a smoker?
  - p(lung cancer|**non-**smoker) = [p(**non**-smoker|lung cancer)*p(lung cancer)] / p(**non**-smoker)
    - p(lung cancer) = 0.06
    - p(**non**-smoker) = 0.86
  - Note that it is often fairly easy to get these two probabilities, therefore Bayes rule is useful whenever p(B|A) is easier to figure out than p(A|B)
    - p(**non**-smoker|lung cancer) = 0.1

p(lung cancer|**non-**smoker) = (0.1 * 0.06) / 0.86

p(lung cancer|**non-**smoker) = 0.007 or 0.7%

Our “magic” coin – prior probabilities

- Back to our coin, let’s say a friend insists that the coin isn’t magic and that it’s just a regular coin
  - The *prior probability* will be quite low (say, 10%), as your friend thinks p(heads) is 0.5
  - So, $p(\theta = 0.5) = 0.1$, but $p(\theta = 0.5|D)$ very much *does not*
    - Imagine that your D is HHHHHHHTHHHHHHHHHH, is $p(\theta = 0.5|D)$ 0.5?

$$P(\theta \mid \mathcal{D}) = \frac{P(\mathcal{D} \mid \theta)P(\theta)}{P(\mathcal{D})}$$

Our “magic” coin – p(θ|D)

- What should our hypothesis of p(H) be given our (training) data?
- Well, how about the *Maximum a posteriori* (MAP) probability, which is the mode of the posterior distribution (or the most likely single value)

Our “magic” coin – p(θ|D)

- What should our hypothesis of p(H) be given our (training) data?
- Well, how about the *Maximum a posteriori* (MAP) probability, which is the mode of the posterior distribution (or the most likely single value)
  - That is, select your θ such that p(θ|D) is the highest possible value
  - Similar in concept to MLE, different in that you’re constrained by *prior knowledge* (represented as the prior probability)
  - Note that as your observed knowledge approaches infinity, your confidence in the prior probability goes to zero (the prior is useless if you’re given the answer)
  - This means that Bayes is most useful when you have a good prior and/or a small amount of training data

Our “magic” coin – using numbers

- Say we have the observations HHTHT

$$P(\theta \mid \mathcal{D}) = \frac{P(\mathcal{D} \mid \theta) P(\theta)}{P(\mathcal{D})}$$

- Select θ to be 0.1 (because, why not?)

p(θ|HHTHT) = [p(HHTHT|θ) * p(θ)] / p(HHTHT)

p(θ|HHTHT) α [[0.1*0.1*0.9*0.1*0.9] * ???]

p(θ|HHTHT) α 0.00081

Our “magic” coin – using numbers

- Say we have the observations HHTHT

$$P(\theta \mid \mathcal{D}) = \frac{P(\mathcal{D} \mid \theta) P(\theta)}{P(\mathcal{D})}$$

- Select θ to be 0.6 (because, that’s probably better)

p(θ|HHTHT) α [p(HHTHT|θ) * p(θ)]

p(θ|HHTHT) α [[0.6*0.6*0.4*0.6*0.4] * ???]

p(θ|HHTHT) α 0.03456 * ???

Our “magic” coin – using numbers

- Say we have the observations HHTHT

$$P(\theta \mid \mathcal{D}) = \frac{P(\mathcal{D} \mid \theta) P(\theta)}{P(\mathcal{D})}$$

- Select θ to be 0.5 (to finish off)

p(θ|HHTHT) α [p(HHTHT|θ) * p(θ)]

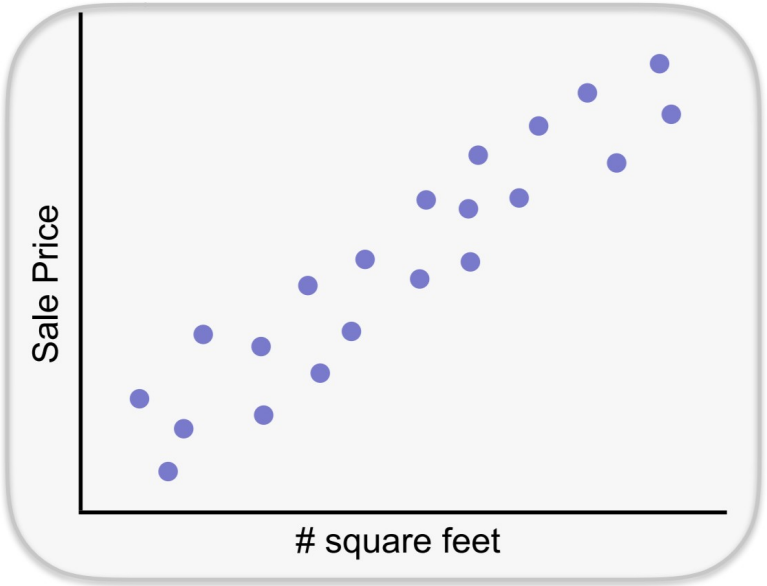
p(θ|HHTHT) α [[0.5*0.5*0.5*0.5*0.5] * ???]

p(θ|HHTHT) α 0.03125 * ???

Lecture 5: Bayesian Inference & Regression

An example regression problem

- Zillow.com – a website that tracks houses for sale & rent
- Given the sales data available, predict what a house will sell for
- y = house sales based on…
- X = square footage, zip code, date of sale, etc., etc.



Training Data:

$$\{(x_i, y_i)\}_{i=1}^{n}$$

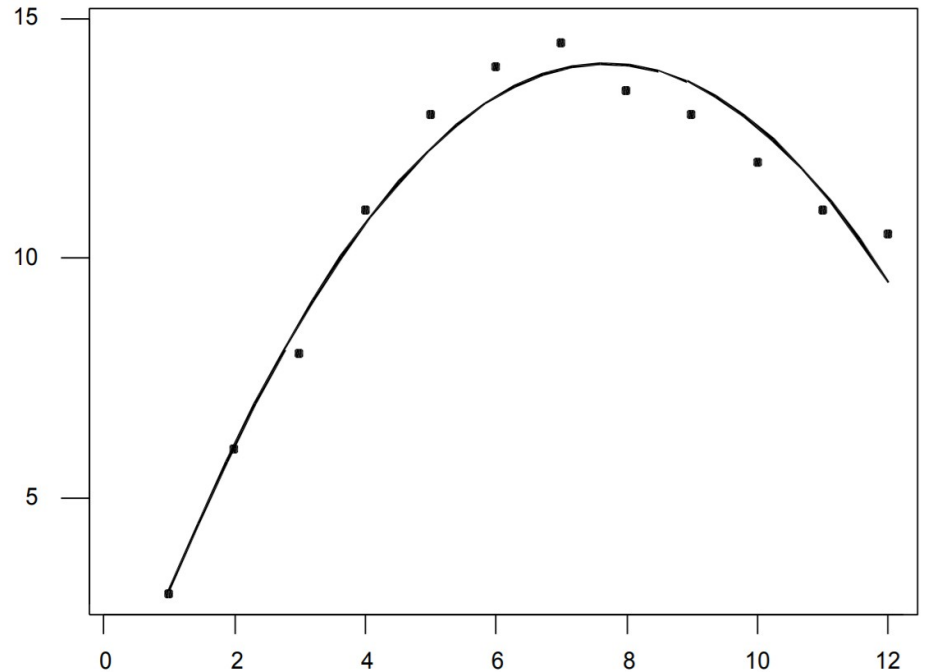
Basis of regression

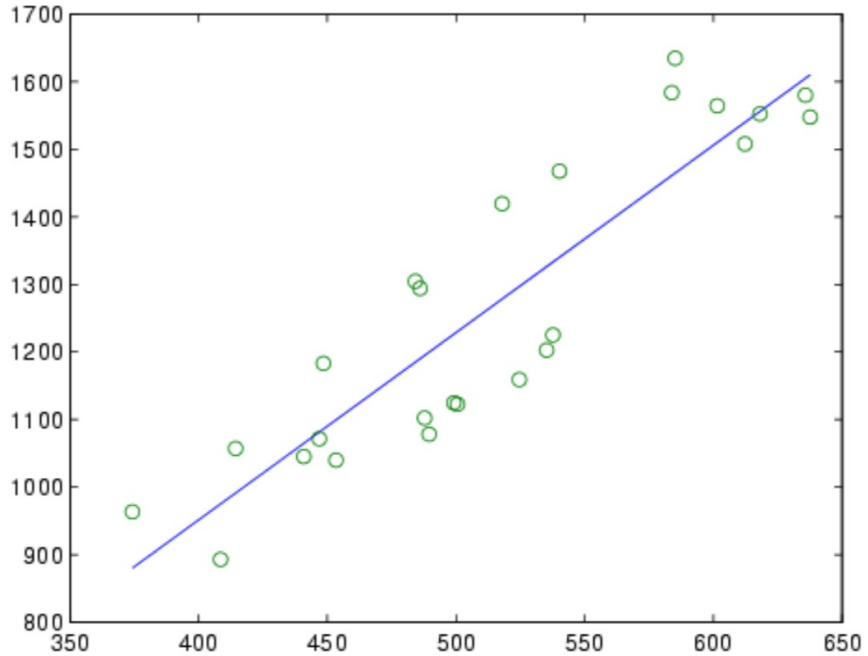
- Useful when you have variables that are related in a ***nondeterministic*** manner (ex: house price and square footage)
- We observe a dependent variable (home price)
- For each dependent variable, we observe predictors (square footage)
- Our goal is to determine the relationship between the dependent variables and predictors

- Note: Dependent variables sometimes called “response(s),” predictors sometimes called “regressor(s)”

Aside: doesn't need to be *linear* regression

- Functions can be (and often are) non-linear
- In this course we will focus on linear regression (other than the next slide) since we're not going to be spending a lot of time on regression in general





Aside: doesn't need to be *linear* regression

- Simple linear regression:

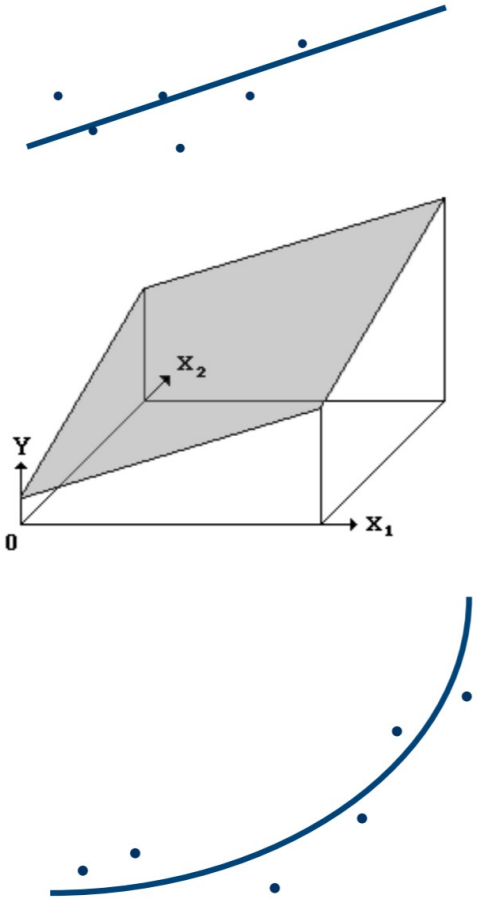
  $Y = B_0 + B_1X + \text{error}$

- Multiple linear regression:

  $Y = B_0 + B_1X_1 + B_2X_2 + \text{error}$

- Polynomial regression:

  $Y = B_0 + B_1X + B_2X^2 + \text{error}$



Linear regression

- Our goal is to find B_0 and B_1 where

  $Y = B_0 + B_1X + \text{error}$

- So we need to find:
  - The intercept, B_0
  - The slope of the line, B_1
- Basic idea (which we'll complicate in a minute):
  - Plot the observations (like a scatter plot)
  - Find the line that best "fits" or follows the plotted points

Estimating regression parameters

- To estimate B_0 and B_1, you want to find the values that minimize the error (specifically, the squared error)
- For observations 1, 2, 3, … i,-1 i, i+1, …, n, minimize:

$$\sum_{i=1}^{n} \left(y_i - (\beta_0 + \beta_1 x_i) \right)^2$$

Least squares (minimizing squared error)

- Minimize the squared error:

$$L = \sum_{i=1}^{n} \epsilon_i^2 = \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2$$

The least squares estimators of β_0 and β_1, say, $\hat{\beta}_0$ and $\hat{\beta}_1$, must satisfy

$$\left.\frac{\partial L}{\partial \beta_0}\right|_{\hat{\beta}_0,\hat{\beta}_1} = -2\sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0$$

$$\left.\frac{\partial L}{\partial \beta_1}\right|_{\hat{\beta}_0,\hat{\beta}_1} = -2\sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) x_i = 0$$

Least squares (minimizing squared error)

The **least squares estimates** of the intercept and slope in the simple linear regression model are

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \tag{11-7}$$

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n} y_i x_i - \frac{\left(\sum_{i=1}^{n} y_i\right)\left(\sum_{i=1}^{n} x_i\right)}{n}}{\sum_{i=1}^{n} x_i^2 - \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n}} \tag{11-8}$$

where $\bar{y} = (1/n)\sum_{i=1}^{n} y_i$ and $\bar{x} = (1/n)\sum_{i=1}^{n} x_i$.

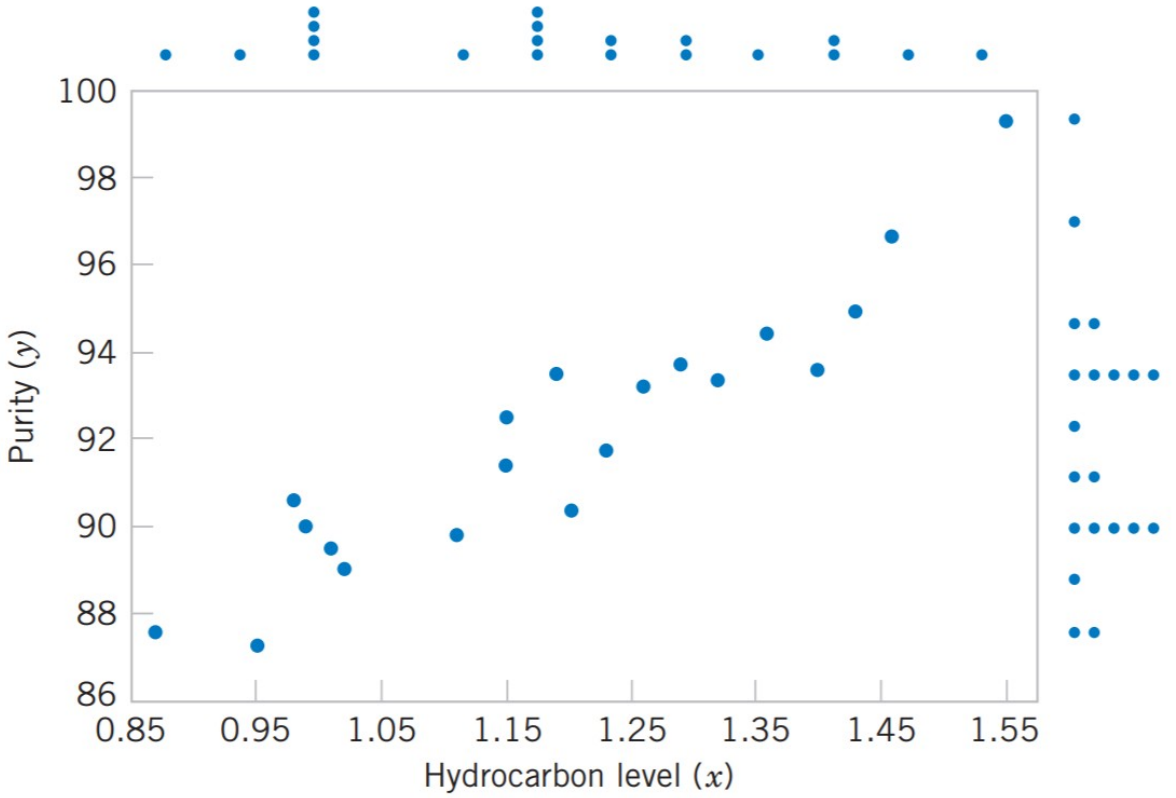
A notation that you might see…

$$S_{xx} = \sum_{i=1}^{n} (x_i - \bar{x})^2 = \sum_{i=1}^{n} x_i^2 - \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n}$$

$$S_{xy} = \sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x}) = \sum_{i=1}^{n} x_i y_i - \frac{\left(\sum_{i=1}^{n} x_i\right)\left(\sum_{i=1}^{n} y_i\right)}{n}$$

Example: oxygen and hydrocarbon levels

Observation Number	Hydrocarbon Level x (%)	Purity y (%)
1	0.99	90.01
2	1.02	89.05
3	1.15	91.43
4	1.29	93.74
5	1.46	96.73
6	1.36	94.45
7	0.87	87.59
8	1.23	91.77
9	1.55	99.42
10	1.40	93.65
11	1.19	93.54
12	1.15	92.52
13	0.98	90.56
14	1.01	89.54
15	1.11	89.85
16	1.20	90.39
17	1.26	93.25
18	1.32	93.41
19	1.43	94.98
20	0.95	87.33



Example: oxygen and hydrocarbon levels

$$n = 20 \quad \sum_{i=1}^{20} x_i = 23.92 \quad \sum_{i=1}^{20} y_i = 1{,}843.21$$

$$\bar{x} = 1.1960 \quad \bar{y} = 92.1605$$

$$\sum_{i=1}^{20} y_i^2 = 170{,}044.5321 \quad \sum_{i=1}^{20} x_i^2 = 29.2892$$

$$\sum_{i=1}^{20} x_i y_i = 2{,}214.6566$$

Example: oxygen and hydrocarbon levels

$$S_{xx} = \sum_{i=1}^{20} x_i^2 - \frac{\left(\sum_{i=1}^{20} x_i\right)^2}{20} = 29.2892 - \frac{(23.92)^2}{20}$$

$$= 0.68088$$

$$S_{xy} = \sum_{i=1}^{20} x_i y_i - \frac{\left(\sum_{i=1}^{20} x_i\right)\left(\sum_{i=1}^{20} y_i\right)}{20}$$

$$= 2{,}214.6566 - \frac{(23.92)(1{,}843.21)}{20} = 10.17744$$

Example: oxygen and hydrocarbon levels

$$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}} = \frac{10.17744}{0.68088} = 14.94748$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = 92.1605 - (14.94748)1.196 = 74.28331$$

Example interpretation

$$\hat{y} = 74.283 + 14.947x$$

$\hat{y} = 89.23\%$ when the hydrocarbon level is $x = 1.00\%$

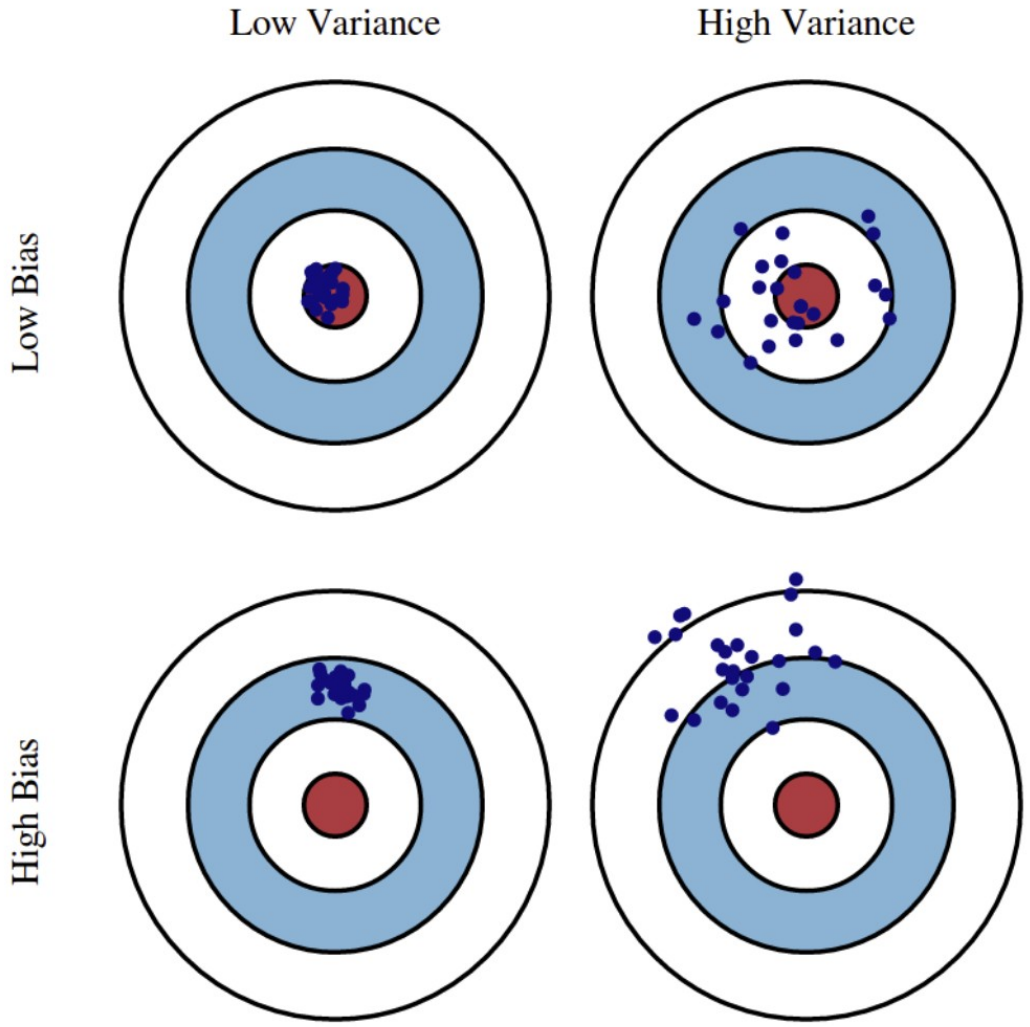
Lecture 6: Regression & Overfitting

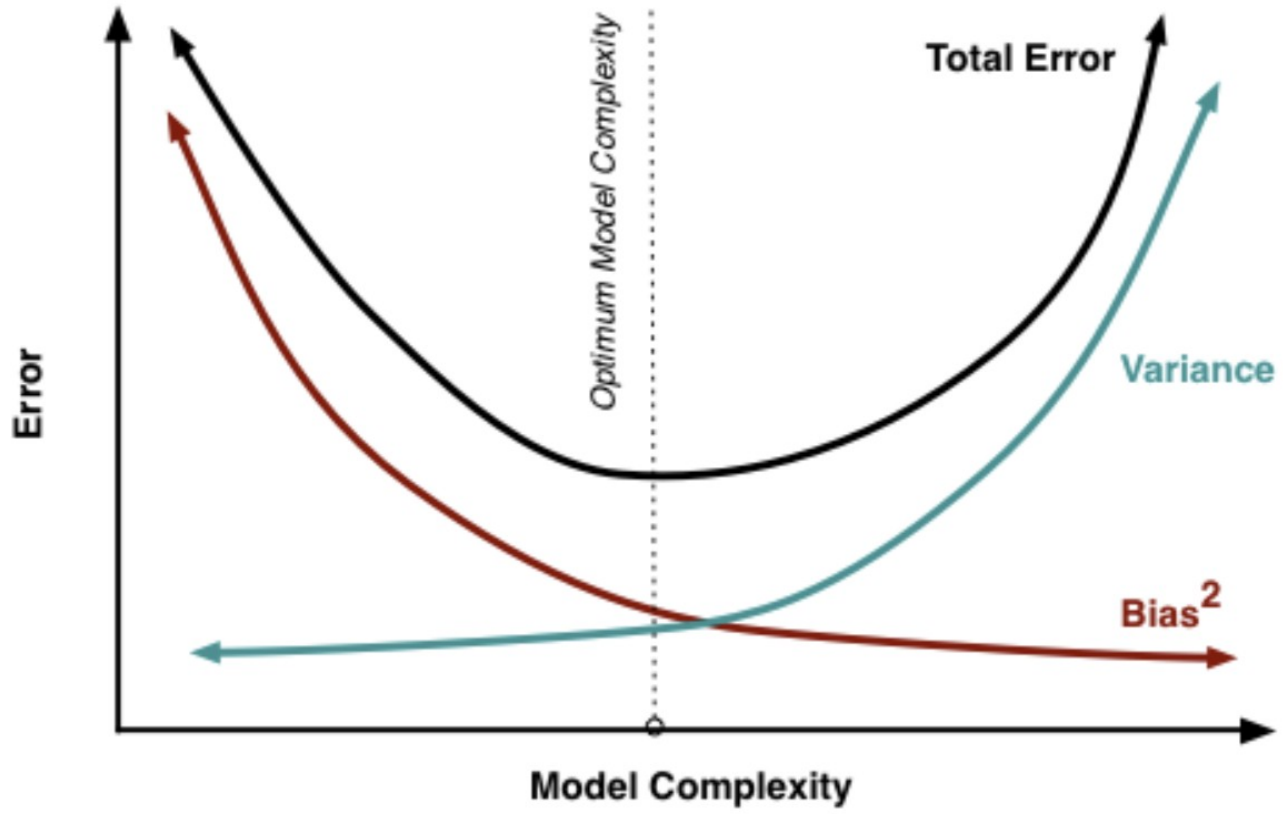
Bias-variance tradeoff

- **Variance**: Captures how much your classifier changes if you train on a different training set. How "over-specialized" is your classifier to a particular training set (“overfitting”)? If we have the best possible model for *our* training data, how far off are we from the average classifier?

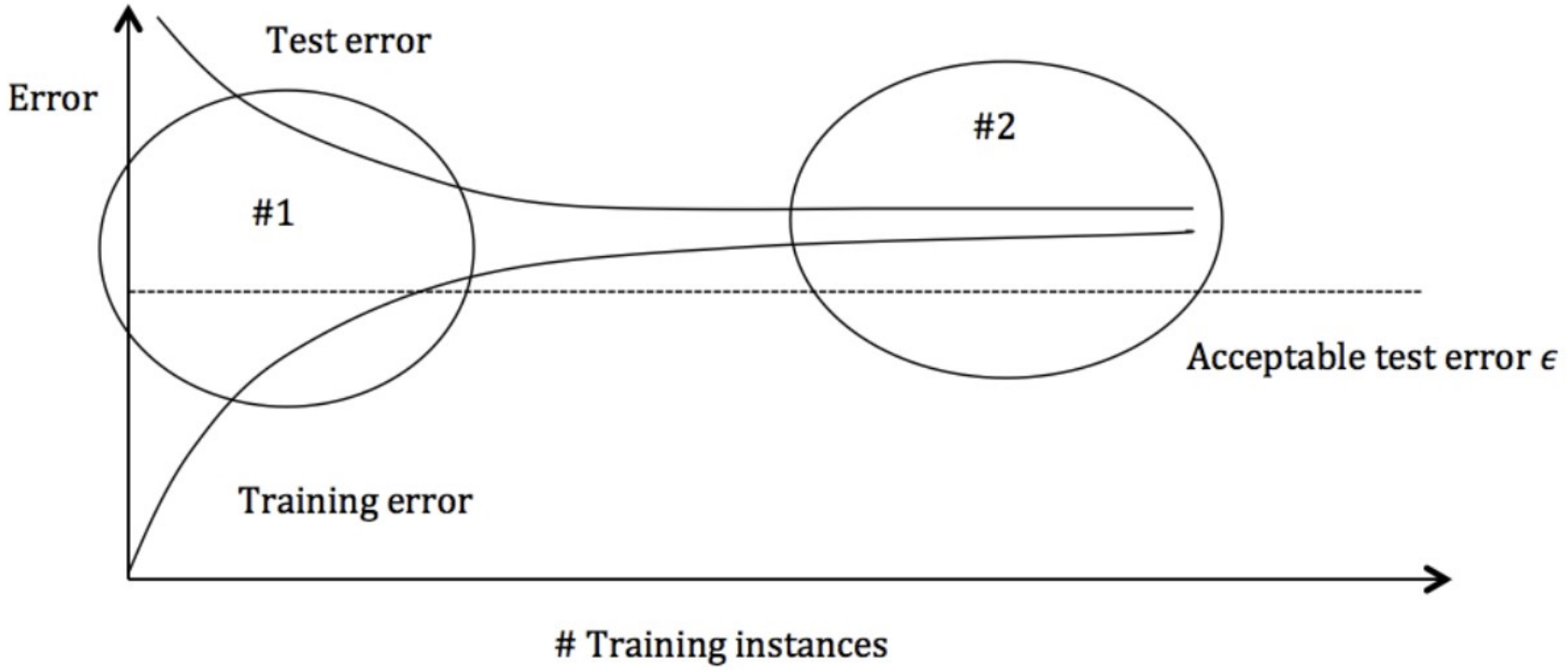
- **Bias**: What is the inherent error that you obtain from your classifier even with infinite training data? This is due to your classifier being "biased" to a particular kind of solution (e.g. linear classifier). In other words, bias is inherent to your model.

Bias-variance *tradeoff*



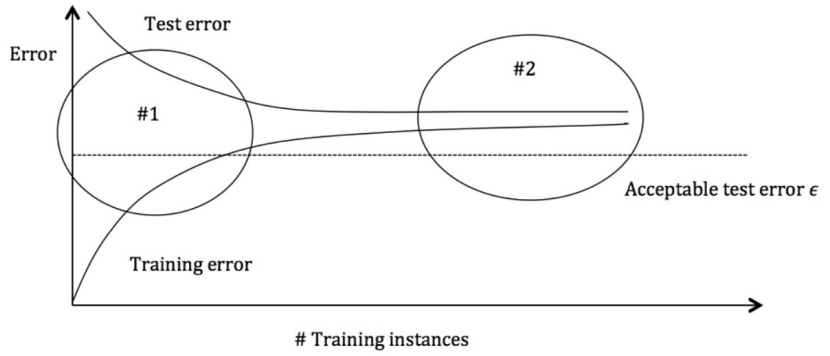


Bias-variance *tradeoff*



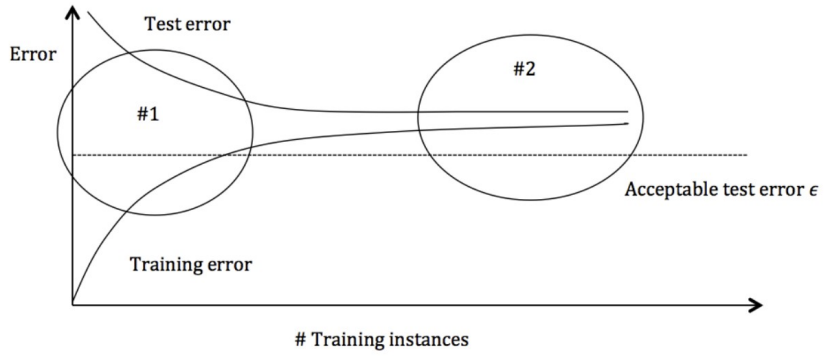
Bias-variance *tradeoff*

- Scenario/plan/regime #1 (high variance):
  - **Variance**: Captures how much your classifier changes if you train on a different training set. How "over-specialized" is your classifier to a particular training set (“overfitting”)? If we have the best possible model for *our* training data, how far off are we from the average classifier?



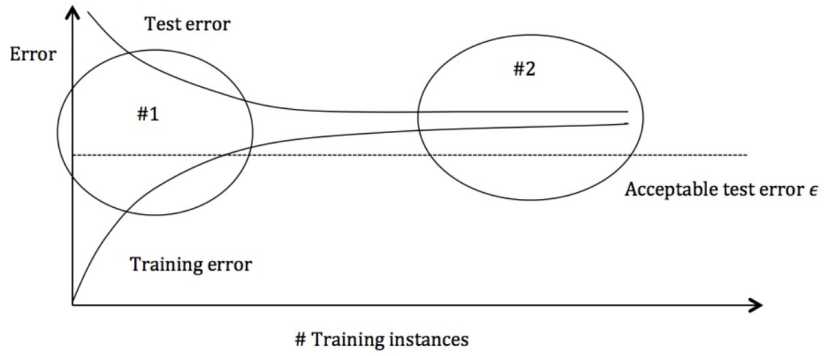
Bias-variance *tradeoff*

- Scenario/plan/regime #1 (high variance):
  - Symptoms:
    - Training error is much lower than test error
    - Training error is lower than ε
    - Test error is greater than ε
  - Remedies:
    - Add more training data
    - Reduce model complexity
    - Bagging



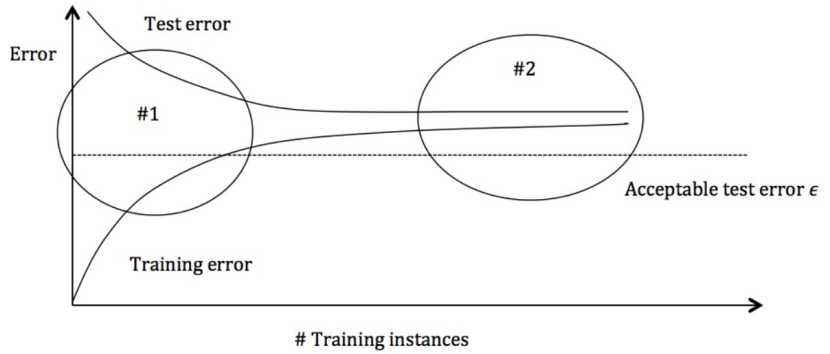
Bias-variance *tradeoff*

- Scenario/plan/regime #2 (high bias):
  - **Bias**: What is the inherent error that you obtain from your classifier even with infinite training data? This is due to your classifier being "biased" to a particular kind of solution (e.g. linear classifier). In other words, bias is inherent to your model.



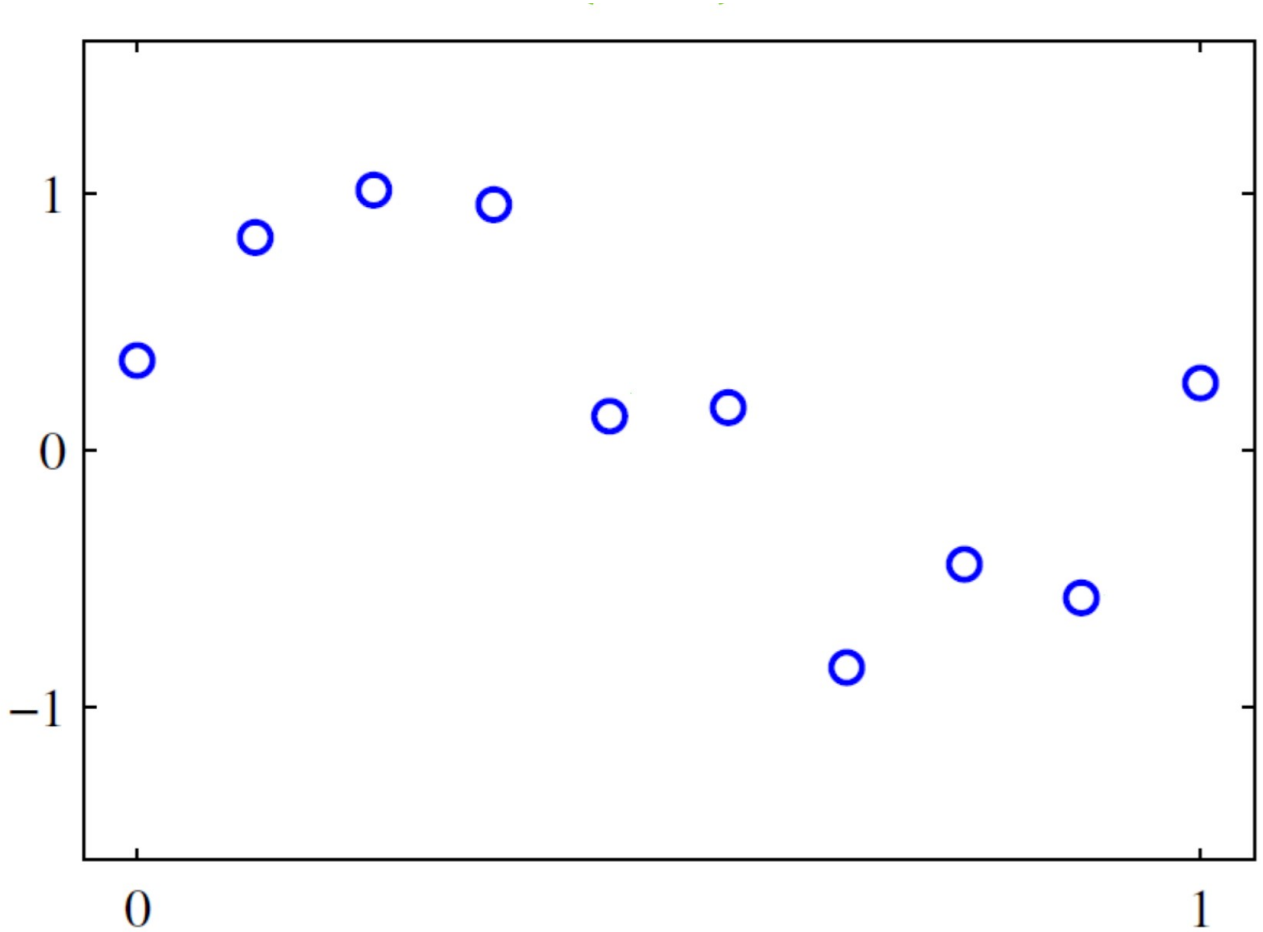
Bias-variance *tradeoff*

- Scenario/plan/regime #2 (high bias):
  - Symptoms (well, symptom):
    - Training error is higher than ε
  - Remedies:
    - Use a more complex model (say, *non*-linear regression)
    - Add features
    - Boosting



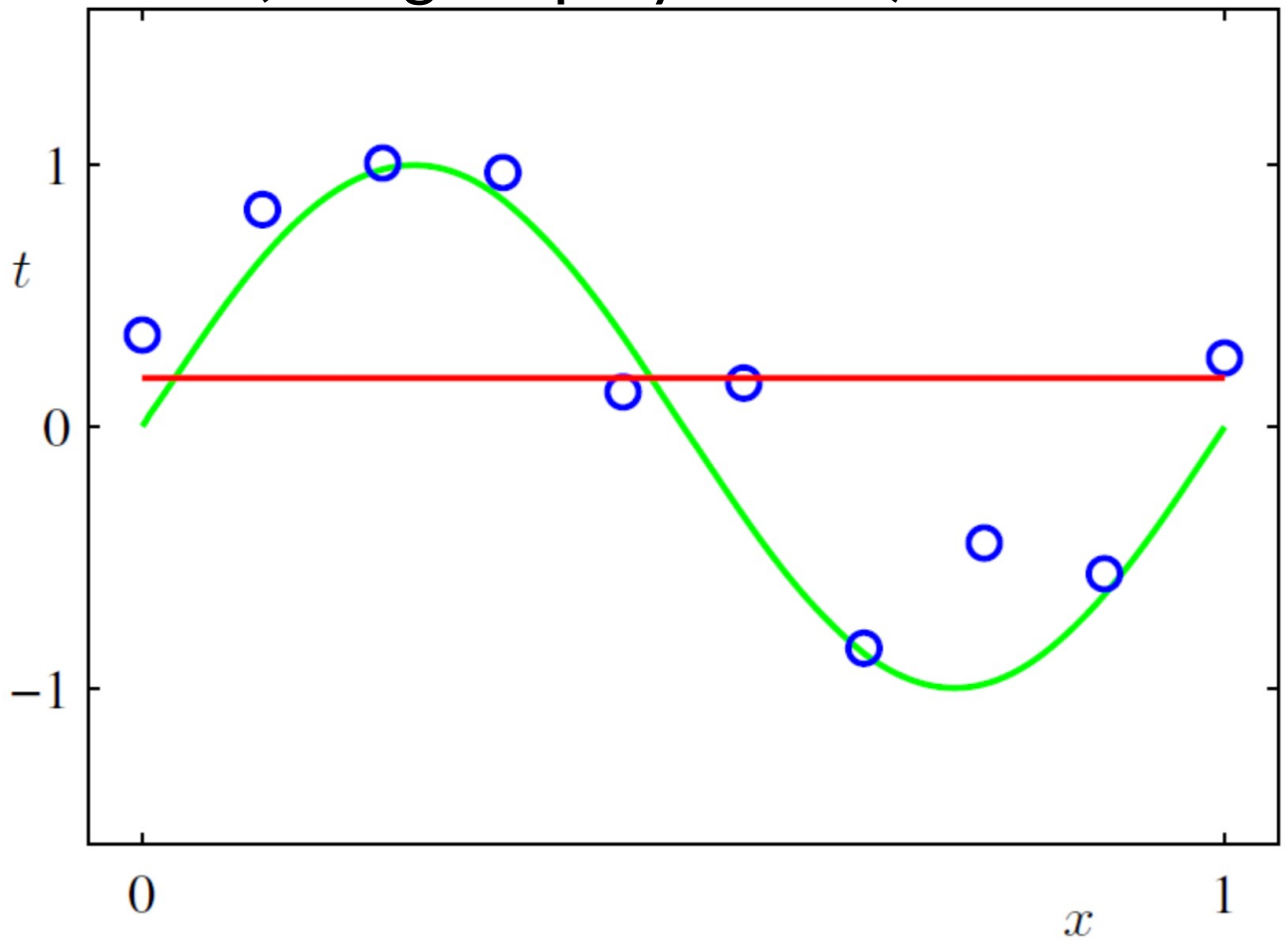
Overfitting

Say you have the following data…



Overfitting

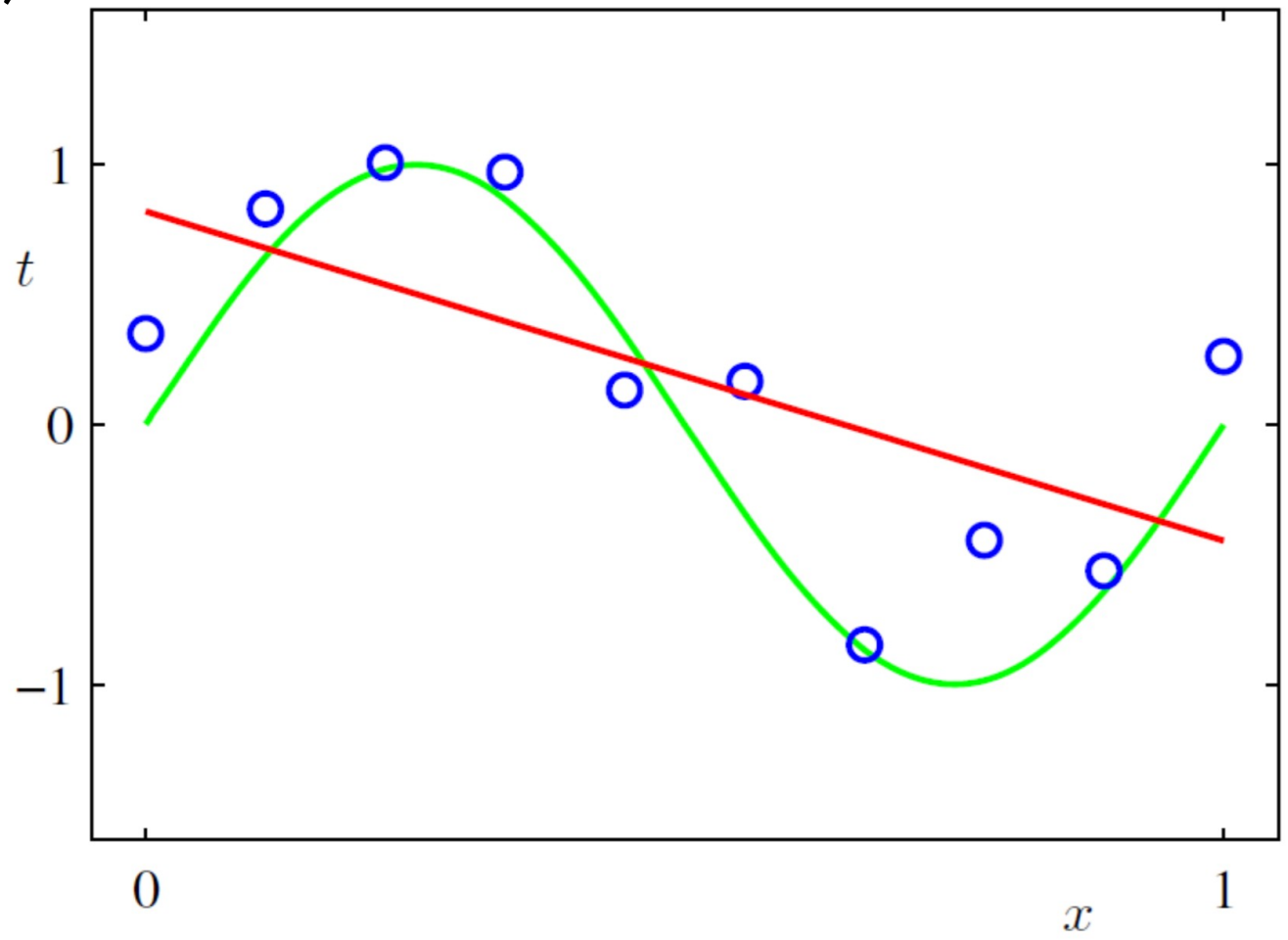
You could fit it like this (0 degree polynomial)…



Overfitting

Or like this (1st)…

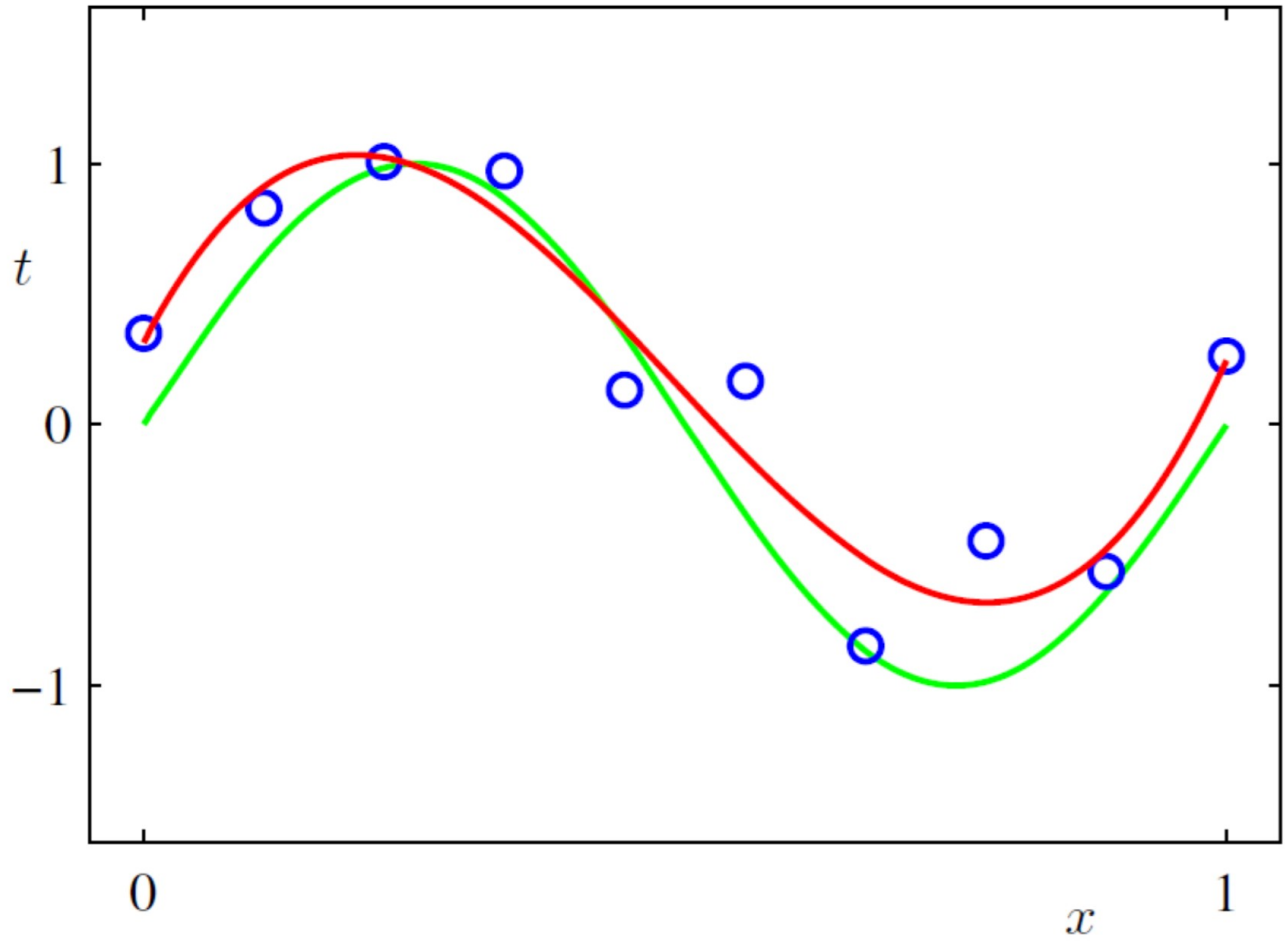
$$t = \sin(2\pi x) + \epsilon$$



Overfitting

Or like this (3rd)…

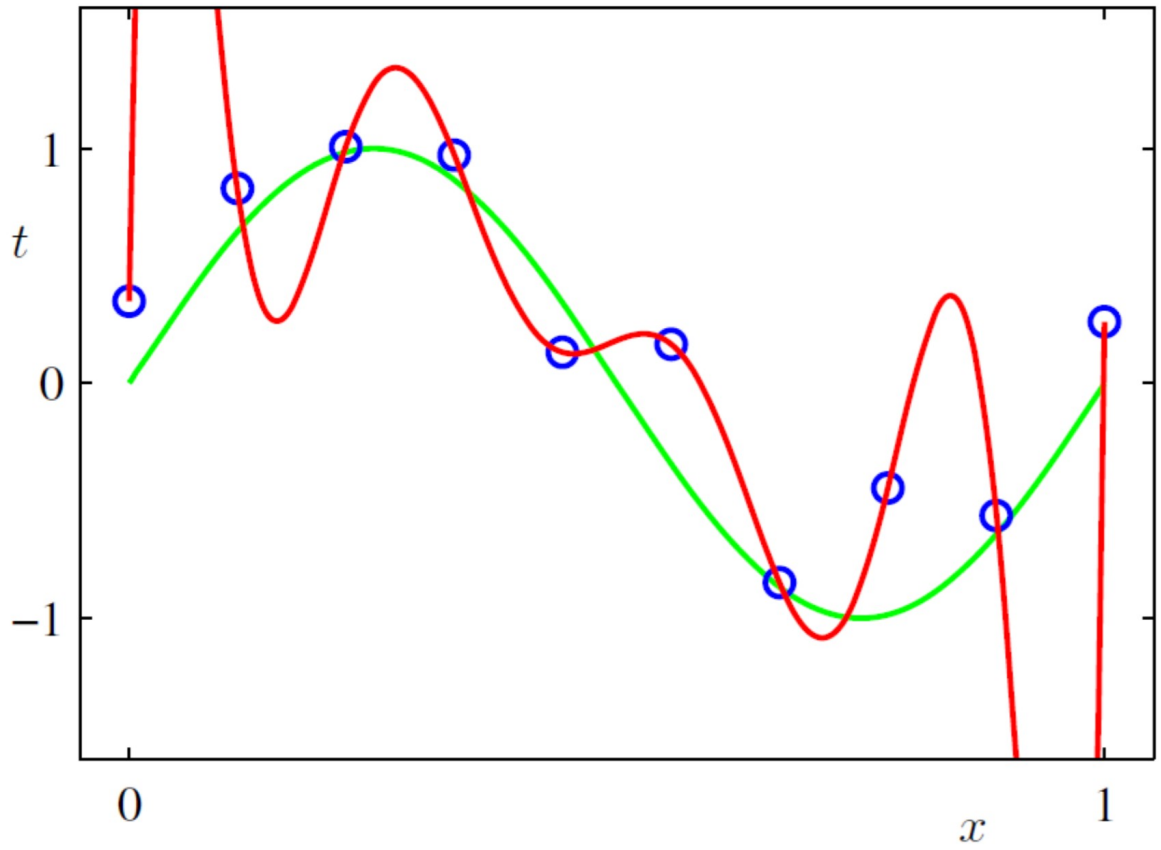
$$t = \sin(2\pi x) + \epsilon$$



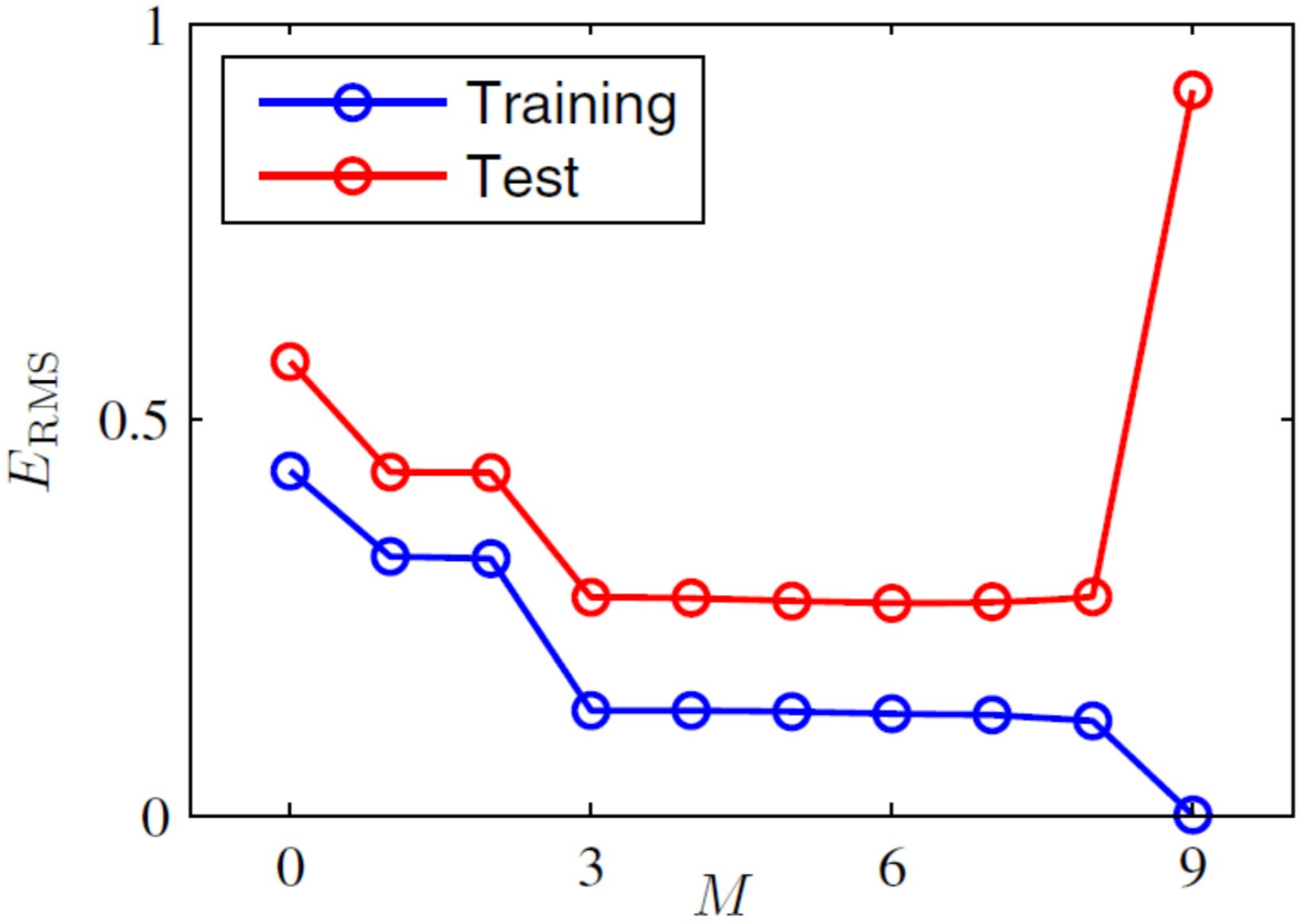
Overfitting

Or like this… (9th degree polynomial)

$$t = \sin(2\pi x) + \epsilon$$



Error vs degree of polynomial



Recap

- So, if you “fit” your training data too well (if you learn too much), you end up describing your training data *very* well, but at the cost of not describing your test data at all (“overfitting”)
- The larger the training data, the less difference there tends to be between the training data and the test data (larger data set helps!)
- The larger the hypothesis class (the more potential models that you allow), the easier it is to find a model that fits very well (and therefore predisposes you more to overfitting)

Too many possible models (hypotheses)?

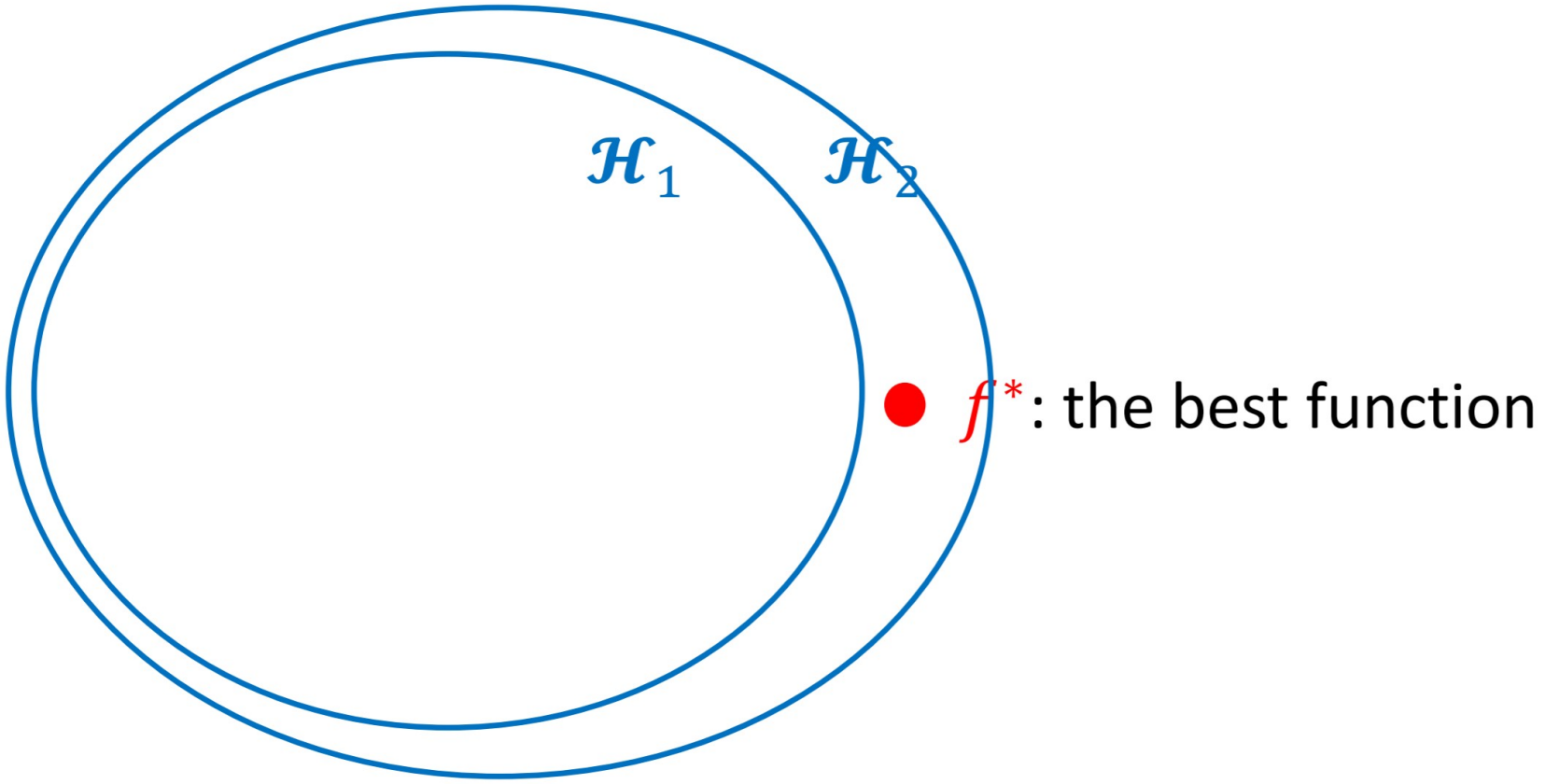
- The larger the hypothesis class (the more potential models that you allow), the easier it is to find a model that fits very well (and therefore predisposes you more to overfitting)
- Therefore, it often is not best to just learn from the data as we did with MLE, but to incorporate prior knowledge, as we did with Bayesian inference
  - But this can be overcome with more data! And is a *good strategy* **if you have no reliable prior knowledge**
  - But throwing out useless hypotheses is always a good idea

Prior knowledge vs data?

- Let's start by looking at the extremes:
  - "Perfect" prior knowledge – you don't need data, your model is equal to the true underlying distribution
  - "Perfect" data – you have infinite data, so your data *is* the underlying distribution

- Practical scenarios: finite data, H, actual model function f^* (you don't know this)

f* in or not in H



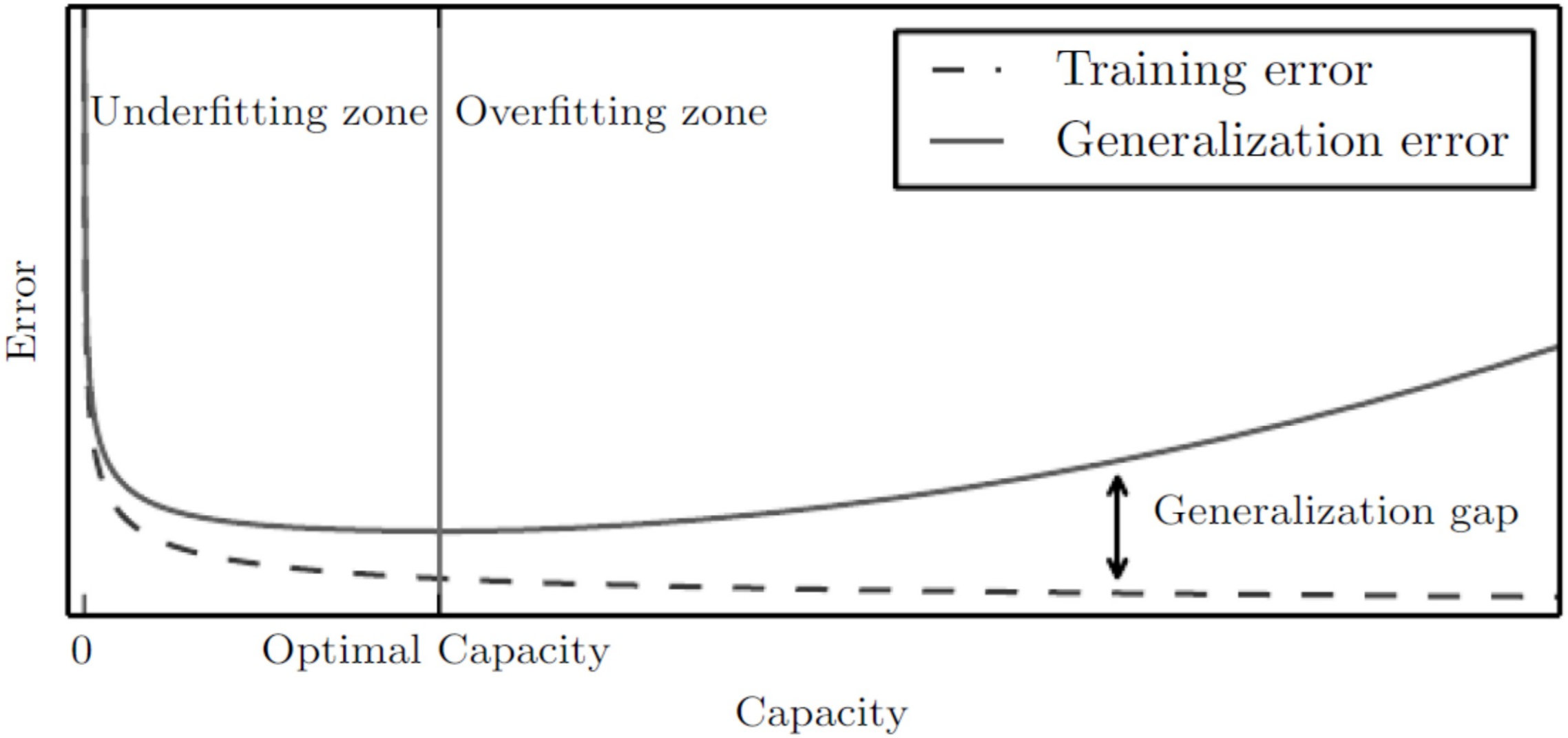
As you get more data, H_1 becomes H_2 and is more likely to encompass the actual model f^*

Prior knowledge vs data?

- Practical scenarios lie between the two extreme cases



Prior knowledge vs data?



Optimal capacity

- How do we select optimal capacity, for example, the best degree for polynomial curve fitting?
- Cannot be done by training data alone (it will always look better to just overfit)
- Solution is simple and easy to understand – cross validation

Cross validation

- Partition your training data into several groups (number of groups varies, but usually at least 3)
- Run your machine learning algorithm using all of the training data except for one of these groups
- Then use the group that you left out as the “test set”
- Then repeat but with a different group left out, etc., etc.

Cross validation

- We will learn (many) other methods for dealing with how well models are working, but cross validation:
  - Is general (you can almost always partition training data)
  - Is simple
- But…
  - Is computationally expensive, especially when there are a lot of parameters to learn

Lecture 7: Model Selection & Assessment

New terminology

- "Validation set" – a data set, like the training and test sets, but that's used to evaluate the model/algorithm used
  - Can be a subset of the training set (as we saw last time)
  - Could also be a former test set:
    - You make a self-driving car using a bunch of training data
    - You set the car up on a course, and how it behaves on this course is the test data
    - Now, take that test data to evaluate your car, it's the validation set
    - If it passes validation, toss the validation set and put it on the road with the user experience being the new test sets

Cross validation

- As we talked about last time, cross validation (or cross-validation) is a way to get example test data by using a fraction of training data as test data (while removing that fraction from your training data)

- It works by separating the data into “groups” and evaluating by training on all groups except one and then testing on that one. You would then switch the “one” that is being used as the test until all groups have been test groups

How many groups should you have? Or, put another way, how many “pieces” of data should be in each group?

Well, let's take the extremes

- Have the training set be a single piece of data, validation set will be all the other pieces. Then, repeat by switching what the one piece is and averaging.
  - Is this good? Why or why not?

- Have the training set by all data except a single piece, validation set will be a single piece of data. Then, repeat by switching what the one piece is and averaging.
  - Is this good? Why or why not?

Leave-one-out (LOO) cross validation

- Consider a validation set with 1 example:
  - D – training data
  - D_j – training data with the jth data point (x_j, y_j) moved to the validation set
- Learn classifier f_{Dj} with D_j dataset
- Estimate the error based on predicting y_j
  - This is an unbiased estimate of the true error of f_{Dj}, which should be ~f_D

Leave-one-out (LOO) cross validation

- Okay, I've got an estimate based on a nearly complete training dataset, but the estimate is based on a validation set of size 1, soo…
- LOO cross validation – average over all possible data points, j:
  - For each j left out, build a new classifier/algorithm f_{Dj}
  - Estimate the error as:

$$\text{error}_{LOO} = \frac{1}{n}\sum_{j=1}^{n}(y_j - f_{\mathcal{D}\setminus j}(x_j))^2$$

Or, the average squared difference between the measurement and the prediction

Leave-one-out (LOO) cross validation

- LOO cross validation (LOOCV) is an *almost* unbiased estimate of the true error (very, *very* close in most cases)
  - You are measuring your error in an unbiased way for f_{Dj}, but it isn't exactly equal to your error for f_D
  - So, it isn't an estimate of the true error if you had all N data points (but if N is large, it is very, *very* close)

- LOO is an extremely powerful, virtually unbiased method for model selection. Feel free to use it for almost any application!

Too good to be true…

- What’s the computational cost of LOO/LOOCV?
- Suppose you:
  - Have 100,000 data points
  - Have an efficient machine learning algorithm that only takes 1 second
  - How long would the validation take?

LOOCV

- A lot of this class will be:
  - “Wow, this is great!”
  - “Well, it doesn’t actually work that well with real data, is slow, etc.”
  - Cross validation is powerful but slow, and as such has many, many potential applications, particularly:
    - Not that much data
    - You don’t care about the time to test your algorithm (say, you want to publish your algorithm)

k-fold Cross Validation

- *Randomly* divide data into *k* equal parts
  - D_1, D_2, …, D_k
- For each i from 1 to k:
  - Learn classifier $f_{D\backslash Di}$ using all data except data in D_i
  - Estimate error of $f_{D\backslash Di}$ on the validation set, D_i:

$$\text{error}_{\mathcal{D}_i} = \frac{1}{|\mathcal{D}_i|} \sum_{(x_j, y_j) \in \mathcal{D}_i} (y_j - f_{\mathcal{D}\backslash\mathcal{D}_i}(x_j))^2$$

$$error_{k-fold} = \frac{1}{k} \sum_{i=1}^{k} error_{\mathcal{D}_i}$$

k-fold Cross Validation

- *k*-fold cross validation is:
  - Much faster to compute than LOO
  - More biased (uses much less data)
  - Depends on user to select *k* (10 is pretty standard but can vary quite a bit)

Limitations of cross validation

- You have relatively little data, say 20 examples
  - *k*-fold cross validation is a bad choice since there won't be much validation data in any one error test
  - LOO would work, but what if you want to know the error for a *particular example x* instead of the whole data?

Bootstrapping

- Given a set of data that are independent and identically distributed (i.i.d.) samples, D = {Z_1, Z_2,…Z_n}, we want to compute θ

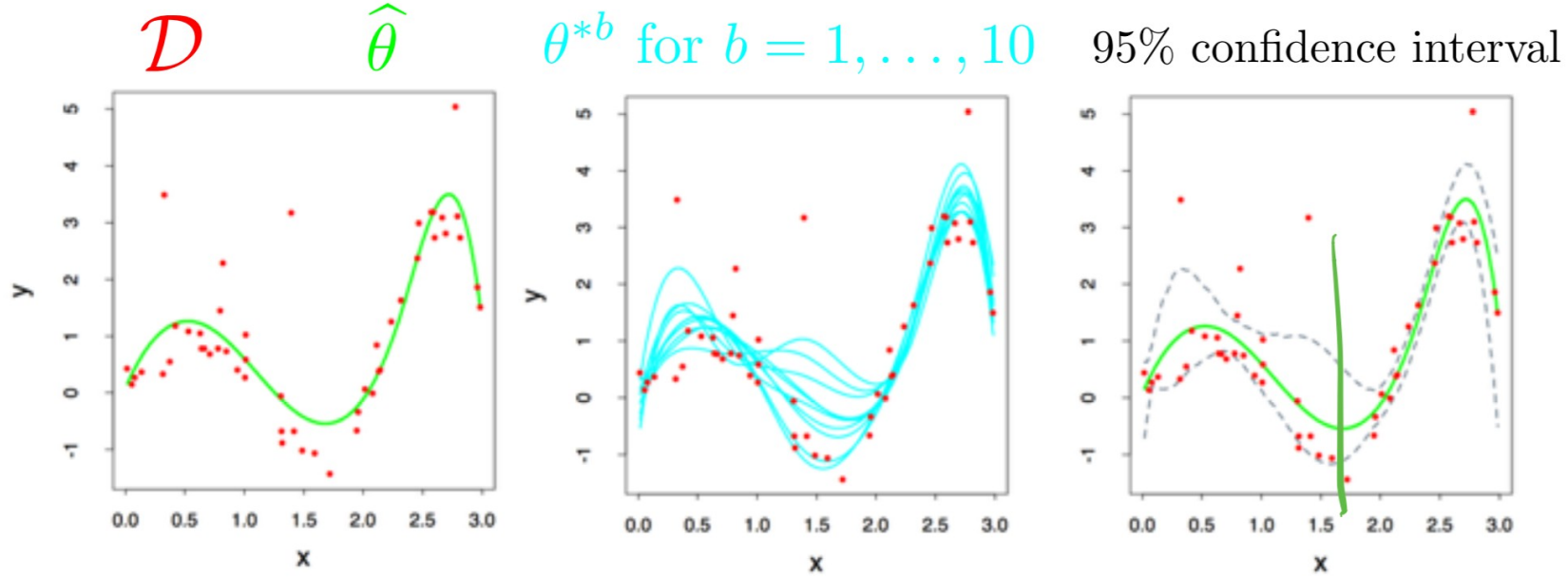
- For b=1, 2,…B, we define the *b*th bootstrapped dataset as drawing n samples ***with replacement*** from D

$$\mathcal{D} = \{z_1, \ldots, z_n\} \overset{i.i.d.}{\sim} F_Z \qquad \widehat{\theta} = t(\mathcal{D})$$

$$\mathcal{D}^{*b} = \{z_1^{*b}, \ldots, z_n^{*b}\} \overset{i.i.d.}{\sim} \widehat{F}_{Z,n} \quad \theta^{*b} = t(\mathcal{D}^{*b})$$

Bootstrap applications

- Common applications of the bootstrap:
  - Estimate parameters that escape simple analysis like the variance or median of an estimate
  - Confidence intervals
  - Estimates of error for a particular example:



Bootstrap takeaways

- Advantages:
  - Bootstrap is very generally applicable. Build a confidence interval around anything
  - Very simple to use
  - Appears to give meaningful results even when the amount of data is very small
  - Very strong asymptotic theory (as num. examples goes to infinity)

Bootstrap takeaways

- Disadvantages:
  - Very few meaningful finite-sample guarantees
  - Potentially computationally intensive
  - Reliability relies on test statistic and rate of convergence of empirical CDF to true CDF, which is unknown
  - Poor performance on “extreme statistics” (e.g., the max)

Lecture 8: (Abbreviated) Lasso Regression & Catch-up

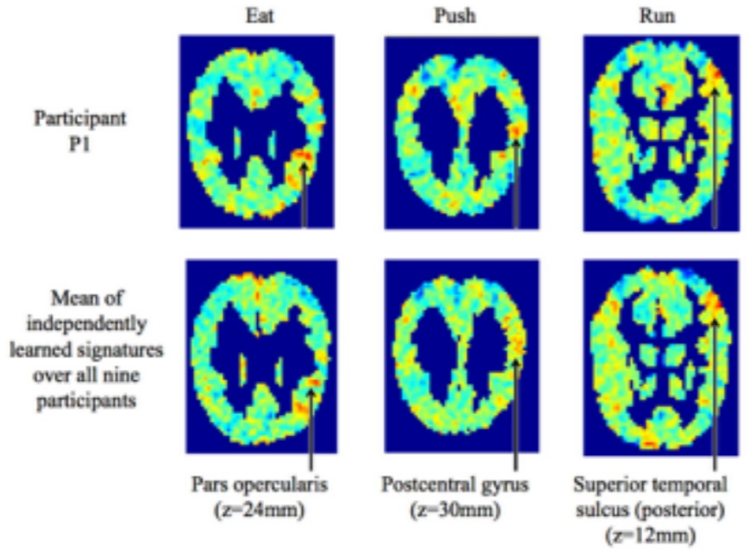
“Sparsity”

- Vector w is sparse if many entries are zero
  - For example, if you roll a single die 3 times and record how many 1’s, 2’s, 3’s, 4’s, 5’s, and 6’s, you will have a sparse vector
  - In reality entries close to zero are sometimes considered “sparse”
    - Two definitions depending on context!!

- Useful in many tasks
  - Efficiency: if size(w) is a billion, each prediction is expensive:
    - If part of an online system, this can be very slow
    - If w is sparse, prediction computation only depends on number of non-zeros

"Sparsity"

- Useful in many tasks
  - Efficiency: if size(w) is a billion, each prediction is expensive:
    - If part of an online system, this can be very slow
    - If w is sparse, prediction computation only depends on number of non-zeros
  - Interpretability: what are the relevant dimensions to make a prediction?
    - What are the parts of the brain associated with particular words?



Question: How do we find the “best” subset among all possible features?

Greedy model selection

- Pick features
- Greedy heuristic:
  - Start from an empty (or simple) set of features, F_0 = null
  - Run learning algorithm for current set of features, F_t
    - Obtain weights for these features
  - Select next best feature, $h_i(x)^*$
    - That is, $h_j(x)$ that results in the lowest training error when using $F_t + \{h_j(x)^*\}$
  - F_{t+1} <- $F_t + \{h_i(x)^*\}$
  - Recurse

Greedy model selection

- Applicable in many situations (some of which we'll talk about later in this course):
  - Logistic regression: selecting features (basis functions)
  - Naïve Bayes: selecting (independent) features $p(X_i|Y)$
  - Decision trees: selecting leaves to expand
- Only a heuristic
  - Finding the best set of k features is computationally intractable!

Greedy model selection

- Recurse until??? When do you stop?
  - When training error is low enough?
  - When test set error is low enough?
  - Using cross validation?

Is there a set solution that makes sense?

Lasso regression

- **L**east **A**bsolute **S**hrinkage and **S**election **O**perator
- Finds the coefficients that should be used (like linear regression)
- Also provides best subset selection, which indicates the most valuable variables for using for machine learning

Lecture 9: Introduction to Classification

Classification

- So far, we've mostly talked about regression – predicting a continuous value given some inputs & training data

- Many examples of machine learning (some we've mentioned) are based on classification rather than continuous data

Classification

- Want to learn f: X -> Y
  - X is the feature(s) that you will use to predict
  - Y is the target classes

- Conditional probability: p(Y|X)
  - That is, you want to know the probability that you belong to a particular class given the features of the data

Remember: Bayes theorem

- $p(A|B) = [p(B|A) * p(A)] / p(B)$

  For our purposes, p(B), the probability of our data without any other information is constant, so:

- $p(A|B) \alpha p(B|A) * p(A)$

  This is useful to us because we don't care about the exact probability, we just want to optimize our model

"What is the most probable hypothesis given our training data?"

Bayes optimal classifier

- Also called:
  - Bayes optimal learner
  - Bayes classifier
  - Bayes optimal decision boundary
  - Bayes optimal discriminant function

- The specific question answered by the Bayes optimal classifier is *“what is the most probable classification of the new instance given the training data?”*

Bayes optimal classifier

- We want to calculate the probability of a new instance (v_j) given the training data (D), given the space of hypotheses (H):

  $$P(vj \mid D) = \text{sum } \{h \text{ in } H\} P(vj \mid hi) * P(hi \mid D)$$

- Selecting the outcome with the highest probability is Bayes *optimal* classification:

  $$\max \text{ sum } \{h \text{ in } H\} P(vj \mid hi) * P(hi \mid D)$$

Bayes optimal classifier

Any system that classifies new instances according to [the equation] is called a Bayes optimal classifier, or Bayes optimal learner. No other classification method using the same hypothesis space and same prior knowledge can outperform this method on average.

-Machine Learning, 1997. Page 175

Bayes optimal classifier

- Due to uncertainty in the training set and inaccuracies in the hypothesis space, the model will make errors
  - However, this error is the minimum possible error
  - **Bayes error**: the minimum possible error that can be made while making predictions

- Note that this methodology can be applied to regression as well as classification (although we won't)

Bayes optimal classifier

Note that it is a theoretical model. A gold standard, an unattainable ideal to pursue, rather than a method that we can use. This is because we cannot know the conditional probability of Y given X.

Bayes optimal classifier

- It is not computationally feasible, or statistically possible, to implement a proper Bayes optimal classifier, so we use simplifications. Two common simplifications are:
  - **Gibbs Algorithm**. Randomly sample hypotheses biased on their posterior probabilities
  - **Naïve Bayes**. Assume that variables in the input are conditionally independent
  - Really, many non-linear machine learning algorithms that are able to make predictions are close approximations of Bayes optimal classifier in practice

Back to where we started: classification

- The problem of classification is to assign a label to a given input data sample

- Bayes theorem is an obvious choice to deal with this, but has severe limitations in most applications:
  - Requires *a lot* of training data
  - Is computationally very expensive
  - This is because of how the states of your training (and test) data relate to one another
    - If you want to predict that an email is spam, the probabilities that it contains the word “congratulations” and is from a previously uncontacted address are related to one another.

Naïve Bayes

- But what if the states *did not* depend on one another?
  - Using the word “congratulations” has no impact on whether you’ve contacted that sender in the past (and vice versa)
  - Fidgeting in an airport has no impact on whether or not you look away from security personnel (and vice versa)
  - …
- Naïve Bayes makes this assumption – the assumption of independence

Naïve Bayes – slight math

- p(you belong to this class|thing you learn from 1 and thing you learn from 2 and thing you learn from 3…thing you learn from n) = something very hard to calculate
  - You need *TONS* of data
  - It'll take forever and half to compute

Unless…

- All of the things you learn from are independent!
- Then, p(you belong to this class|all that stuff) = p(you belong to this class|thing 1) * p(you belong to this class|thing 2) * …

Naïve Bayes – slight math example

- p(email is spam|it is from unknown sender & it uses non-letter text & it isn't from @conncoll.edu…& it says "you won") = something very hard to calculate
  - You need *TONS* of data
  - It'll take forever and half to compute

Unless…

- All of the things you learn from are independent!
- Then, p(email is spam|all that stuff) = p(email is spam|unknown sender) * p(email is spam|uses non-letter text) * …

Naïve Bayes

- p(you belong to this class|all that stuff) = p(you belong to this class|thing 1) * p(you belong to this class|thing 2) * …
  - p(you belong to this class|thing 1) α p(thing 1|you belong to this class) * p(you belong to this class)
    - p(you belong to this class) is trivial to estimate: examples of this class in training data / total examples in your training data

      -or-

    - p(spam email) = # spam email in training data / total email in training data

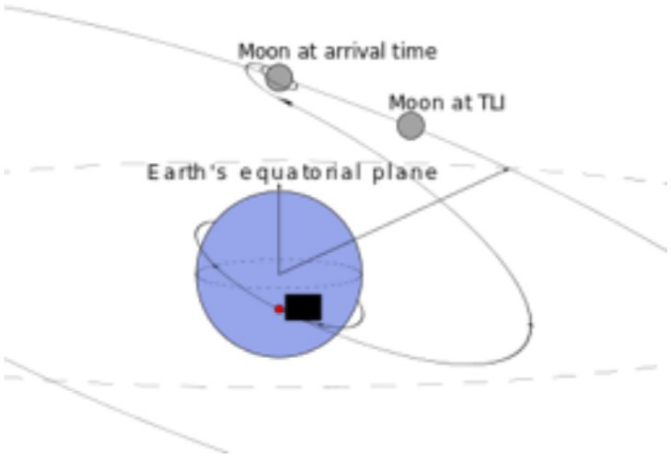
Naïve Bayes

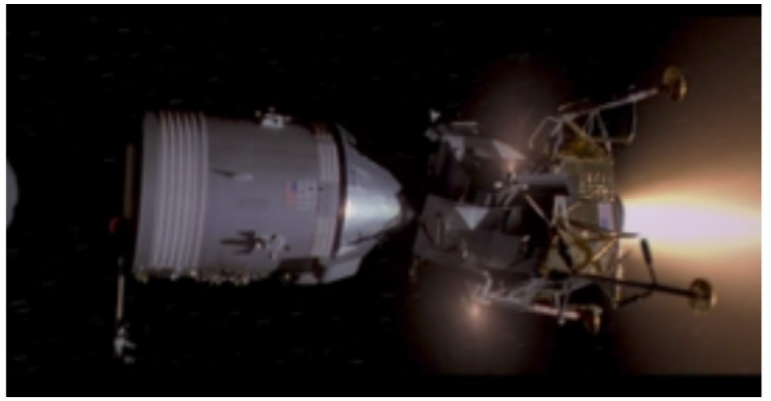
- p(you belong to this class|thing 1) α p(thing 1|you belong to this class) * p(you belong to this class)
  - p(thing 1|you belong to this class) isn't very hard either (although harder)
    - p(unknown sender|it's a spam email)
    - It depends very strongly on what kind of data you're looking at
      - Is it numerical? Binary? Categorical?
      - You can envision using temperature as a feature that you learn on. Or sunny vs not. Or season.
      - You may (and often will) have different kinds of data in your algorithm. This isn't a problem, just use the appropriate method for each feature!
      - Binary data uses the binomial distribution, categorical data uses multinomial distribution, and numeric data uses Gaussian distribution
      - You can also just approximate from the data.

Lecture 10: Online Learning

Online learning example: to the moon (the first time)

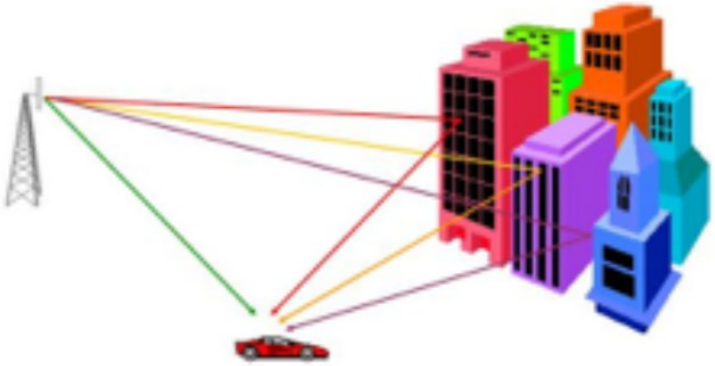
- Guidance computer predicts trajectories to the moon and back with
  - Noisy sensors
  - Imperfect models
  - Little computational power





Online learning -- recently

The original "Big Data"



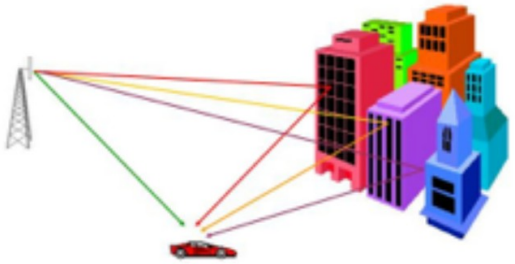
Wifi/cell-phones are *constantly* solving least squares to invert out multipath



Low power devices, high data rates

Online learning -- recently

The original "Big Data"



Wifi/cell-phones are *constantly* solving least squares to invert out multipath



Low power devices, high data rates



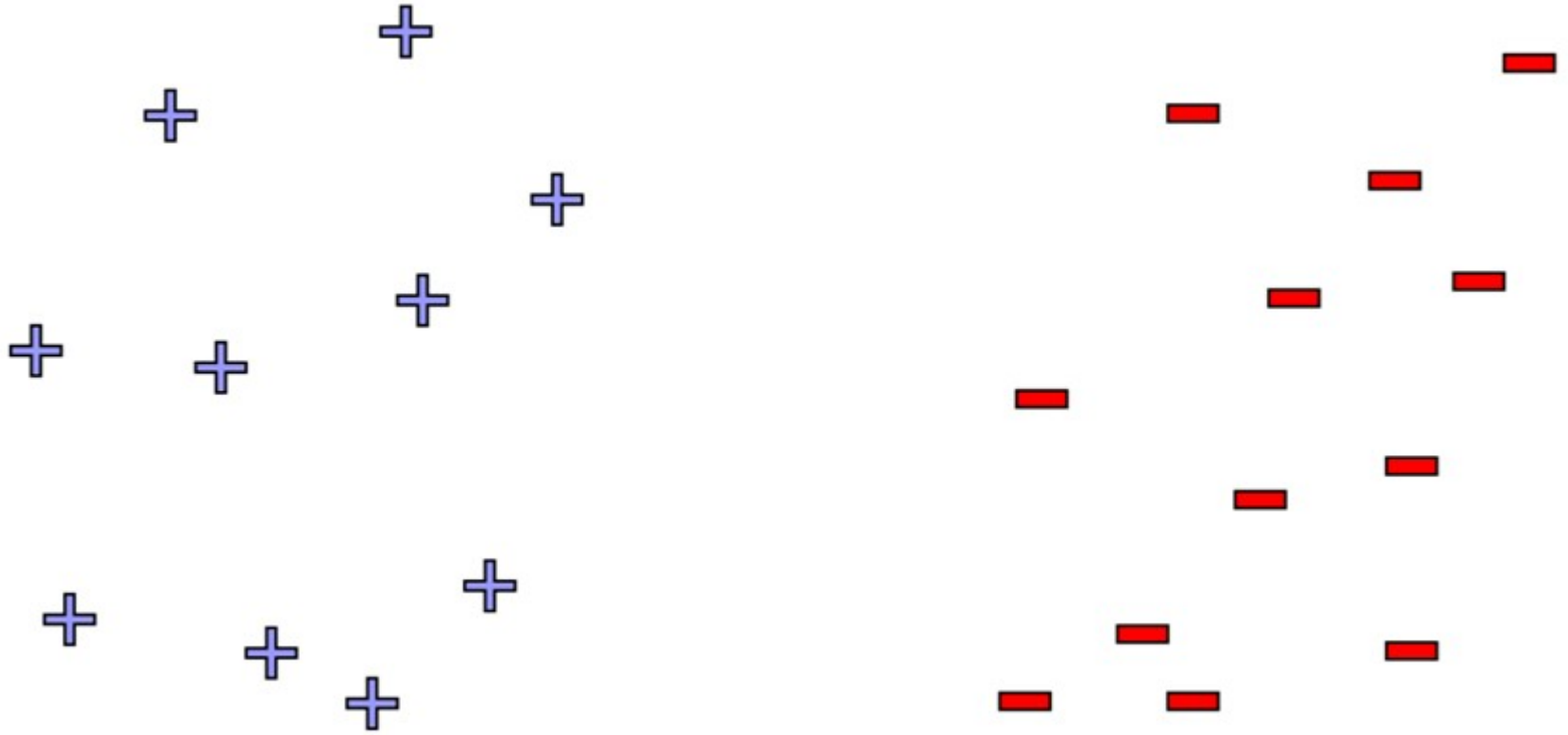
Gigabytes of data per second



Online learning

- Predictive ads deal with streaming data
  - User enters a query, and an ad must be selected
    - Observe the query, X_j, predict the ad – clicked or not clicked, Y_j!
  - User either clicks the ad or doesn't click the ad
    - Label on Y_j revealed afterwards
      - Google (or Facebook or whoever) gets a reward if the ad is clicked
  - Update the model for next time

Online classification

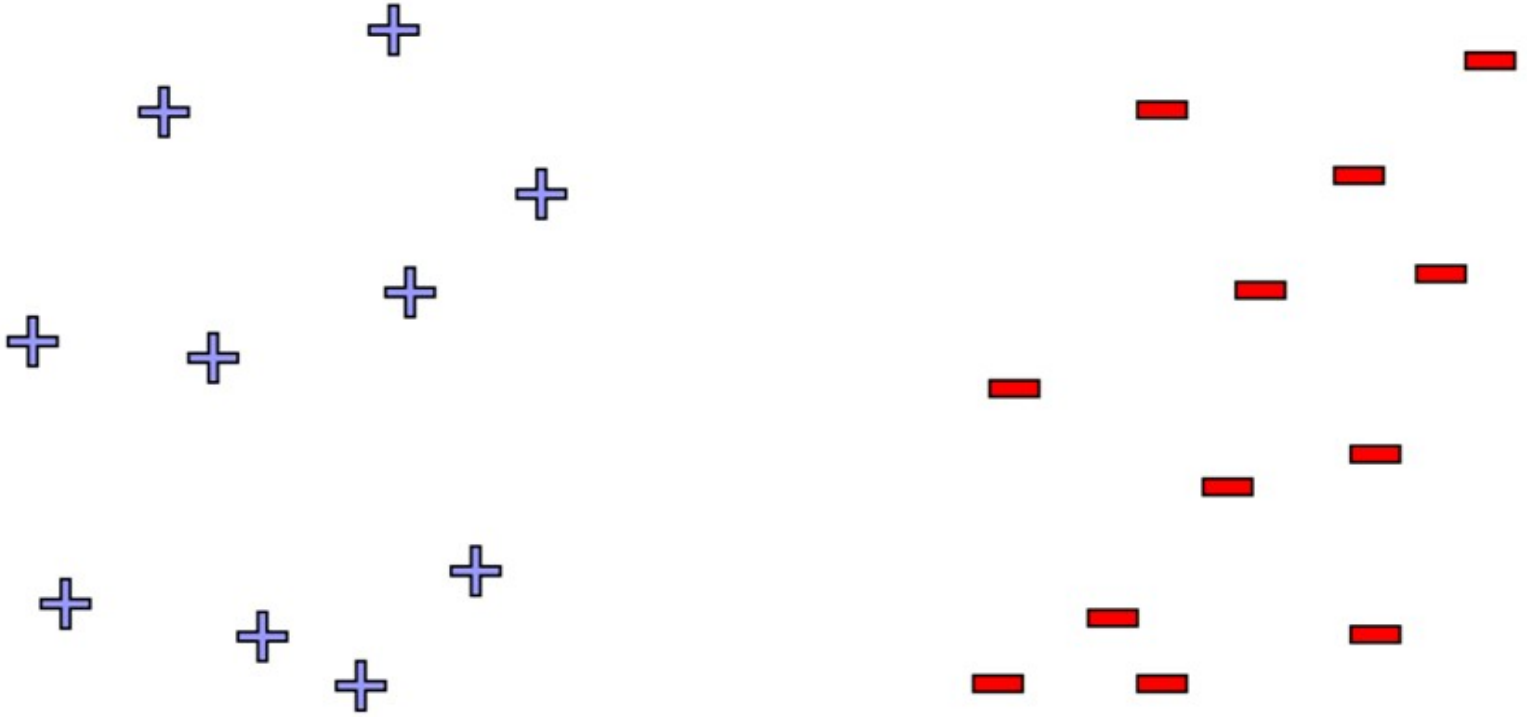


The perceptron algorithm

- Say y takes values {-1, 1}
- Linear model – prediction $y_k \sim \sin(x_k w + b)$
- Training:
  - Initialize weights (w_0, b_0)
  - At each step, k:
    - Observe features
    - Make predictions
    - Observe true class
    - Update model *if prediction is not equal to the truth*

Perceptron algorithm

- Perceptron is guaranteed to converge as long as the data is linearly separable



Perceptron algorithm

- The algorithm has that limitation, but is otherwise really cool!
  - No assumption about data distribution, *does not need to be i.i.d.*
  - Could be generated by an oblivious adversary
  - Makes a fixed number of mistakes, and it's done forever

    Even as you see infinite data
- But…perceptron is useless in practice
  - Real world data are not linearly separable
  - If it isn't separable, algorithm cycles forever
  - Even if separable, algorithm may not give good accuracy (may have small margin)

Lecture 11: Support Vector Machines

Binary classification

- Given training data (x_i, y_i) for i = 1, 2, …, N where each y_i is -1 or +1
- You want to learn a classifier f(x) such that:

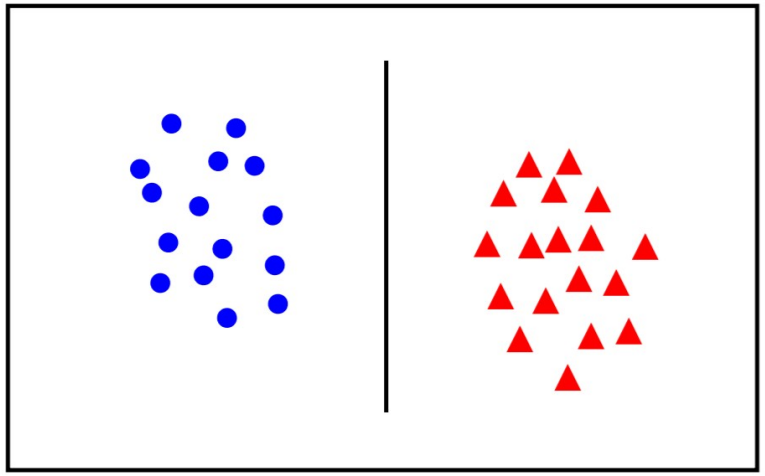
$$f(\mathbf{x}_i) \begin{cases} \geq 0 & y_i = +1 \\ < 0 & y_i = -1 \end{cases}$$

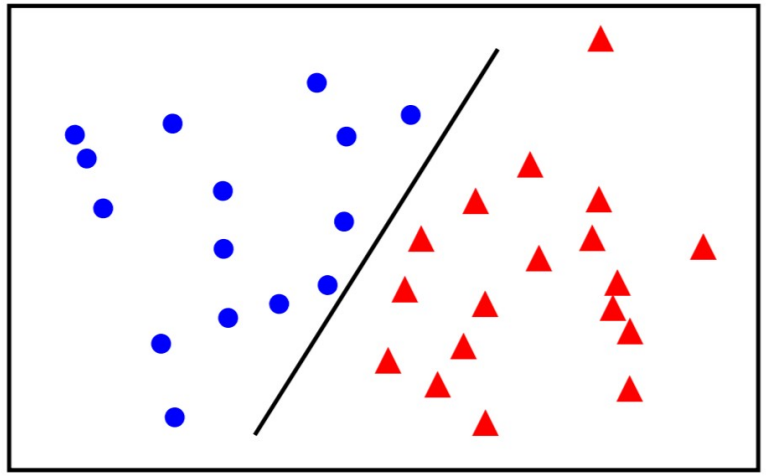
Math is largely subjective, but this has the advantage of:

$y_i * f(x_i) > 0$ for a correct classification

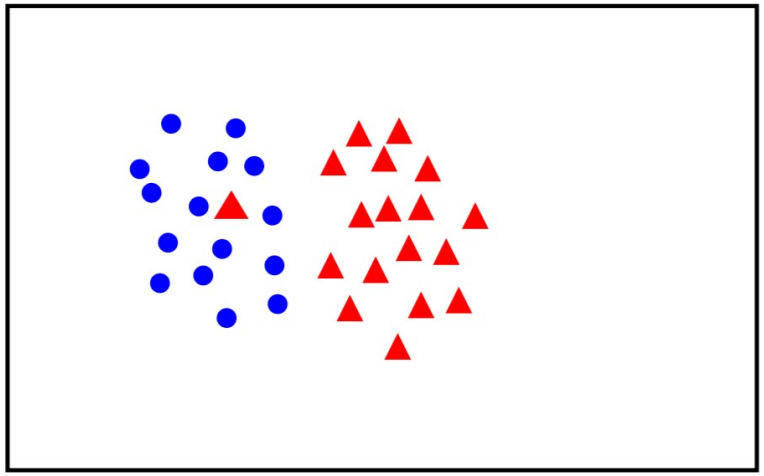
Linear separability

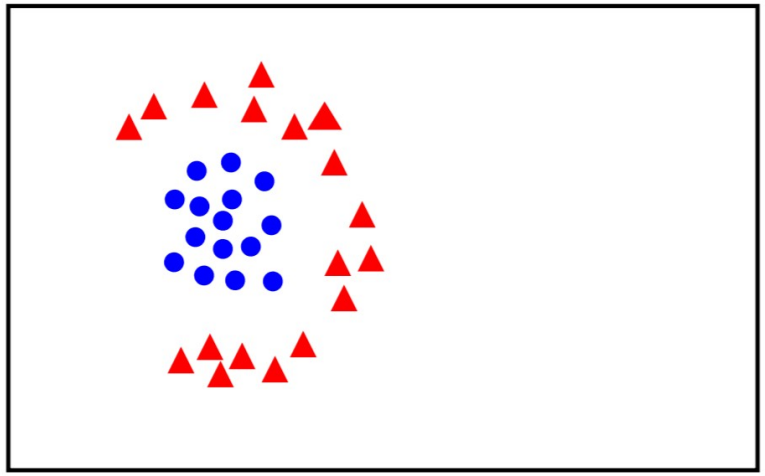
linearly separable





not linearly separable

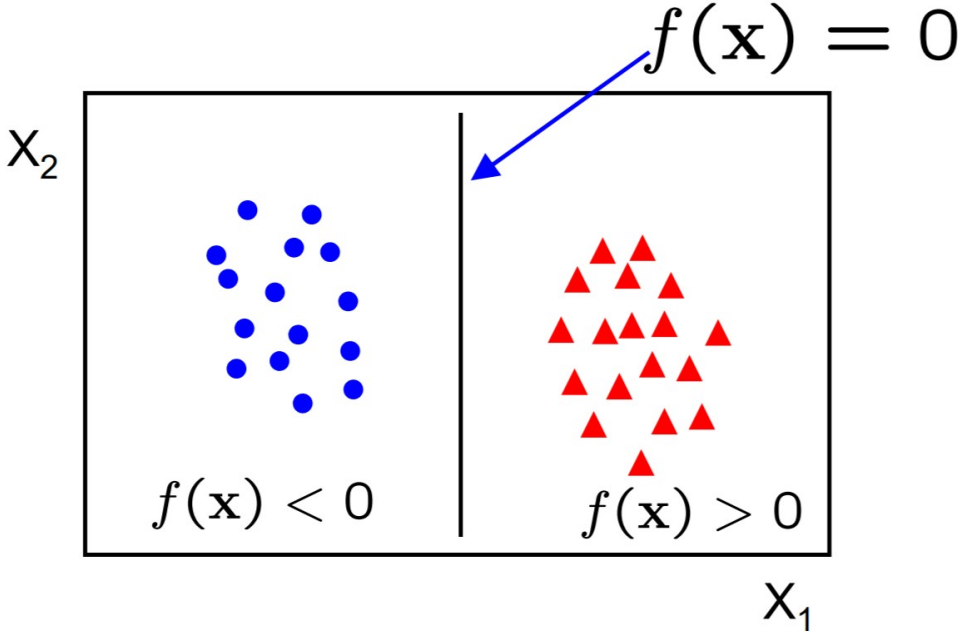




Linear classifiers

- A linear classifier has the form $f(x) = w^T x + b$

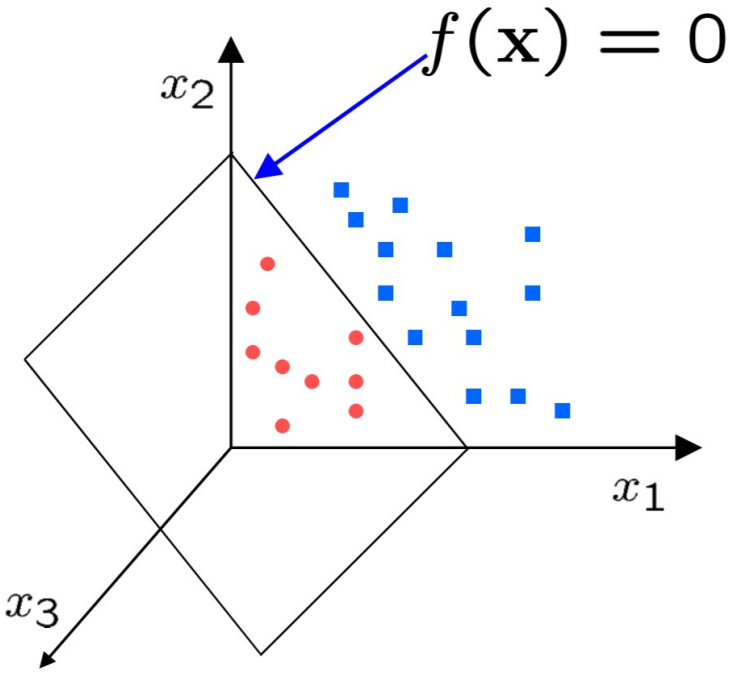
- In 2D, discriminant is a line
- w is the normal to the line, b is the bias
- w is known as the weight vector



Linear classifiers

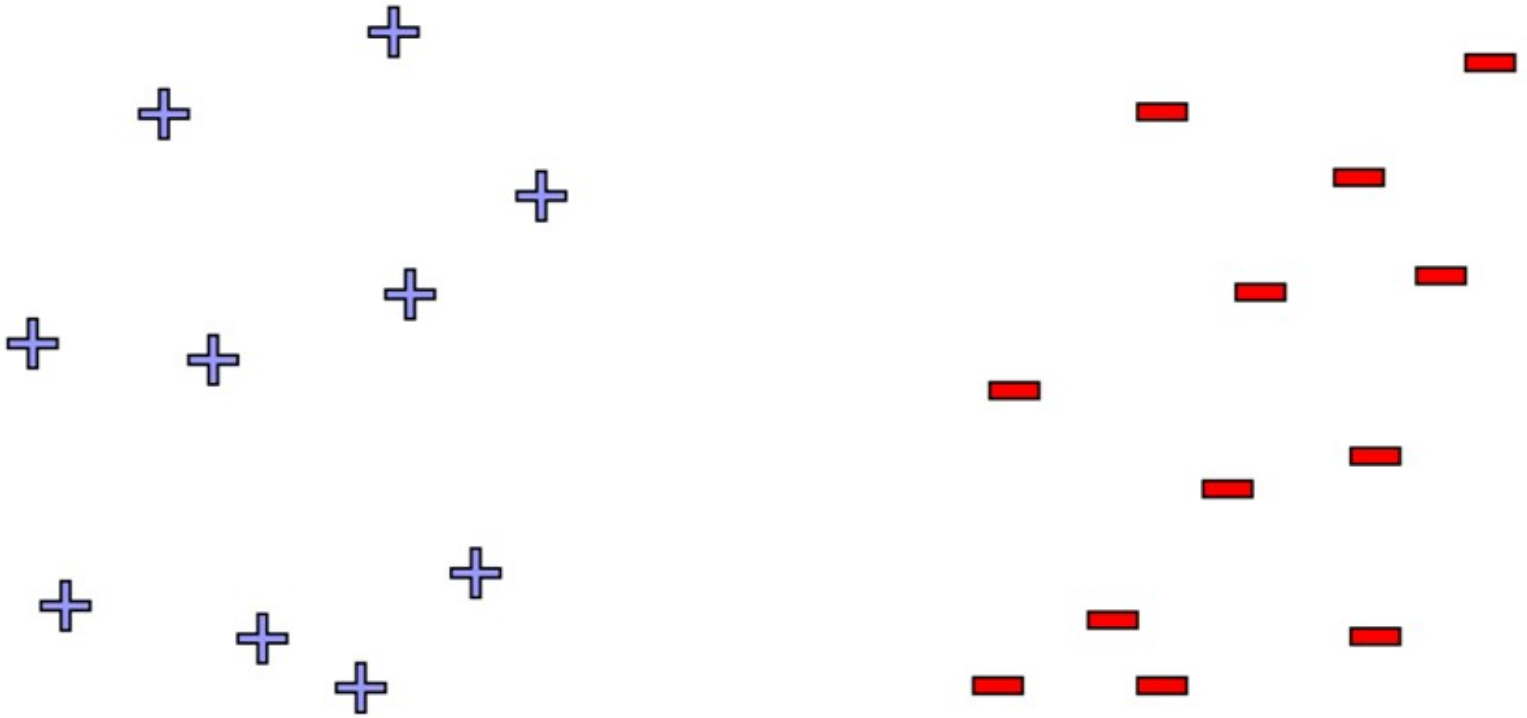
- A linear classifier has the form $f(x) = w^T x + b$

- In 3D, discriminant is a plane
- For a linear classifier, the training data is used to learn w and then discarded
- Only w is needed for classifying new data

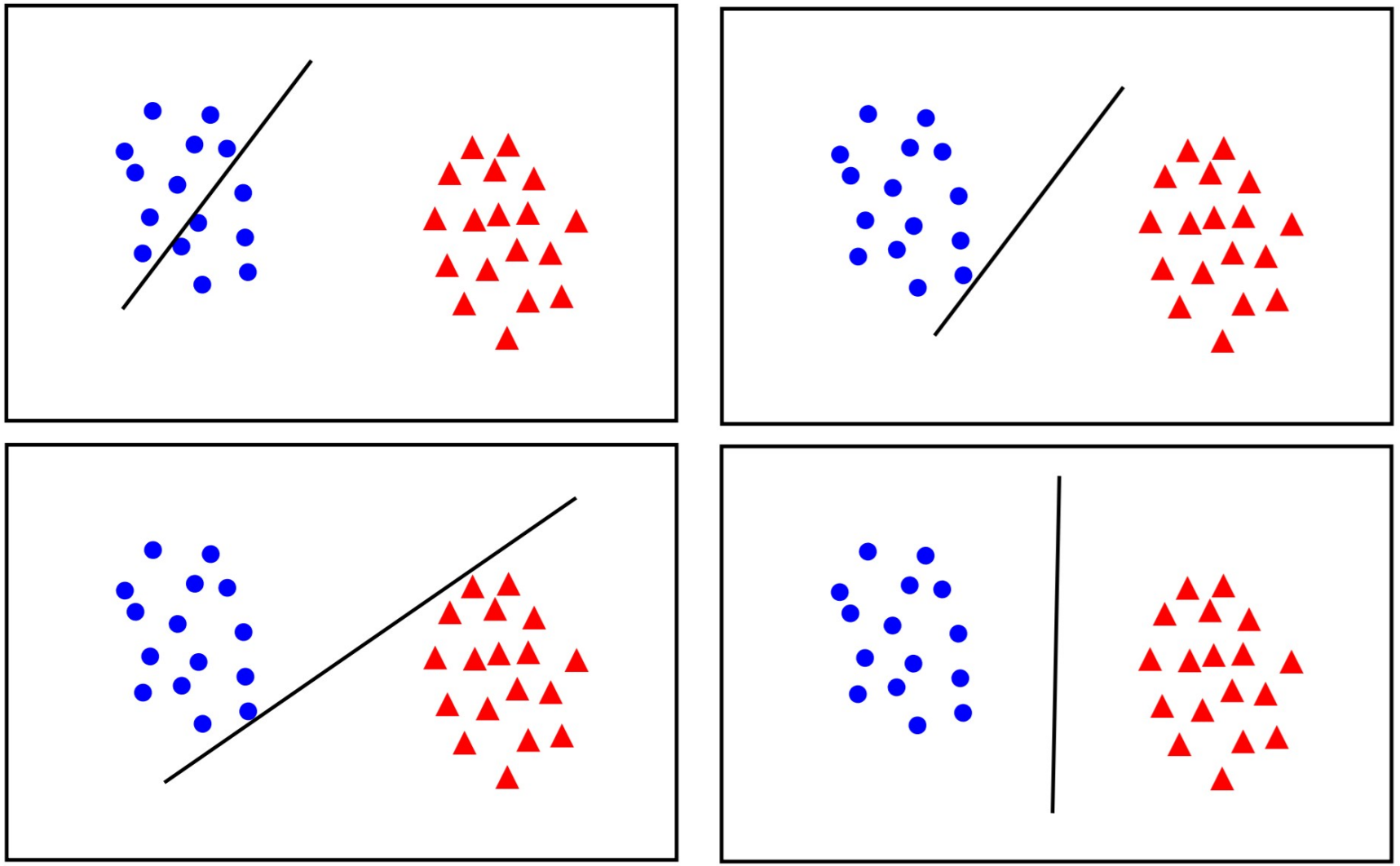


Reminder: perception algorithm

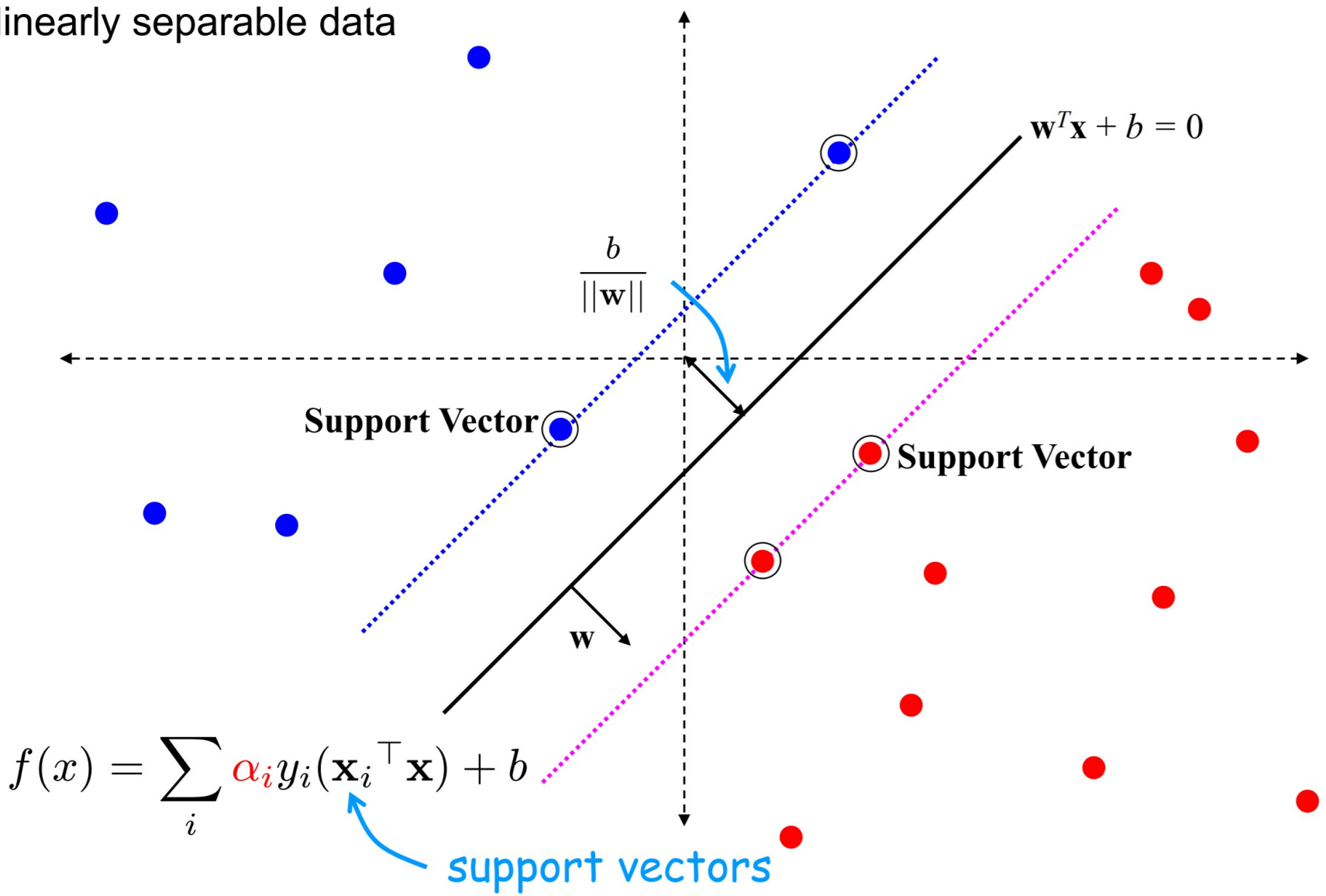
- Perception is guaranteed to converge as long as the data is linearly separable



Perception algorithm – where to draw the line?



Support **V**ector **M**achine (SVM)

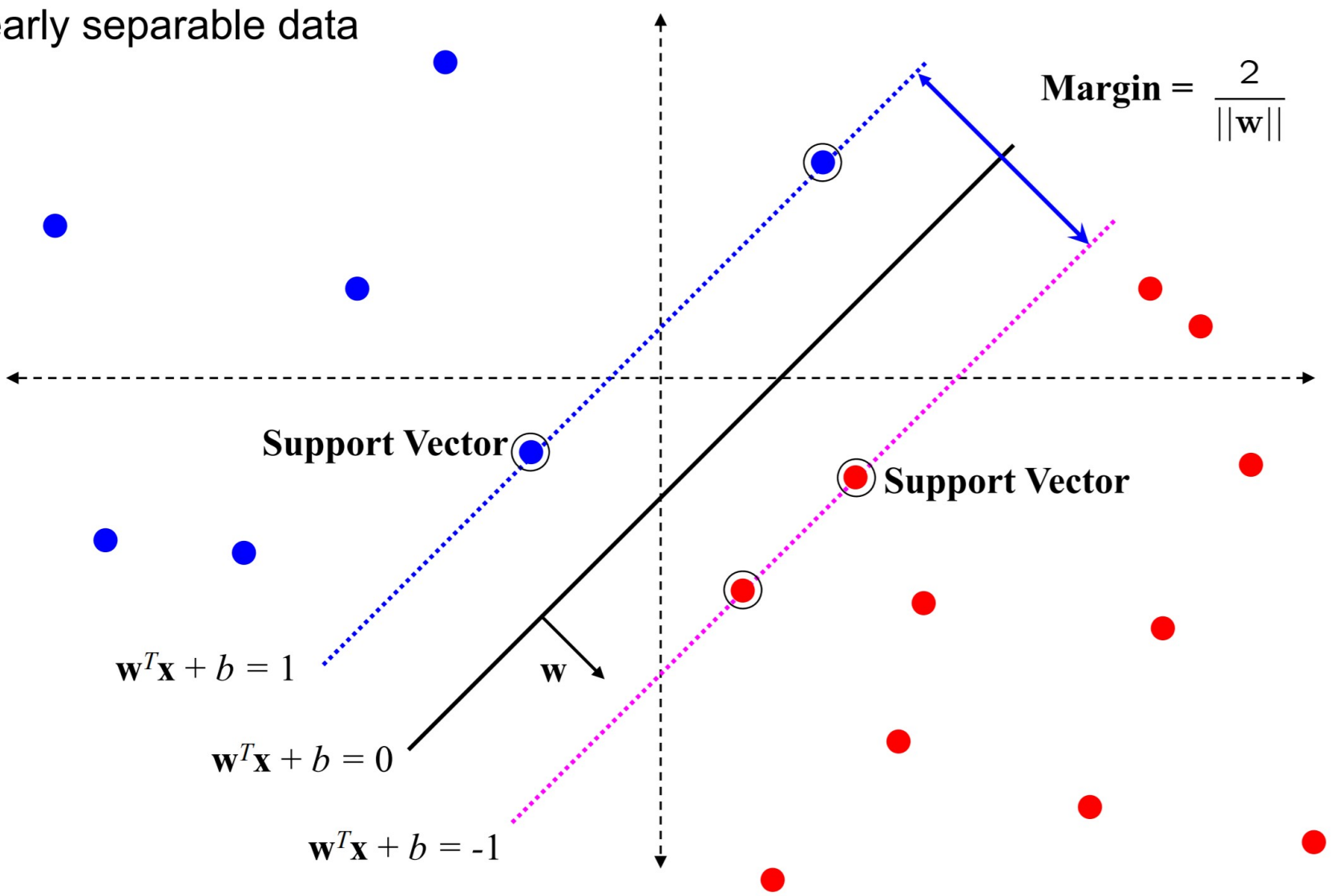


SVM

- Since $w^T x + b = 0$ and $c(w^T x + b) = 0$ define the same plane, we have the freedom to choose the normalization of w
- Choose normalization such that $w^T x + b = +1$ and $w^T x + b = -1$ for the positive and negative support vectors respectively
- Then the margin is given by:

$$\frac{\mathbf{w}}{||\mathbf{w}||} \cdot \left(\mathbf{x}_+ - \mathbf{x}_-\right) = \frac{\mathbf{w}^\top \left(\mathbf{x}_+ - \mathbf{x}_-\right)}{||\mathbf{w}||} = \frac{2}{||\mathbf{w}||}$$

SVM


linearly separable data
Margin = 2 / ||w||
Support Vector
Support Vector
w
$\mathbf{w}^T\mathbf{x} + b = 1$
$\mathbf{w}^T\mathbf{x} + b = 0$
$\mathbf{w}^T\mathbf{x} + b = -1$

SVM -- optimization

- Learning the SVM can be formulated as an optimization:

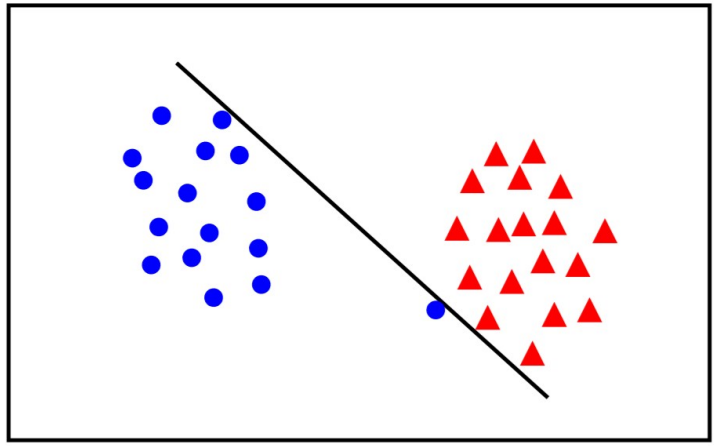
$$\max_{\mathbf{w}} \frac{2}{||\mathbf{w}||} \text{ subject to } \mathbf{w}^\top \mathbf{x}_i + b \begin{array}{ll} \geq 1 & \text{if } y_i = +1 \\ \leq -1 & \text{if } y_i = -1 \end{array} \text{ for } i = 1 \ldots N$$

- Or equivalently:

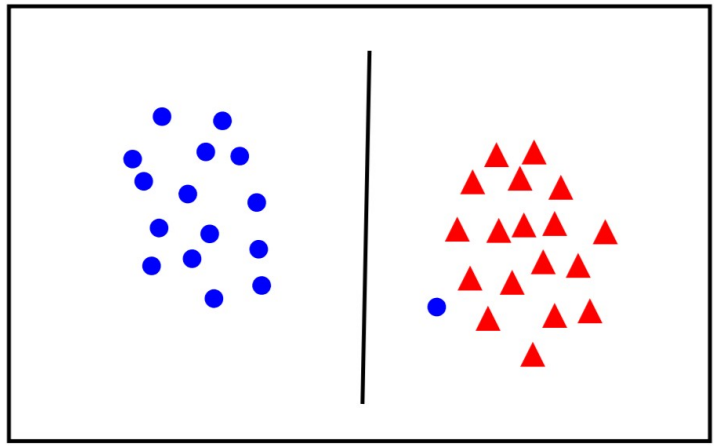
$$\min_{\mathbf{w}} ||\mathbf{w}||^2 \text{ subject to } y_i \left(\mathbf{w}^\top \mathbf{x}_i + b \right) \geq 1 \text{ for } i = 1 \ldots N$$

- This is a quadratic optimization problem subject to linear constraints and there is a unique minimum

Selecting the best w



• the points can be linearly separated but there is a very narrow margin



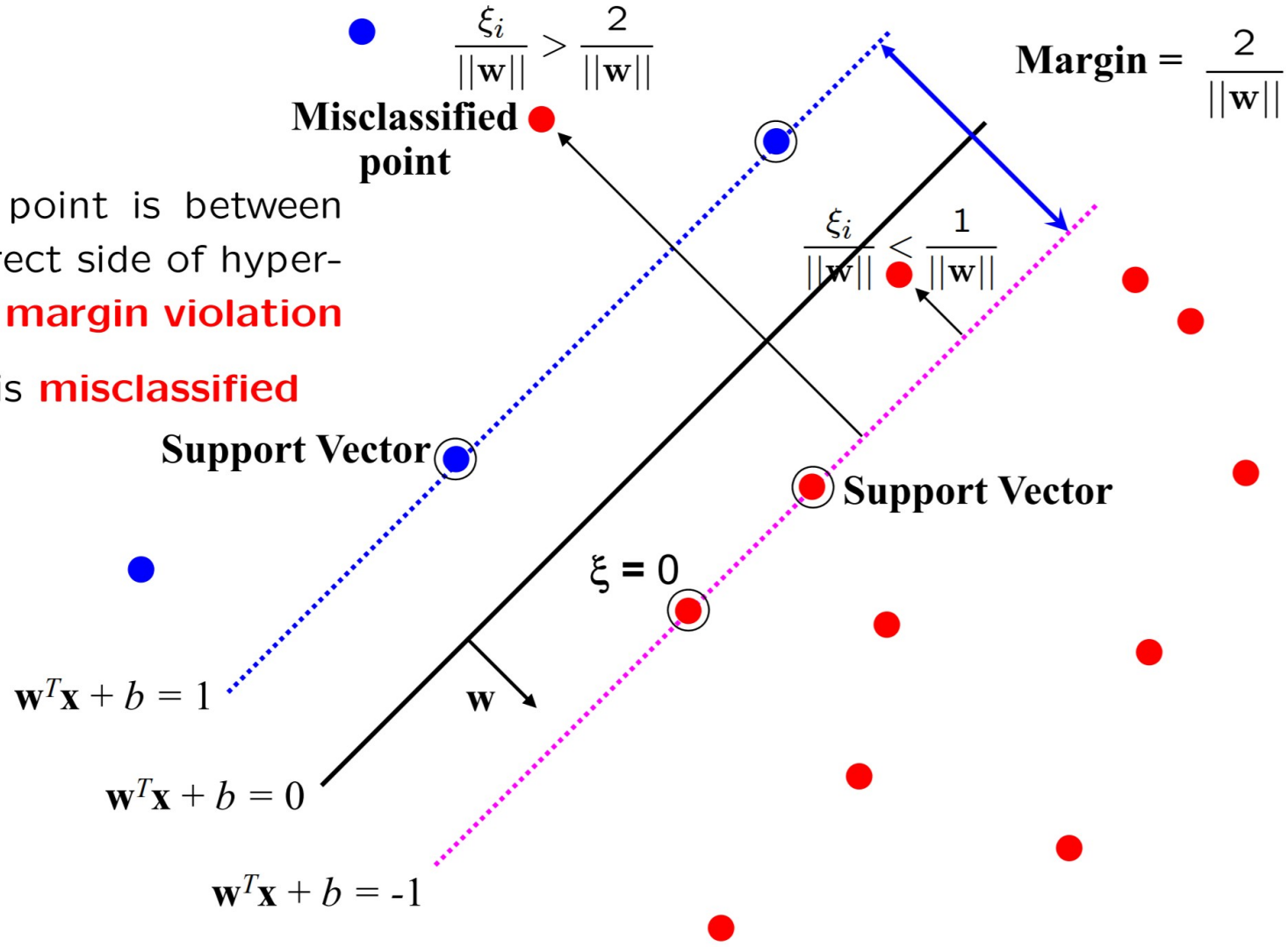
• but possibly the large margin solution is better, even though one constraint is violated

In general there is a trade off between the margin and the number of mistakes on the training data

“Slack” variables

$$\xi_i \geq 0$$

- for $0 < \xi \leq 1$ point is between margin and correct side of hyper-plane. This is a **margin violation**
- for $\xi > 1$ point is **misclassified**



“Soft” margin solution

The optimization problem becomes

$$\min_{\mathbf{w}\in\mathbb{R}^d,\xi_i\in\mathbb{R}^+} ||\mathbf{w}||^2+C\sum_i^N \xi_i$$

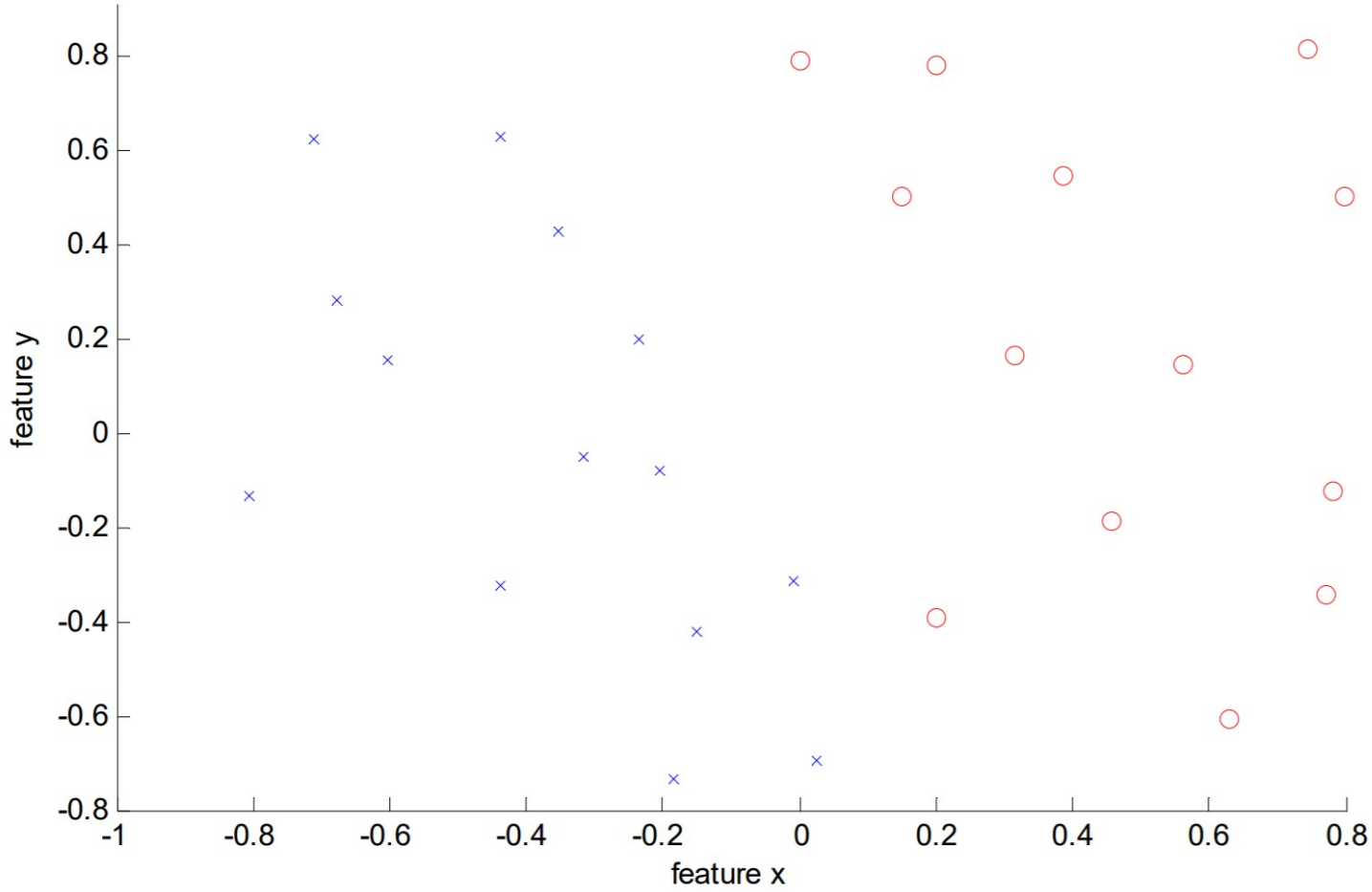
subject to

$$y_i\left(\mathbf{w}^\top\mathbf{x}_i+b\right)\geq 1-\xi_i \text{ for } i=1\ldots N$$

- Every constraint can be satisfied if ξ_i is sufficiently large
- C is a regularization parameter:
  - small C allows constraints to be easily ignored $\rightarrow$ large margin
  - large C makes constraints hard to ignore $\rightarrow$ narrow margin
  - $C=\infty$ enforces all constraints: hard margin
- This is still a quadratic optimization problem and there is a unique minimum. Note, there is only one parameter, C.

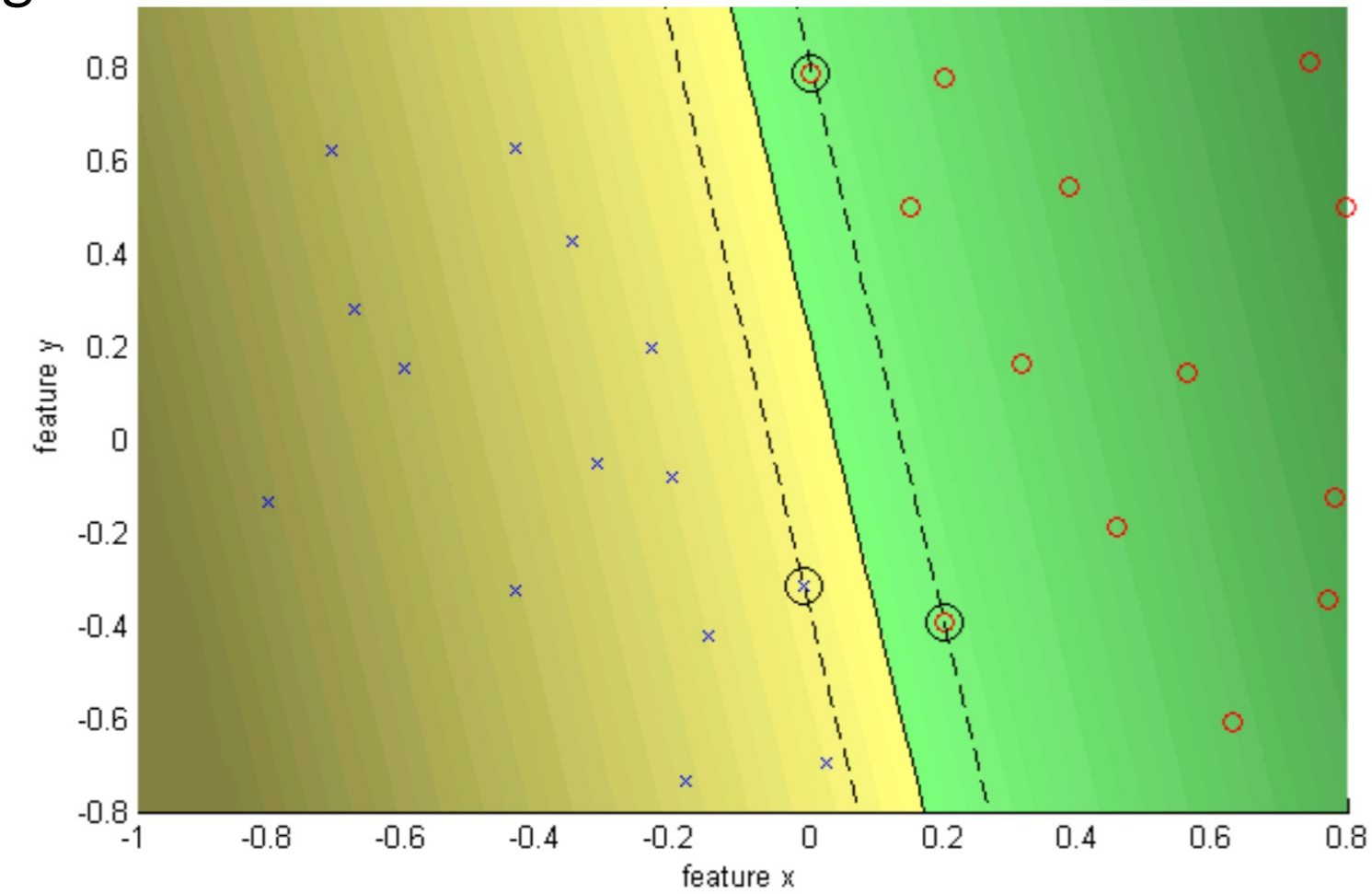
SVM

- Data is linearly separable, but only in a narrow margin



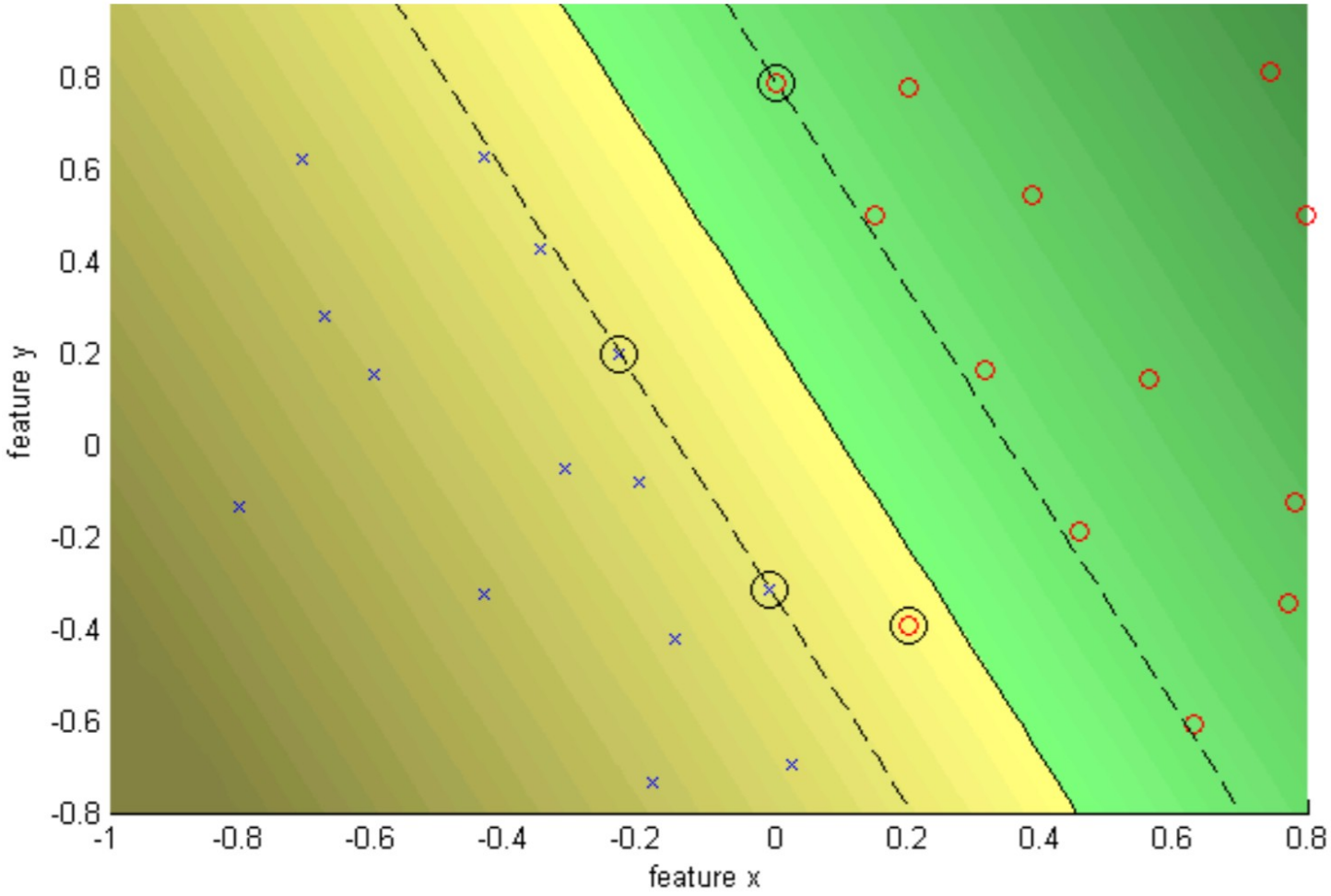
SVM – C = infinity

- Hard margin



SVM – C = 10

- Soft margin



Soft vs hard margin

- Also sometimes called “soft constraint” and “hard constraint”
- Refers to whether or not all of the training data is “correctly” classified by the classifier
  - If all “dots” sort as they should, it’s a hard constraint
  - If there are “dots” in the inappropriate region, it’s a soft constraint
  - The presence of “slack” makes a soft margin

Application: pedestrian detection in computer vision

- Objective: detect (localize) standing humans in an image (face detection with a sliding window classifier)
  - Reduces object detection to binary classification
  - Does an image window contain a person or not?

Training data and features

Positive data:



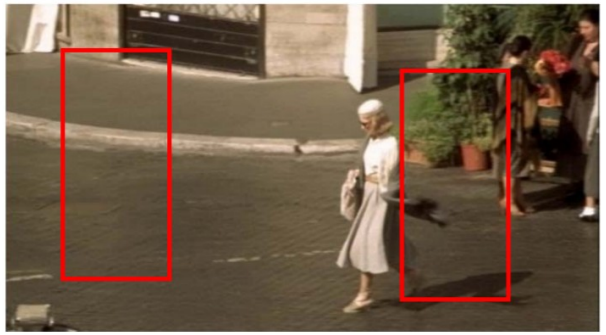






Negative data:





SVM “big picture”

- We want to maximize the margin over the training data – that is the line should be maximally far away from the most number of data points

- This means that you essentially want to minimize w
  - This optimum w is often termed w^*

SVM “big picture”

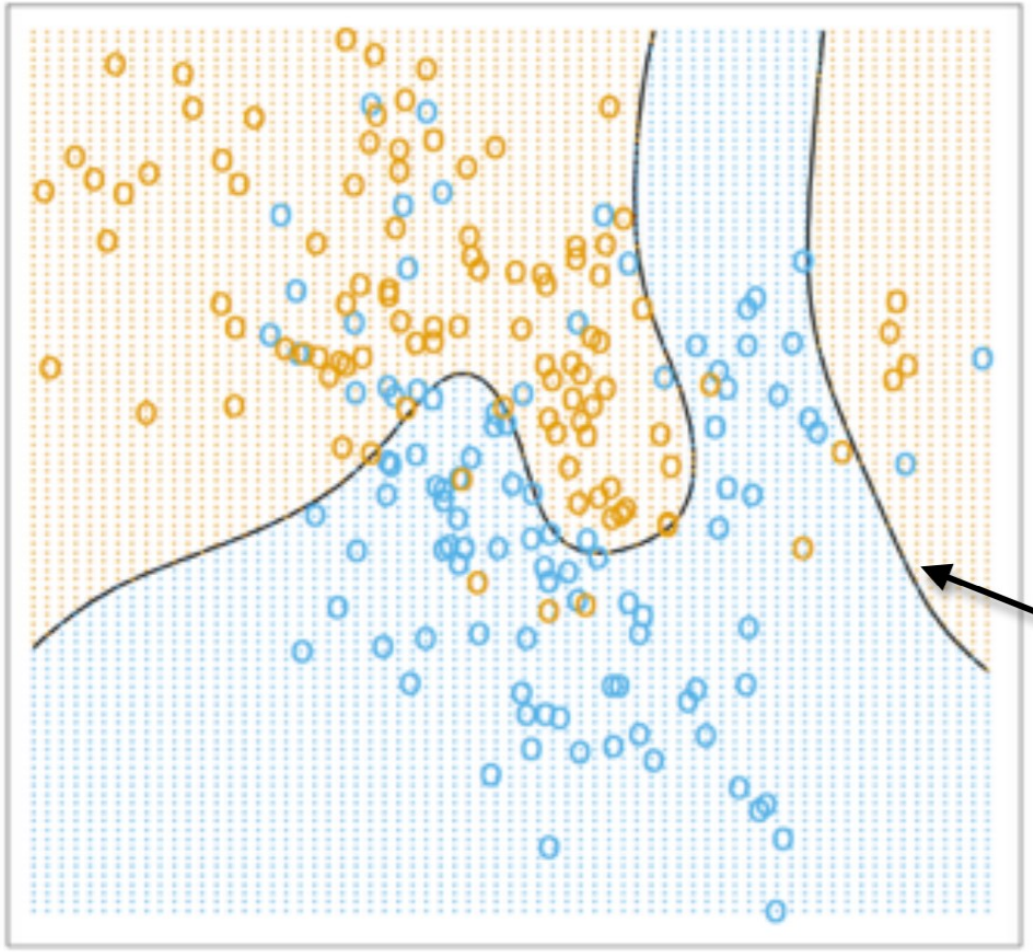
- Variables:
  - Cost parameter, C
    - Too large, may have overfitting
    - Too small, may have underfitting
  - Epsilon, ε, determines how far away from the margin the support vectors are
    - You want enough support vectors to get good results, not so many that it takes long…

Lecture 12: Local Methods & Kernels

Nearest neighbor methods

- Imagine that you have points on an axis, and you want to categorize a new point (a piece of test data)
  - One strategy could just be: what data point are you closest to? Well, then, you have the same category as that point

Classification data: Bayes classifier



Training data:

True label: +1

True label: -1

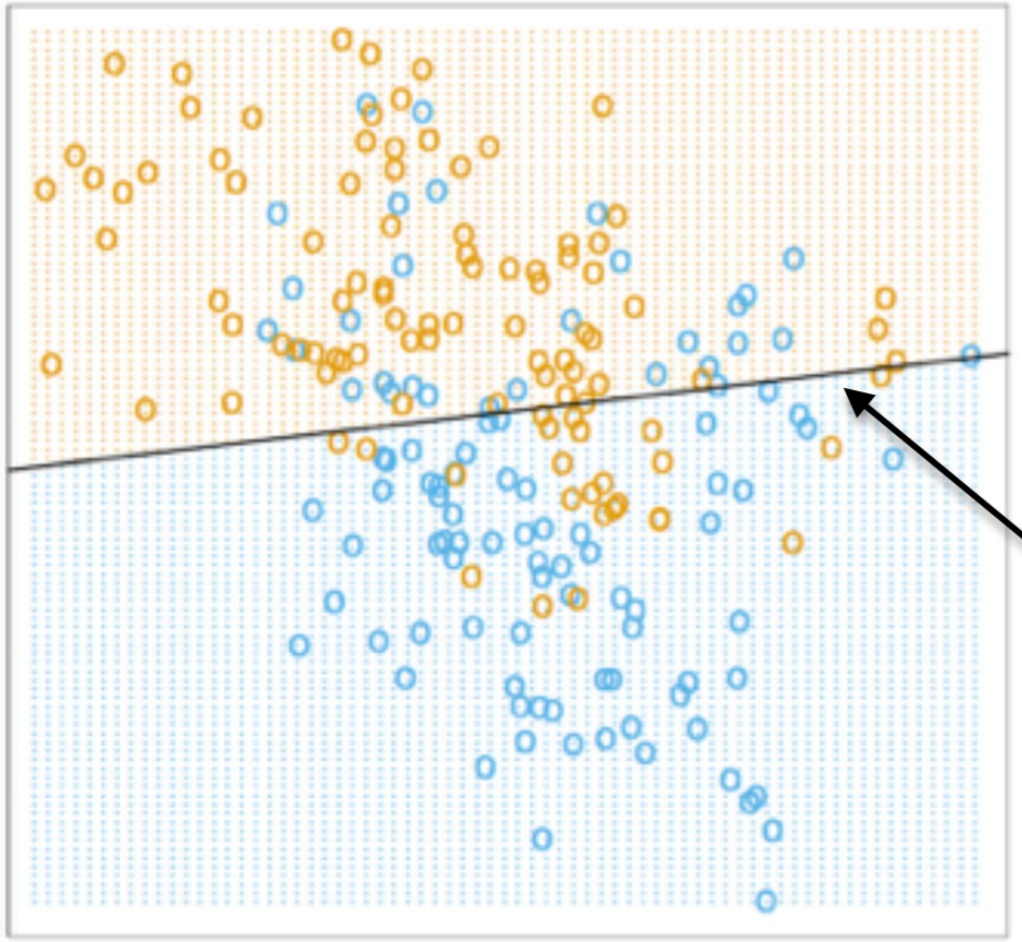
Optimal “Bayes” classifier:

$$\mathbb{P}(Y = 1|X = x) = \frac{1}{2}$$

Predicted label: +1

Predicted label: -1

Classification data: linear decision boundary



Training data:

True label: +1

True label: -1

Learned:

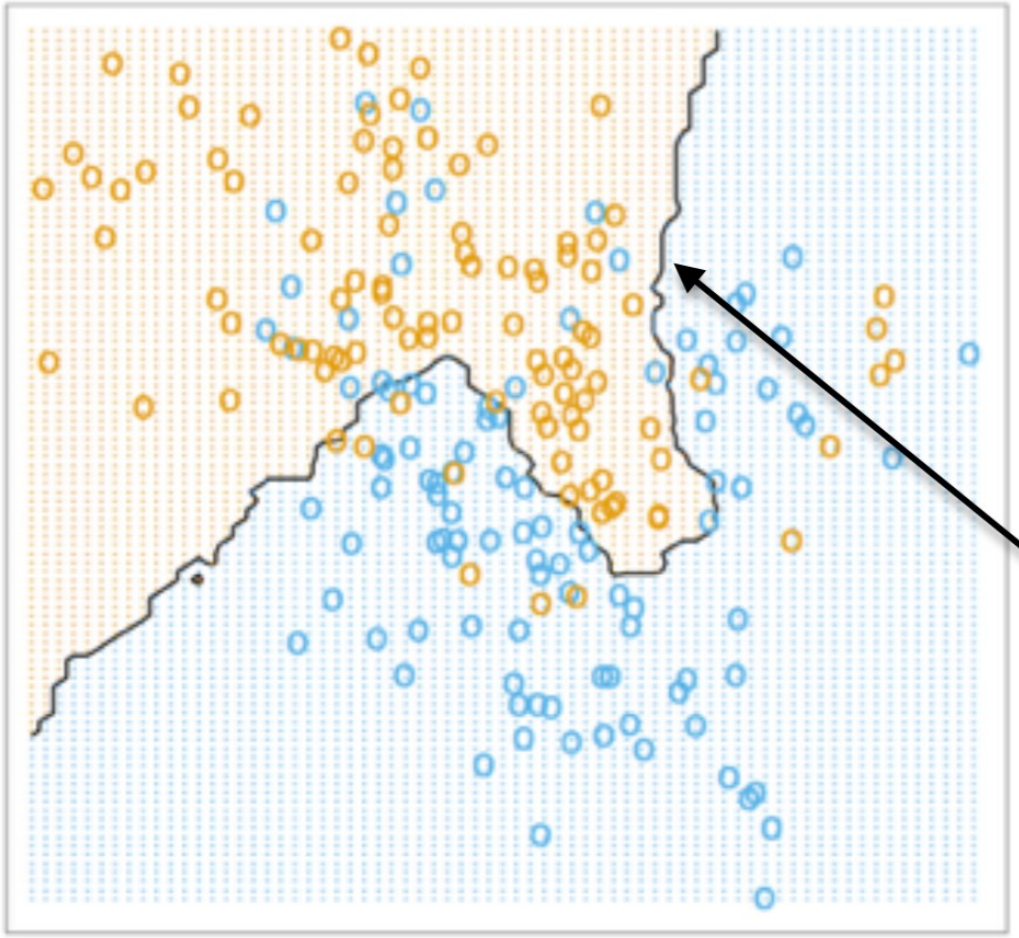
Linear Decision boundary

$$x^T w + b = 0$$

Predicted label: +1

Predicted label: -1

15 nearest neighbor boundary



Training data:

True label: +1

True label: -1

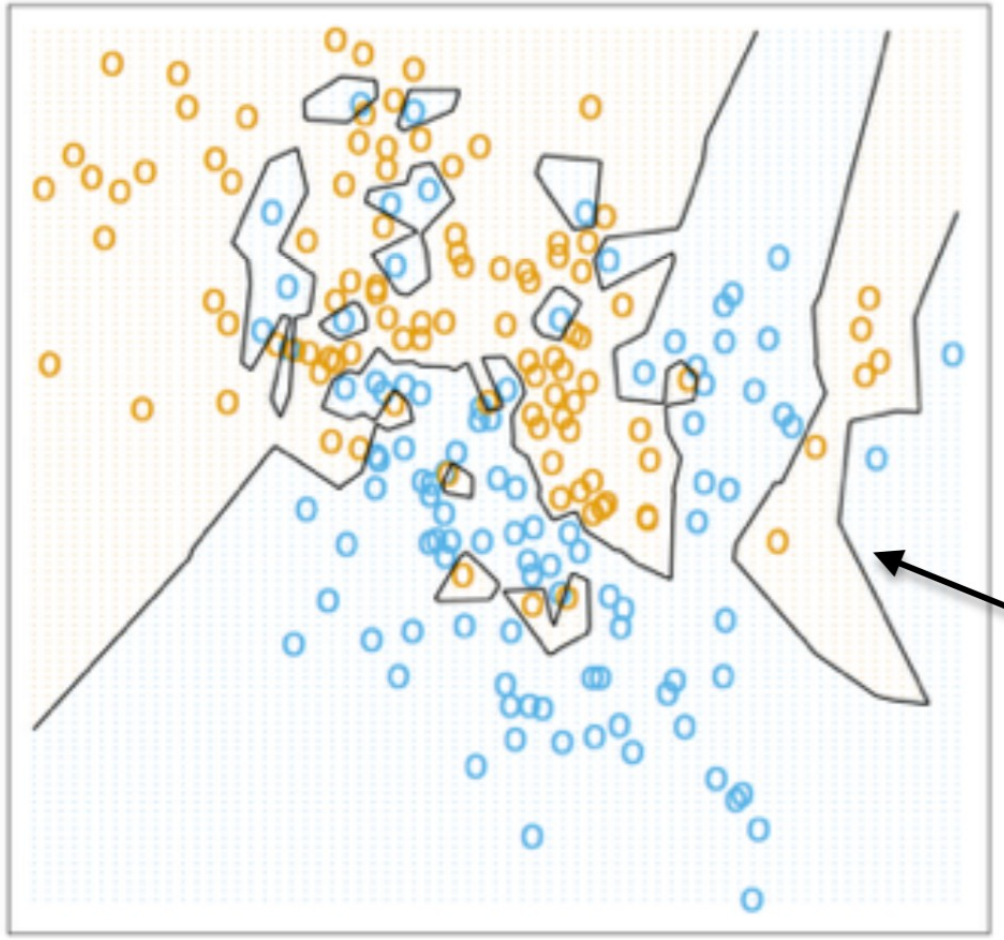
Learned:

15 nearest neighbor decision boundary (majority vote)

Predicted label: +1

Predicted label: -1

1 nearest neighbor boundary



Training data:

True label: +1

True label: -1

Learned:

1 nearest neighbor decision boundary (majority vote)

Predicted label: +1

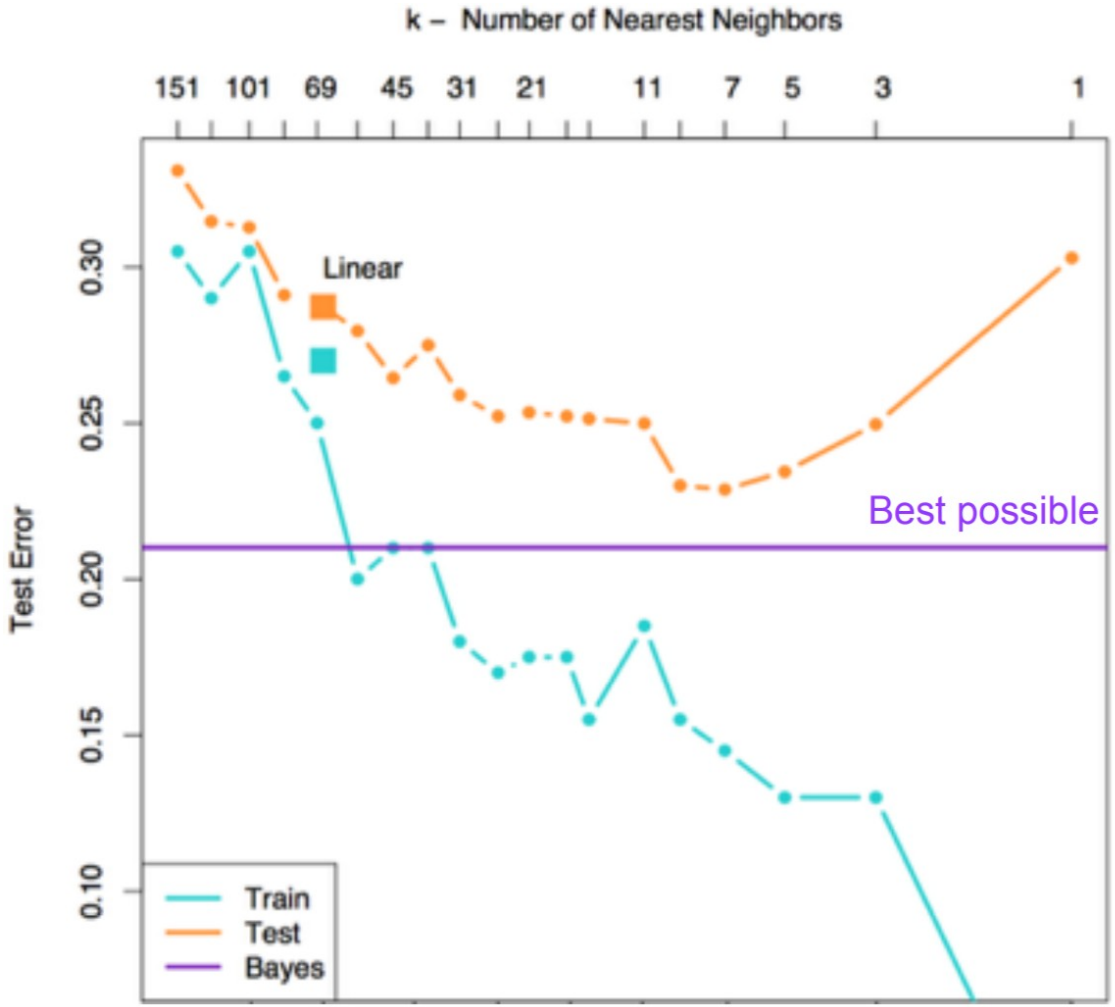
Predicted label: -1

Nearest neighbor

- Nearest neighbor has tradeoffs (pros and cons)

- And a decision point: we looked at 15 vs 1 nearest neighbor—what's the difference and what's the "best" number to use (if such a thing exists)

k-nearest neighbor & error



Bias-Variance tradeoff

As k->infinity?

Bias:

Variance:

As k->1?

Bias:

Variance:

k-nearest neighbor & error

- As n->∞
  - Your data becomes *dense*
  - Your error decreases
  - Your error remains roughly twice the Bayes error

- Regardless of your n, method performance largely depends on how well your training data represents the overall data
  - Which is generally true of machine learning, but especially true here
  - Draw an example of when this is especially true

Nearest neighbor summary

- Very simple to explain and implement
- No training! *Can* be computationally demanding depending on dataset size and k
- Can use any form of distance (not just Euclidean, as we did)
- Smoothing with kernels can improve performance

- With a lot of data, “local methods” have strong, simple theoretical guarantees. With not a lot of data, neighbors aren’t local and methods suffer for it

Kernel methods

- We've seen them before (SVM) and we'll see them again (PCA)
- Motivation: as the number of features increases, computational difficulty of learning can skyrocket
- Problem in machine learning: Represent (often high-dimensional) data
- Problem, re-framed: Calculate a similarity measure in the feature space, then apply machine learning algorithm to this similarity measure

What exactly is a kernel?

- A function that takes as input vectors in the original space and returns vectors in the feature space. This function is called a kernel function.

- Can be thought of as modeling the difference between each individual pair of data and then modeling those differences rather than trying to learn how every feature interacts across all of the data

Common kernels (you don't need to know these)

- Polynomials of degree exactly d

$$K(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v})^d$$

- Polynomials of degree up to d

$$K(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v} + 1)^d$$

- Gaussian (squared exponential) kernel

$$K(\mathbf{u}, \mathbf{v}) = \exp\left(-\frac{||\mathbf{u} - \mathbf{v}||_2^2}{2\sigma^2}\right)$$

- Sigmoid

$$K(\mathbf{u}, \mathbf{v}) = \tanh(\eta \mathbf{u} \cdot \mathbf{v} + \nu)$$

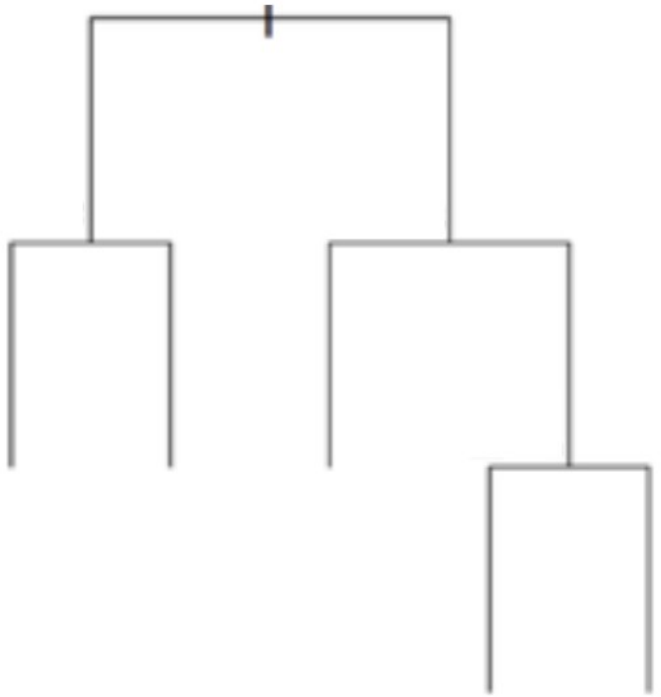
Lecture 13: Trees

Trees vs trees

- “Trees” are common
  - You have likely heard about trees as the things growing outside
  - You may have heard about decision trees
  - You have probably not heard of the 2014 Slovenian film
  - You have also likely heard about computer science trees as a data structure
    - The trees that we will discuss today are related to, *but quite distinct from*, these trees

Machine learning trees

- Represented in a similar way to how phylogenetic trees are represented in biology



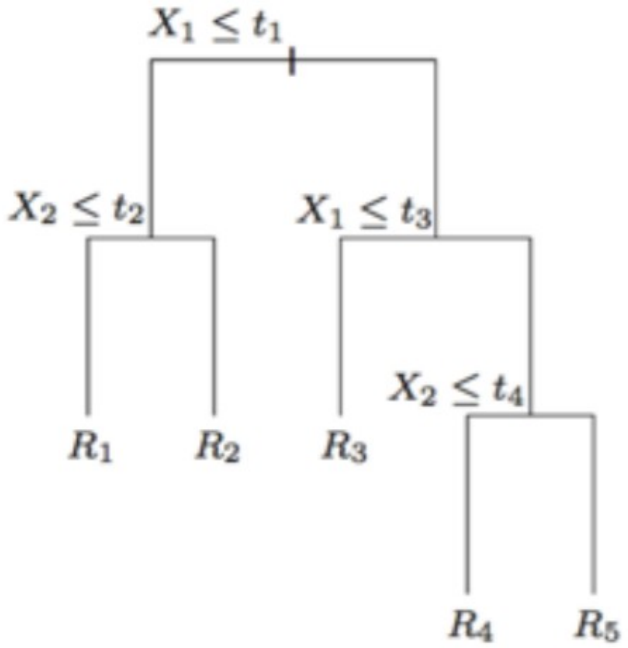
Machine learning trees

- Represented in a similar way to how phylogenetic trees are represented in biology
- Built as binary trees
- Data are stored at leaves with split points along axes

- How do you split? Greedily
- When do you stop? When points in leaves <= m

Learning *decision* trees

- Start with an empty decision tree
- Split on the next best feature/attribute
  - That is, split on $\text{argmax}_i(X_i) = \text{argmax}_i H(Y) - H(Y|X_i)$
- Recurse
- Prune



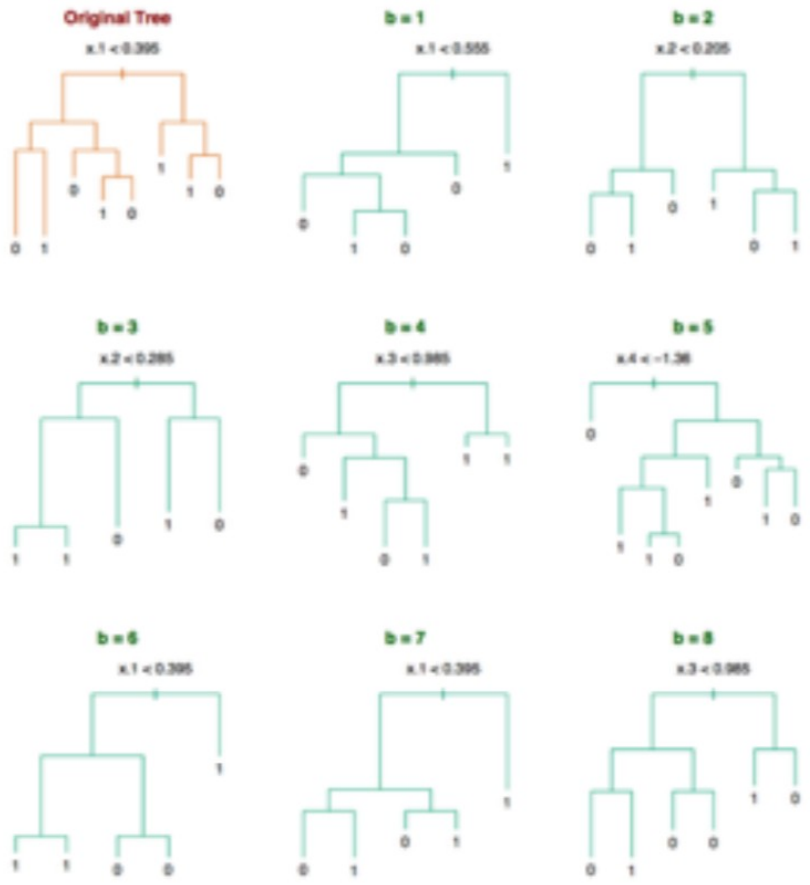
Trees

- Have low bias, high variance
- Deal quite well with categorical variables
- Intuitive & easily understandable
- Some theoretical guarantees (that we won’t go into)

Random forests

- Tree methods have low bias but high variance
- One way to reduce this variance is to generate a lot of related trees, assign weights to these trees, then average them
- This is “bagging” or “**b**ootstrap **agg**regat**ing**”

Random forests



Random forests

- To grow a random forest, continue until you reach the desired number of nodes:
  - Select m variables at random from among p variables
  - Pick the best variable (split point) among the m
  - Split the node into two daughter nodes
- Output the ensemble of trees

- This can be done for regression (mean tree) of classification (mode tree)

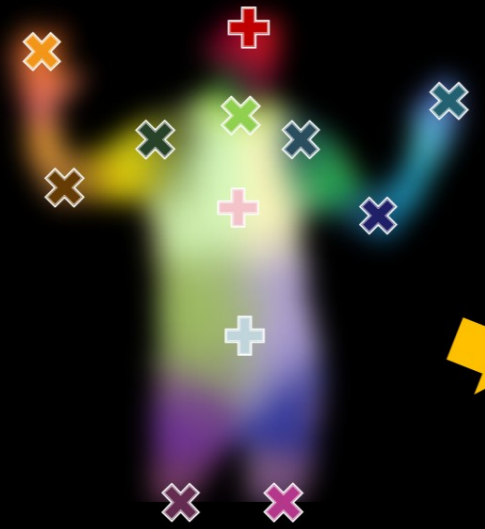
The Kinect pose estimation pipeline



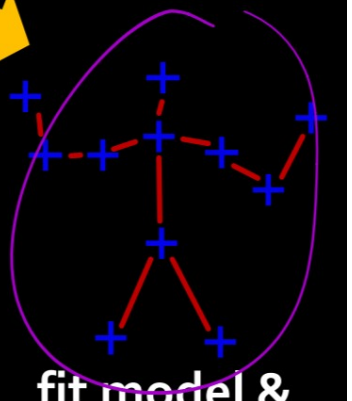
capture
depth image &
remove bg



infer
body parts
per pixel



cluster pixels to
hypothesize
body joint
positions

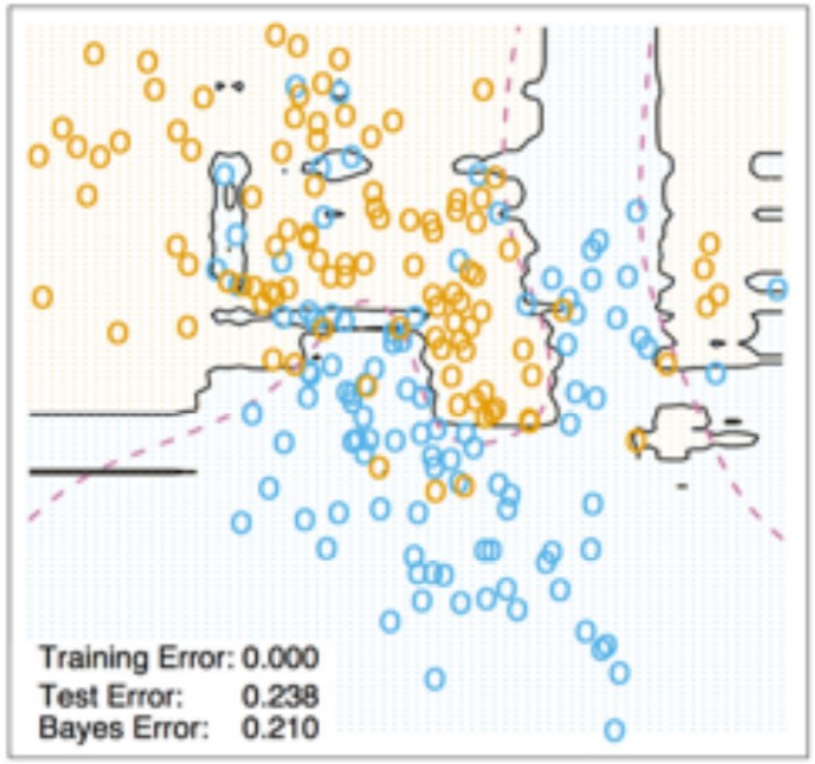


fit model &
track skeleton

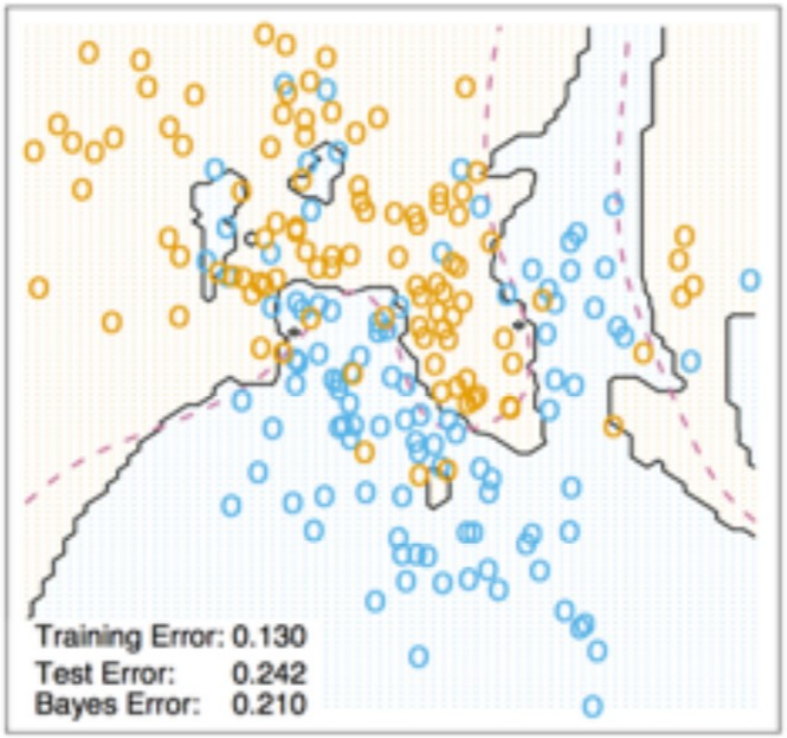
https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/CVPR20201120-20Final20Video.mp4

Random forest vs nearest neighbor

Random forrest



3 nearest neighbor



Random forests

- Have low bias, low variance (corrects decision trees' habit of overfitting)
- Deal with categorical variables well
- Not that intuitive or interpretable
- Some theoretical guarantees
- Can still overfit (although typically less likely than a single tree)

Ways to fix (or minimize) this: Bagging and Boosting next time

Lecture 14: Bagging & Boosting

Bootstrapping

- Given a set of data that are independent and identically distributed (i.i.d.) samples, D = {Z_1, Z_2,…Z_n}, we want to compute θ

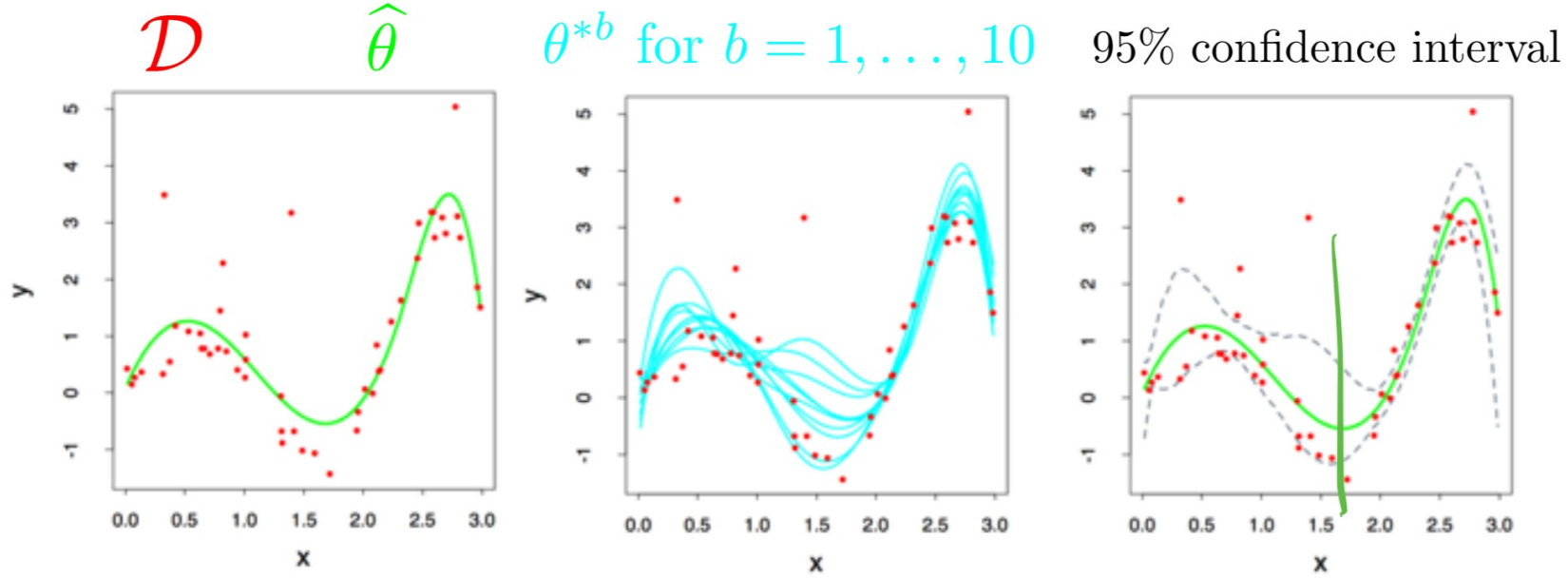
- For b=1, 2,…B, we define the *b*th bootstrapped dataset as drawing n samples ***with replacement*** from D

$$\mathcal{D} = \{z_1, \ldots, z_n\} \overset{i.i.d.}{\sim} F_Z \qquad \widehat{\theta} = t(\mathcal{D})$$

$$\mathcal{D}^{*b} = \{z_1^{*b}, \ldots, z_n^{*b}\} \overset{i.i.d.}{\sim} \widehat{F}_{Z,n} \quad \theta^{*b} = t(\mathcal{D}^{*b})$$

Bootstrap applications

- Common applications of the bootstrap:
  - Estimate parameters that escape simple analysis like the variance or median of an estimate
  - Confidence intervals
  - Estimates of error for a particular example:



Bootstrapping vs k-fold cross validation

- Bootstrapping is effectively randomly sampling *with replacement*, while in k-fold cross validation everyone gets a turn as part of a validation dataset

Bootstrap takeaways

- Advantages:
  - Bootstrap is very generally applicable. Build a confidence interval around anything
  - Very simple to use
  - Appears to give meaningful results even when the amount of data is very small
  - Very strong asymptotic theory (as num examples goes to infinity)

Bootstrap takeaways

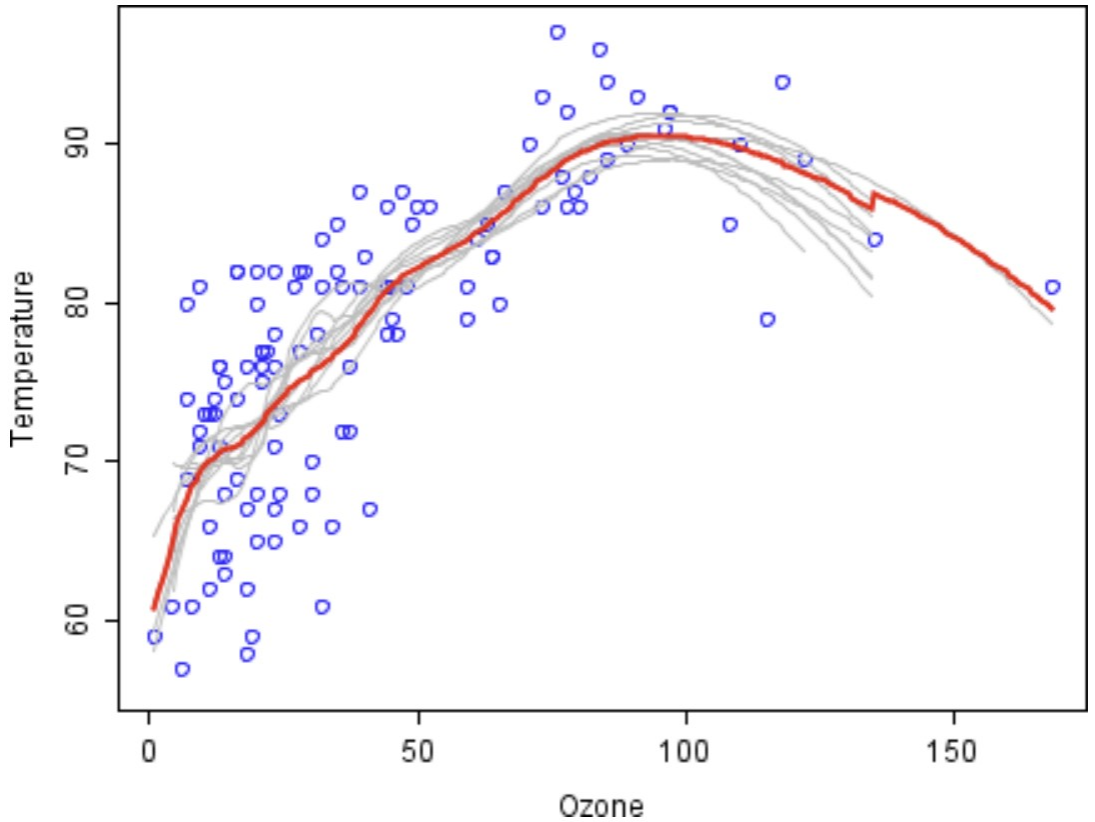
- Disadvantages:
  - Very few meaningful finite-sample guarantees
  - Potentially computationally intensive
  - Reliability relies on test statistic and rate of convergence of empirical CDF to true CDF, which is unknown
  - Poor performance on “extreme statistics” (e.g., the max)

Bagging

- Stands for “**b**ootstrap **agg**regat**ing**”
- Works with both classification and regression algorithms
- Reduces variances
- Helps avoid overfitting
- Very often applied to decision tree methods, but generalizable to any method
- Is a “model averaging” approach

Bagging

- Basically, the goal of bagging is to average over many bootstrapped learners to get a more representative fit



Bagging

- You’ve essentially already learned bagging!
  - Random forests are averages over several decision trees, so essentially an “aggregate” tree

- Any time that you can average your learned variables (choice of average is up to you!), you can use bagging

Boosting

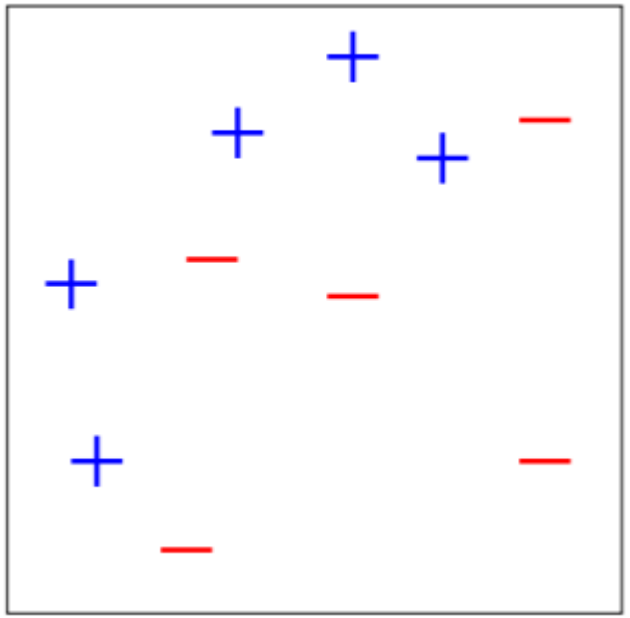
- Motivated by the question “can a set of weak learners create a single strong learner?”

- Similar to, but different from, bagging:
  - Bagging *averages* many low-bias, lightly dependent classifiers to **reduce variance** (bootstrapping is a low-bias technique for validation)
  - Boosting *learns* linear combinations of high-bias, highly dependent classifiers to **reduce error**

- We will first look at boosting visually

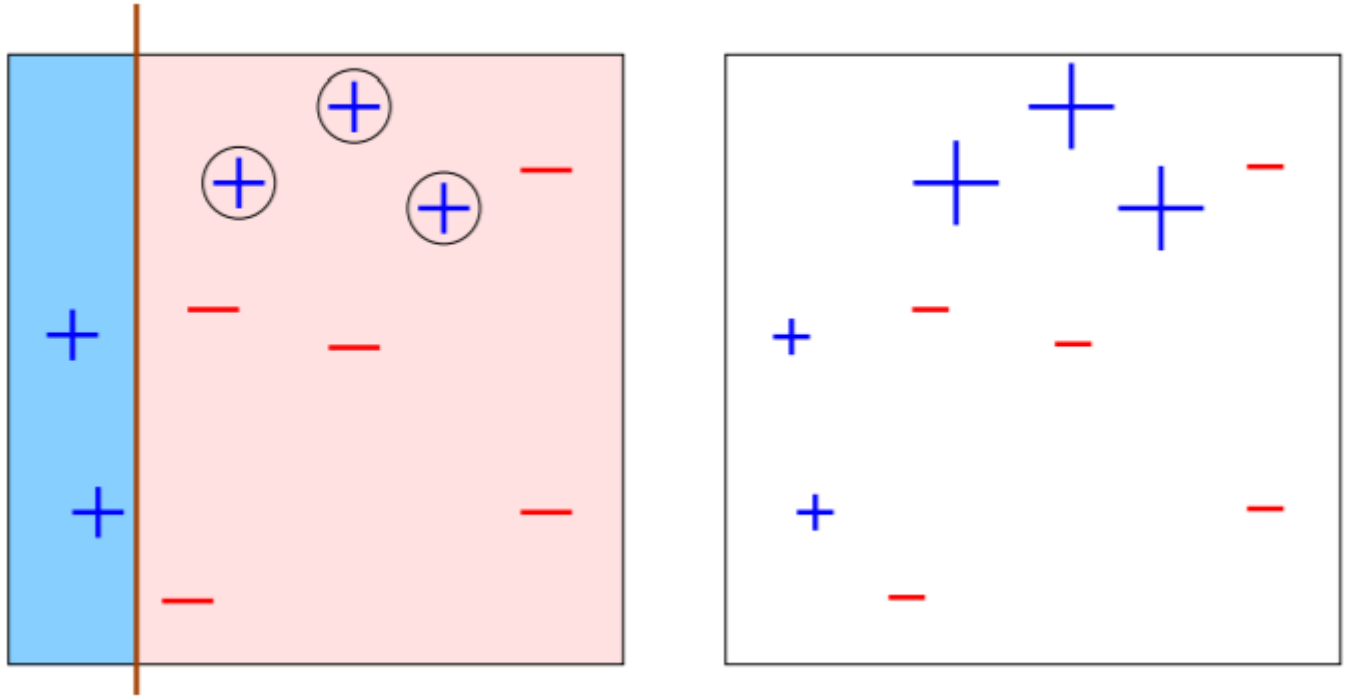
Boosting

- Start with 2 features, 10 data points:
  - Data clearly not linearly separable
  - All data points have equal weight



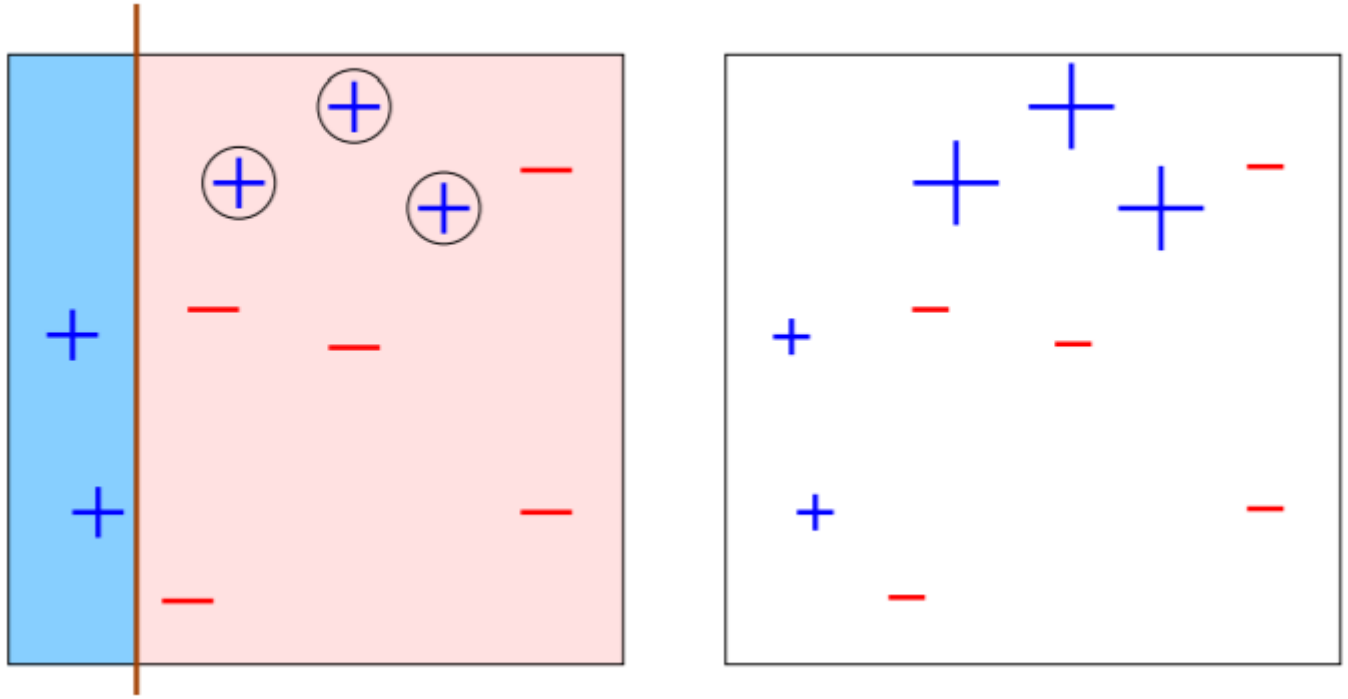
Boosting – step 1

- Make a “base classifier” – do the best you can to separate the data with horizontal or vertical lines (“decision stumps”)



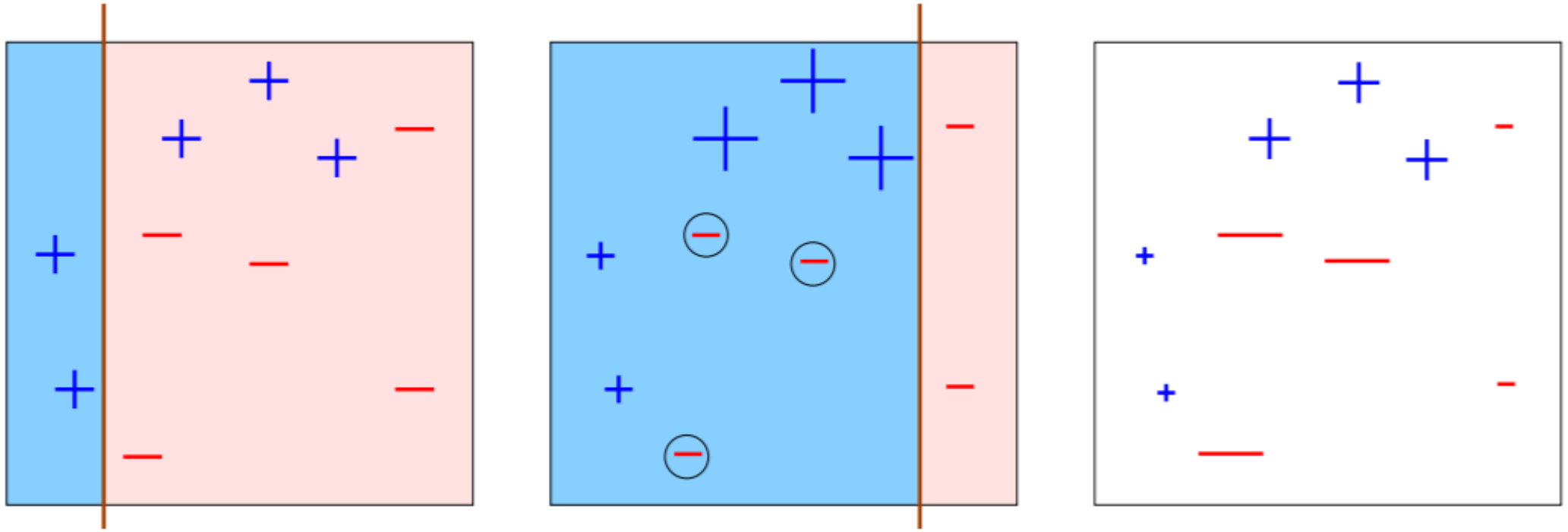
Boosting – step 1

- 3 misclassified points (error 30%) – those receive more/larger/bigger weight (shown here as being larger)



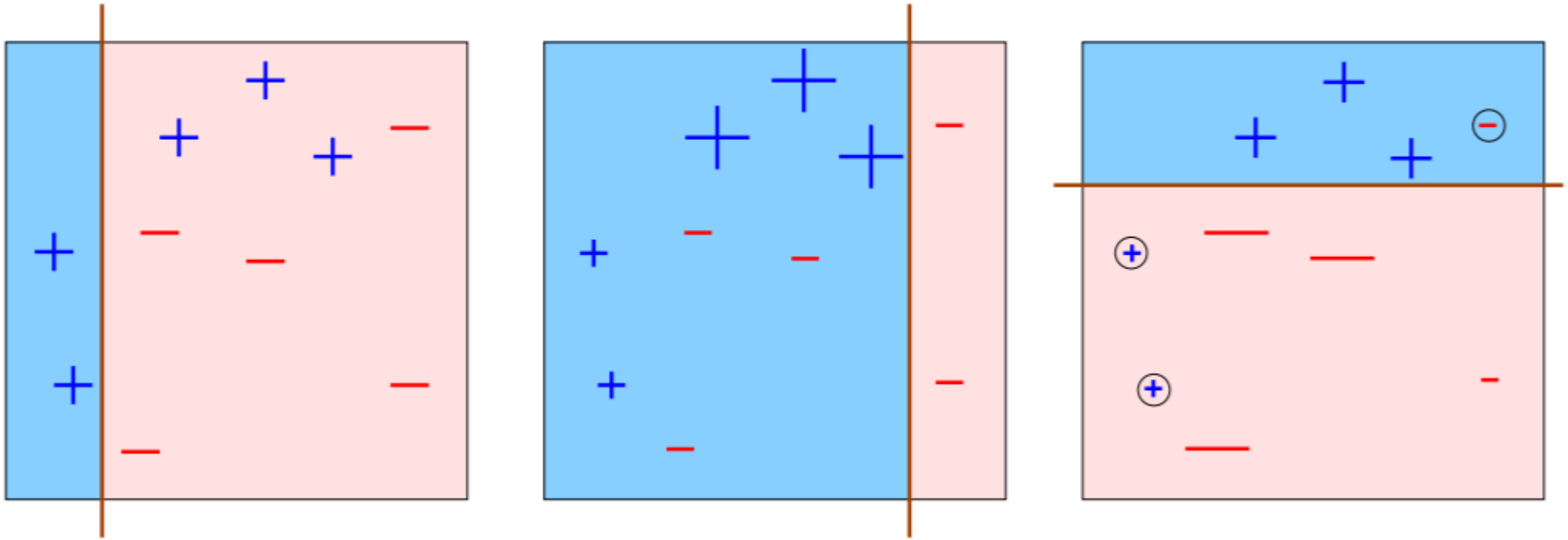
Boosting – step/round 2

- 3 misclassified points (*error is 0.21*) – those receive more/larger/bigger weight (shown here as being larger)



Boosting – step/round 3

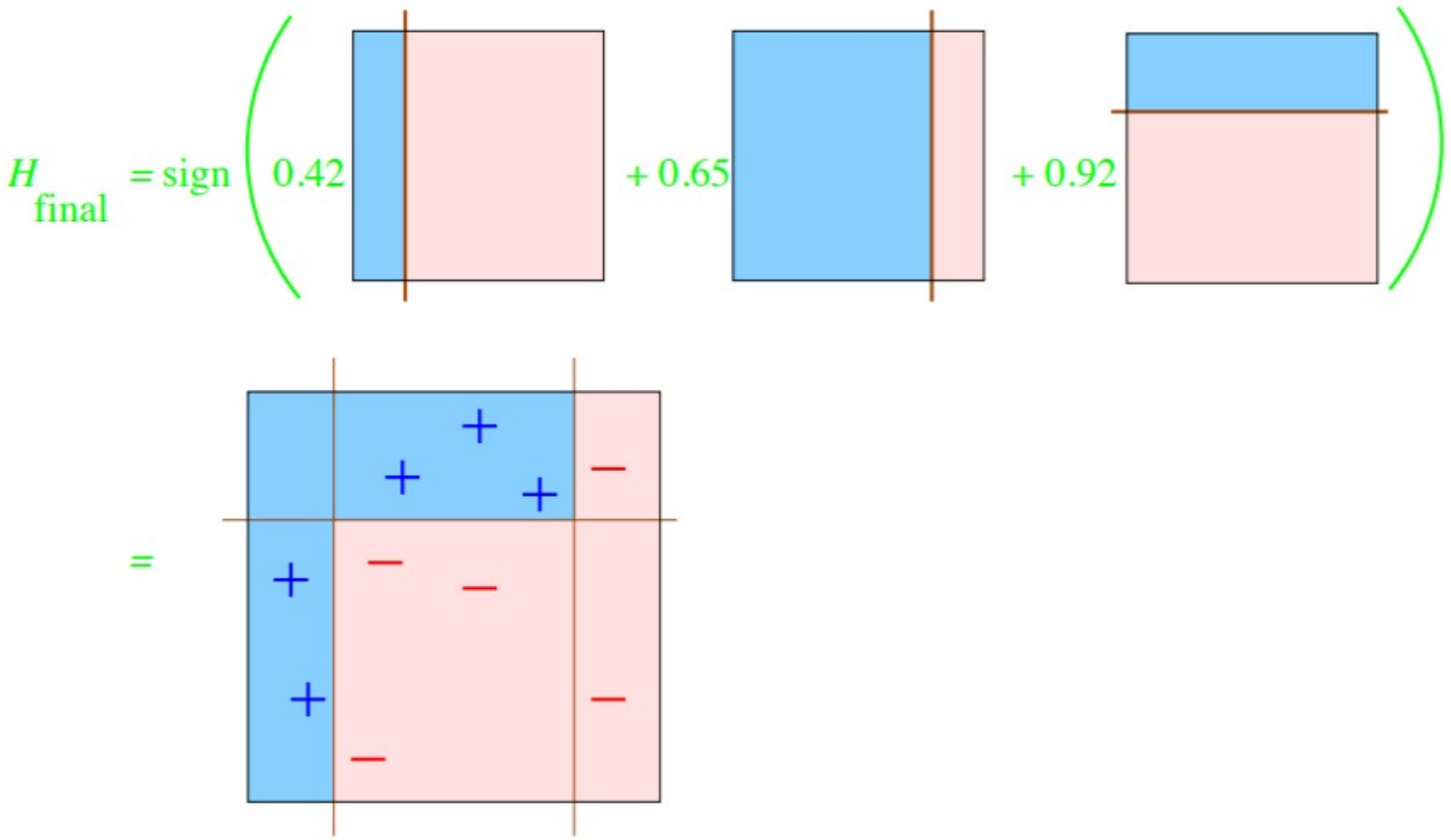
- 3 misclassified points (*error is 0.14*) – those receive more/larger/bigger weight (shown here as being larger)



Boosting

- Since you’ve successfully classified these points before, this round’s mistake will hopefully not majorly impact the overall prediction

Final classifier



How to decide where to draw lines?

- Can we simply enumerate all possible lines? Is that computationally feasible?
  - Yes! Can sort data in O(nlogn) time for each feature
  - Evaluate thresholds in O(n) time
  - Overall, quite fast! Makes it attractive in practice as well as in theory